LIM 4.0 Specs (EMS)

Jump to bottom

Maximilien Noal edited this page on Oct 9, 2023 · 2 revisions

LOTUS /INTEL /MICROSOFT

EXPANDED MEMORY SPECIFICATION

Version 4.0

300275-005

October, 1987

Copyright (c) 1987

Lotus Development Corporation
55 Cambridge Parkway
Cambridge, MA 02142

Intel Corporation
5200 NE Elam Young Parkway
Hillsboro, OR 97124

Microsoft Corporation
16011 NE 36TH Way
Box 97017
Redmond, WA 98073

This specification was jointly developed by Lotus Development Corporation, Intel Corporation, and Microsoft Corporation. Although it has been released into the public domain and is not confidential or proprietary, the specification is still the copyright and property of Lotus Development Corporation, Intel Corporation, and Microsoft Corporation.

DISCLAIMER OF WARRANTY

LOTUS DEVELOPMENT CORPORATION, INTEL CORPORATION, AND MICROSOFT CORPORATION EXCLUDE ANY AND ALL IMPLIED WARRANTIES, INCLUDING WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE. NEITHER LOTUS NOR INTEL NOR MICROSOFT MAKE ANY WARRANTY OF REPRESENTATION, EITHER EXPRESS OR IMPLIED, WITH RESPECT TO THIS SPECIFICATION, ITS QUALITY, PERFORMANCE, MERCHANTABILITY, OR FITNESS FOR A PARTICULAR PURPOSE. NEITHER LOTUS NOR INTEL NOR MICROSOFT SHALL HAVE ANY LIABILITY FOR SPECIAL, INCIDENTAL, OR CONSEQUENTIAL DAMAGES ARISING OUT OF OR RESULTING FROM THE USE OR MODIFICATION OF THIS SPECIFICATION.

This specification uses the following trademarks:

Intel is a trademark of Intel Corporation
Lotus is a trademark of Lotus Development Corporation
Microsoft is a trademark of Microsoft Corporation

1

CHAPTER 1

INTRODUCTION

Because even the maximum amount (640K bytes) of conventional memory isn't always enough for large application programs, Lotus Development Corporation, Intel Corporation, and Microsoft Corporation created the Lotus/Intel/Microsoft (LIM) Expanded Memory Specification.

The LIM Expanded Memory Specification defines the software interface between the Expanded Memory Manager (EMM) -- a device driver that controls and manages expanded memory -- and application programs that use expanded memory.

WHAT IS EXPANDED MEMORY?

Expanded memory is memory beyond DOS's 640K-byte limit. The LIM specification supports up to 32M bytes of expanded memory. Because the 8086, 8088, and 80286 (in real mode) microprocessors can physically address only 1M byte of memory, they access expanded memory through a window in their physical address range. The next section explains how this is done.

HOW EXPANDED MEMORY WORKS

Expanded memory is divided into segments called logical pages. These pages are typically 16K-bytes of memory. Your computer accesses logical pages through a physical block of memory called a page frame. The page frame contains multiple physical pages, pages that the microprocessor can address directly. Physical pages are also typically 16K bytes of memory.

This page frame serves as a window into expanded memory. Just as your computer screen is a window into a large spreadsheet, so the page frame is a window into expanded memory.

A logical page of expanded memory can be mapped into (made to appear in) any one of the physical pages in the page frame. Thus, a read or write to the physical page actually becomes a read or write to the associated logical page. One logical page can be mapped into the page frame for each physical page.

The page frame is located above 640K bytes. Normally, only video adapters, network cards, and similar devices exist between 640K and 1024K.

This specification also defines methods for operating systems and environments to access expanded memory through physical pages below 640K bytes. These methods are intended for operating system/environment developers only.

CHAPTER 2

WRITING PROGRAMS THAT USE EXPANDED MEMORY

This chapter describes what every program must do to use expanded memory and describes more advanced techniques of using expanded memory.

This chapter also lists programming guidelines you should follow when writing programs that use expanded memory and provides the listings of some example programs.

WHAT EVERY PROGRAM MUST DO

This section describes the steps every program must take to use expanded memory.

In order to use expanded memory, applications must perform these steps in the following order:

1. Determine if EMM is installed.

2. Determine if enough expanded memory pages exist for your application. (Function 3)

3. Allocate expanded memory pages. (Function 4 or 18)

4. Get the page frame base address. (Function 2)

5. Map in expanded memory pages. (Function 5 or 17)

6. Read/write/execute data in expanded memory, just as if it were conventional memory.

7. Return expanded memory pages to expanded memory pool before exiting. (Function 6 or 18)

Table 2-1 overviews the functions while Chapter 3 describes each of these functions in detail. Example programs at the end of this chapter illustrate using expanded memory.

TABLE 2-1. THE BASIC FUNCTIONS

Function Description

---

1 The Get Status function returns a status code indicating whether the memory manager hardware is working correctly.

2 The Get Page Frame Address function returns the address where the 64K-byte page frame is located.

3 The Get Unallocated Page Count function returns the number of unallocated pages (pages available to your program) and the total number of pages in expanded memory.

4 The Allocate Pages function allocates the number of pages requested and assigns a unique EMM handle to these pages.

5 The Map/Unmap Handle Page function maps a logical page to a specific physical page anywhere in the mappable regions of system memory.

6 The Deallocate Pages deallocates the logical pages currently allocated to an EMM handle.

7 The Get Version function returns the version number of the memory manager software.

ADVANCED PROGRAMMING

In addition to the basic functions, the Lotus/Intel/Microsoft Expanded Memory Specification provides several advanced functions which enhance the capabilities of software that uses expanded memory.

The following sections describe the advanced programming capabilities and list the advanced EMM functions.

Note.....

Before using the advanced functions, programs should first call Function 7 (Get Version) to determine whether the installed version of EMM supports these functions.

SAVING THE STATE OF MAPPING HARDWARE

Some software (such as interrupt service routines, device drivers, and resident software) must save the current state of the mapping hardware, switch mapping contexts, manipulate sections of expanded memory, and restore the original context of the memory mapping hardware. Use Functions 8 and 9 or 15 and 16 to save the state of the hardware.

RETRIEVING HANDLE AND PAGE COUNTS

Some utility programs need to keep track of how expanded memory is being used; use Functions 12 through 14 to do this.

MAPPING AND UNMAPPING MULTIPLE PAGES

Mapping multiple pages reduces the overhead an application must perform during mapping. Function 17 lets a program map (or unmap) multiple pages at one time.

In addition, you can map pages using segment addresses instead of physical pages. For example, if the page frame base address is set to D000, you can map to either physical page 0 or segment D000. Function 25 (Get Mappable Physical Address Array) returns a cross reference between all expanded memory physical pages and their corresponding segment values.

REALLOCATING PAGES

Reallocating pages (Function 18) lets applications dynamically allocate expanded memory pages without acquiring another handle or obtain a handle without allocating pages. Reallocating pages is an efficient means for applications to obtain and release expanded memory pages.

USING HANDLES AND ASSIGNING NAMES TO HANDLES

This specification lets you associate a name with a handle, so a family of applications can share information in expanded memory. For example, a software package consisting of a word processor, spreadsheet, and print spooler can share the same data among the different applications. The print spooler could use a handle name to reference data that either the spreadsheet or word processor put in expanded memory and could check for data in a particular handle name's expanded memory pages.

Use Function 20 (Set Handle Name subfunction) to assign a handle name to an EMM handle or Function 21 (Search for Named Handle subfunction) to obtain the EMM handle associated with the handle name. In addition, you can use Function 14 (Get Handle Pages) to determine the number of expanded memory pages allocated to an EMM handle.

USING HANDLE ATTRIBUTES

In addition to naming a handle, you can use Function 19 to associate an attribute (volatile or non-volatile) with a handle name. A non-volatile attribute enables expanded memory pages to preserve their data through a warmboot.

With a volatile attribute, the data is not preserved. The default attribute for handles is volatile.

Because using this function depends on the capabilities of the expanded memory hardware installed in the system, you should use the Get Attribute Capability subfunction before attempting to assign an attribute to a handle's pages.

ALTERING PAGE MAPS AND JUMPING/CALLING

You can use Functions 22 (Alter Page Map and Jump) and 23 (Alter Page Map and Call) to map a new set of values into the map registers and transfer program control to a specified address within expanded memory. These functions can be used to load and execute code in expanded memory. An application using this feature can significantly reduce the amount of conventional memory it requires. Programs can load needed modules into expanded memory at run time and use Functions 22 and 23 to transfer control to these modules.

Using expanded memory to store code can improve program execution in many ways. For example, sometimes programs need to be divided into small overlays because of conventional memory size limitations. Overlays targeted for expanded memory can be very large because LIM EMS 4.0 supports up to 32M bytes of expanded memory. This method of loading overlays improves overall system performance by conserving

conventional memory and eliminating conventional memory allocation errors.

MOVING OR EXCHANGING MEMORY REGIONS

Using Function 24 (Move/Exchange Memory Region), you can easily move and exchange data between conventional and expanded memory. Function 24 can manipulate up to one megabyte of data with one function call. Although applications can perform this operation without this function, having the expanded memory manager do it reduces the amount of overhead for the application. In addition, this function checks for overlapping regions and performs all the necessary mapping, preserving the mapping context from before the exchange/move call.

GETTING THE AMOUNT OF MAPPABLE MEMORY

Function 25 enables applications to determine the total amount of mappable memory the hardware/system currently supports. Not all expanded memory boards supply the same number of physical pages (map registers).

The Get Mappable Physical Address Array Entries subfunction returns the total number of physical pages the expanded memory hardware/system is capable of supporting. The Get Mappable Physical Address Array subfunction returns a cross reference between physical page numbers and the actual segment address for each of the physical pages.

OPERATING SYSTEM FUNCTIONS

In addition to the functions for application programs, this specification defines functions for operating systems/environments. These functions can be disabled at any time by the operating system/environment, so programs should not depend on their presence. Applications that avoid this warning and use these functions run a great risk of being incompatible with other programs, including the operating system.

TABLE 2-2. THE ADVANCED FUNCTIONS

Function	Description
8	The Save Page Map saves the contents of the page mapping registers from all expanded memory boards in an internal save area.
9	The Restore Page Map function restores (from an internal save area) the page mapping register contents on the expanded memory boards for a particular EMM handle.
10	Reserved.
11	Reserved.
12	The Get Handle Count function returns the number of open EMM handles in the system.

13	The Get Handle Pages function returns the number of pages allocated to a specific EMM handle.
14	The Get All Handle Pages function returns an array of the active EMM handles and the number of pages allocated to each one.
15	The Get/Set Page Map subfunction saves or restores the mapping context for all mappable memory regions (conventional and expanded) in a destination array which the application supplies.
16	The Get/Set Partial Page Map subfunction provides a mechanism for saving a partial mapping context for specific mappable memory regions in a system.
17	The Map/Unmap Multiple Handle Pages function can, in a single invocation, map (or unmap) logical pages into as many physical pages as the system supports.

18 The Reallocate Pages function can increase or decrease the amount of expanded memory allocated to a handle.

19 The Get/Set Handle Attribute function allows an application program to determine and set the attribute associated with a handle.

20 The Get/Set Handle Name function gets the eight character name currently assigned to a handle and can assign an eight character name to a handle.

21 The Get Handle Directory function returns information about active handles and the names assigned to each.

22 The Alter Page Map & Jump function alters the memory mapping context and transfers control to the specified address.

23 The Alter Page Map & Call function alters the specified mapping context and transfers control to the specified address. A return can then restore the context and return control to the caller.

24 The Move/Exchange Memory Region function copies or exchanges a region of memory from conventional to conventional memory, conventional to expanded memory, expanded to conventional memory, or expanded to expanded memory.

25 The Get Mappable Physical Address Array function returns an array containing the segment address and physical page number for each mappable physical page in a system.

26 The Get Expanded Memory Hardware Information function returns an array containing the hardware capabilities of the expanded memory system.

27 The Allocate Standard/Raw Pages function allocates the number of standard or non-standard size pages that the operating system requests and assigns a unique EMM handle to these pages.

28 The Alternate Map Register Set function enables an application to simulate alternate sets of hardware mapping registers.

29 The Prepare Expanded Memory Hardware for Warm Boot function prepares the expanded memory hardware for an "impending" warm boot.

30 The Enable/Disable OS/E function enables operating systems developers to enable and disable functions designed for operating system use.

8

PROGRAMMING GUIDELINES

The following section contains guidelines for programmers writing applications that use EMM.

- Do not put a program's stack in expanded memory.

- Do not replace interrupt 67h. This is the interrupt vector the EMM uses. Replacing interrupt 67h could result in disabling the Expanded Memory Manager.

- Do not map into conventional memory address space your application doesn't own. Applications that use the EMM to swap into conventional memory space, must first allocate this space from the operating system. If the operating system is not aware that a region of memory it manages is in use, it will think it is available. This could have disastrous results. EMM should not be used to 'allocate' conventional memory. DOS is the proper manager of conventional memory space. EMM should only be used to swap data in conventional memory space previously allocated from DOS.

- Applications that plan on using data aliasing in expanded memory must check for the presence of expanded memory hardware. Data aliasing occurs when mapping one logical page into two or more mappable segments. This makes one 16K-byte expanded memory page appear to be in more than one 16K-byte memory address space. Data aliasing is legal and sometimes useful for applications.

  Software-only expanded memory emulators cannot perform data aliasing. A simple way to distinguish software emulators from actual expanded memory hardware is to attempt data aliasing and check the results. For example, map one logical page into four physical pages. Write to physical page 0. Read physical pages 1-3 to see if the data is there as well. If the data appears in all four physical pages, then expanded memory hardware is installed in the system, and data aliasing is supported.

- Applications should always return expanded memory pages to the expanded memory manager upon termination. These pages will be made available for other applications. If unneeded pages are not returned to the expanded memory manager, the system could 'run out' of expanded memory pages or expanded memory handles.

- Terminate and stay resident programs (TSR's) should ALWAYS save the state of the map registers before changing them. Since TSR's may interrupt other programs which may be using expanded memory, they must not change the state of the page mapping registers without first saving them. Before exiting, TSR's must restore the state of the map registers.

  The following sections describe the three ways to save and restore the state of the map registers.

  1 Save Page Map and Restore Page Map (Functions 8 and 9). This is the simplest of the three methods. The EMM saves the map register contents in its own data structures -- the application does not need to provide extra storage

  locations for the mapping context. The last mapping context to be saved, under a particular handle, will be restored when a call to Restore Page Map is issued with the same handle. This method is limited to one mapping context for each handle and saves the context for only LIM standard 64K-byte page frames.

  2 Get/Set Page Map (Function 15). This method requires the application to allocate space for the storage array. The EMM saves the mapping context in an array whose address is passed to the EMM. When restoring the mapping context with this method, an application passes the address of an array which contains a previously stored mapping context.

  This method is preferable if an application needs to do more than one save before a restore. It provides a mechanism for switching between more than one mapping context.

3 Get/Set Partial Page Map (Function 16). This method provides a way for saving a partial mapping context. It should be used when the application does not need to save the context of all mappable memory. This function also requires that the storage array be part of the application's data.

o All functions using pointers to data structures must have those data structures in memory which will not be mapped out. Functions 22 and 23 (Alter Map and Call and Alter Map and Jump) are the only exceptions.

CHAPTER 3

EMM FUNCTIONS

This chapter provides you with a standardized set of expanded memory functions. Because they are standardized, you avoid potential compatibility problems with other expanded memory programs that also adhere to the memory manager specification. Programs that deal directly with the hardware or that don't adhere to this specification will be incompatible.

Table 3-1 presents a sequential list of the EMM functions. The remainder of this chapter provides detailed descriptions of each function.

TABLE 3-1. LIST OF EMM FUNCTIONS

Number	Function Name	Hex Value	Page
1	Get Status	40h	14
2	Get Page Frame Segment Address	41h	15
3	Get Unallocated Page Count	42h	16
4	Allocate Pages	43h	17
5	Map/Unmap Handle Page	44h	20
6	Deallocate Pages	45h	22
7	Get Version	46h	24

8 Save Page Map 47h 26

9 Restore Page Map 48h 28

10 Reserved 49h 30

11 Reserved 4Ah 31

12 Get Handle Count 4Bh 32

13 Get Handle Pages 4Ch 34

14 Get All Handle Pages 4Dh 36

15 Get Page Map 4E00h 38

Set Page Map 4E01h 40

Get & Set Page Map 4E02h 42

Get Size of Page Map Save Array 4E03h 44

11

16 Get Partial Page Map 4F00h 46

Set Partial Page Map 4F01h 48

Get Size of Partial Page Map Save Array 4F02h 50

17 Map/Unmap Multiple Handle Pages
(Physical page number mode) 5000h 52

Map/Unmap Multiple Handle Pages
(Segment address mode) 5001h 56

18 Reallocate Pages 51h 59

19 Get Handle Attribute 5200h 62

Set Handle Attribute 5201h 65

Get Handle Attribute Capability 5202h 67

20 Get Handle Name 5300h 68

Set Handle Name 5301h 70

21 Get Handle Directory 5400h 72

Search for Named Handle 5401h 74

Get Total Handles 5402h 76

22 Alter Page Map & Jump
(Physical page number mode) 5500h 77

Alter Page Map & Jump
(Segment address mode) 5501h 77

23 Alter Page Map & Call
(Physical page number mode) 5600h 80

Alter Page Map & Call
(Segment address mode) 5601h 80

Get Page Map Stack Space Size 5602h 84

24 Move Memory Region 5700h 86

Exchange Memory Region 5701h 91

25 Get Mappable Physical Address Array 5800h 96

Get Mappable Physical Address Array Entries 5801h 99

26 Get Hardware Configuration Array 5900h 101

Get Unallocated Raw Page Count 5901h 104

27 Allocate Standard Pages 5A00h 106

12

Allocate Raw Pages 5A01h 109

28 Get Alternate Map Register Set 5B00h 112

Set Alternate Map Register Set 5B01h 117

Get Alternate Map Save Array Size 5B02h 120

Allocate Alternate Map Register Set 5B03h 122

Deallocate Alternate Map Register Set 5B04h 124

Allocate DMA Register Set 5B05h 126

Enable DMA on Alternate Map Register Set 5B06h 128

Disable DMA on Alternate Map Register Set 5B07h 130

Deallocate DMA Register Set 5B08h 132

29 Prepare Expanded Memory Hardware for Warmboot 5Ch 134

30 Enable OS/E Function Set 5D00h 135

Disable OS/E Function Set 5D01h 138

Return OS/E Access Key 5D02h 140

13

FUNCTION 1 GET STATUS

PURPOSE

The Get Status function returns a status code indicating whether the memory manager is present and the hardware is working correctly.

CALLING PARAMETERS

AH = 40h
Contains the Get Status function.

REGISTERS MODIFIED

AX

STATUS

AH = 0 SUCCESSFUL
The manager is present in the system, and the hardware is working correctly.

AH = 80h NON-RECOVERABLE
The manager detected a malfunction in the memory manager software.

AH = 81h NON-RECOVERABLE
The manager detected a malfunction in the expanded memory hardware.

AH = 84h NON-RECOVERABLE
The function code passed to the memory manager is not defined.

EXAMPLE

```
MOV  AH, 40h                ; load function code
INT  67h                    ; call the memory manager
OR   AH, AH                 ; check EMM status
JNZ  emm_err_handler        ; jump to error handler on error
```


FUNCTION 2 GET PAGE FRAME ADDRESS

PURPOSE

The Get Page Frame Address function returns the segment address where the page frame is located.

CALLING PARAMETERS

AH = 41h
Contains the Get Page Frame function.

RESULTS

These results are valid only if the status returned is zero.

BX = page frame segment address
Contains the segment address of the page frame.

REGISTERS MODIFIED

AX, BX

STATUS

AH = 0 SUCCESSFUL
The manager has returned the page frame address in the BX register.

AH = 80h NON-RECOVERABLE
The manager detected a malfunction in the memory manager software.

AH = 81h NON-RECOVERABLE
The manager detected a malfunction in the expanded memory hardware.

AH = 84h NON-RECOVERABLE
The function code passed to the memory manager is not defined.

EXAMPLE

```
page_frame_segment              DW  ?

MOV  AH, 41h                    ; load function code
INT  67h                        ; call the memory manager
OR   AH, AH                     ; check EMM status
JNZ  emm_err_handler            ; jump to error handler on error
MOV  page_frame_segment, BX     ; save page frame address
```

15

FUNCTION 3 GET UNALLOCATED PAGE COUNT

PURPOSE

The Get Unallocated Page Count function returns the number of unallocated pages and the total number of expanded memory pages.

CALLING PARAMETERS

AH = 42h Contains the Get Unallocated Page Count function.

RESULTS

These results are valid only if the status returned is zero.

BX = unallocated pages
: The number of expanded memory pages that are currently available for use (unallocated).

DX = total pages
: The total number expanded memory pages.

REGISTERS MODIFIED

AX, BX, DX

STATUS

AH = 0 SUCCESSFUL
: The manager has returned the number of unallocated pages and the number of total pages in expanded memory.

AH = 80h NON-RECOVERABLE
: The manager detected a malfunction in the memory manager software.

AH = 81h NON-RECOVERABLE
: The manager detected a malfunction in the expanded memory hardware.

AH = 84h NON-RECOVERABLE

The function code passed to the memory manager is not defined.

EXAMPLE

```
un_alloc_pages          DW  ?
total_pages             DW  ?

MOV  AH, 42h            ; load function code
INT  67h                ; call the memory manager
OR   AH, AH             ; check EMM status
JNZ  emm_err_handler    ; jump to error handler on error
MOV  un_alloc_pages, BX ; save unallocated page count
MOV  total_pages, DX    ; save total page count
```


FUNCTION 4 ALLOCATE PAGES

PURPOSE

The Allocate Pages function allocates the number of pages requested and assigns a unique EMM handle to these pages. The EMM handle owns these pages until the application deallocates them.

Handles which are assigned using this function will have 16K-byte pages, the size of a standard expanded memory page. If the expanded memory board hardware isn't able to supply 16K-byte pages, it will emulate them by combining multiple non-standard size pages to form a single 16K-byte page. All application programs and functions that use the handles this function returns will deal with 16K-byte pages.

The numeric value of the handles the EMM returns are in the range of 1 to 254 decimal (0001h to 00FEh). The OS handle (handle value 0) is never returned by the Allocate Pages function. Also, the uppermost byte of the handle will be zero and cannot be used by the application. A memory manager should be able to supply up to 255 handles, including the OS handle. An application can use Function 21 to find out how many handles an EMM supports.

Allocating zero pages to a handle is not valid. If an application needs to allocate 0 pages to a handle it should use Function 27 provided for this purpose.

Note.....

This note affects expanded memory manager implementers and operating system developers only. Applications should not use the following characteristic of the memory manager. An application violating this rule will be incompatible with future versions of Microsoft's operating systems and environments.

To be compatible with this specification, an expanded memory manager will provide a special handle which is available to the operating system only. This handle will have a value of 0000h and will have a set of pages allocated to it when the expanded memory manager driver installs. The pages that the memory manager will automatically allocate to handle 0000h are those that backfill conventional memory. Typically, this backfill occurs between addresses 40000h (256K) and 9FFFFh (640K). However, the range can extend below and above this limit if the hardware and memory manager have the capability.

An operating system won't have to invoke Function 4 to obtain this handle because it can assume the handle already exists and is available for use immediately after the expanded memory device driver installs. When an operating system wants to use this handle, it uses the special handle value of 0000h. The operating system will be able to invoke any EMM function using this special handle value. To allocate pages to this handle, the operating system need only invoke Function 18 (Reallocate pages)..

There are two special cases for this handle:

1. Function 4 (Allocate Pages). This function must never return zero as a handle value. Applications must always invoke Function 4 to allocate pages and obtain a handle which identifies the pages which belong to it. Since Function 4 never returns a handle value of zero, an application will never gain access to this special handle.

2. Function 6 (Deallocate Pages). If the operating system uses it to deallocate the pages which are allocated to this special handle, the pages the handle owns will be returned to the manager for use. But the handle will not be available for reassignment. The manager should treat a deallocate pages function request for this handle the same as a reallocate pages function request, where the number of pages to reallocate to this handle is zero.

CALLING PARAMETERS

AH = 43h

Contains the Allocate Pages function.

BX = num_of_pages_to_alloc

Contains the number of pages you want your program to allocate.

RESULTS

These results are valid only if the status returned is zero.

DX = handle

Contains a unique EMM handle. Your program must use this EMM handle (as a parameter) in any function that requires it. You can use up to 255 handles. The uppermost byte of the handle will be zero and cannot be used by the application.

REGISTERS MODIFIED

AX, DX

STATUS

AH = 0 SUCCESSFUL

The manager has allocated the requested pages to the assigned EMM handle.

AH = 80h NON-RECOVERABLE

The manager detected a malfunction in the memory manager software.

AH = 81h NON-RECOVERABLE

The manager detected a malfunction in the expanded memory hardware.

AH = 84h NON-RECOVERABLE

The function code passed to the memory manager is not defined.

AH = 85h RECOVERABLE

All EMM handles are being used.

AH = 87h RECOVERABLE

There aren't enough expanded memory pages present in the system to satisfy your program's request.

AH = 88h RECOVERABLE

There aren't enough unallocated pages to satisfy your program's request.

AH = 89h RECOVERABLE

Your program attempted to allocate zero pages.

EXAMPLE

```
num_of_pages_to_alloc       DW  ?
emm_handle                  DW  ?

MOV  BX, num_of_pages_to_alloc    ; load number of pages
MOV  AH, 43h                      ; load function code
INT  67h                        ; call the memory manager
OR   AH, AH                       ; check EMM status
JNZ  emm_err_handler              ; jump to error handler on error
MOV  emm_handle, DX               ; save EMM handle
```


FUNCTION 5 MAP/UNMAP HANDLE PAGES

PURPOSE

The Map/Unmap Handle Page function maps a logical page at a specific physical page anywhere in the mappable regions of system memory. The lowest valued physical page numbers are associated with regions of memory outside the conventional memory range. Use Function 25 (Get Mappable Physical Address Array) to determine which physical pages within a system are mappable and determine the segment addresses which correspond to a specific physical page number. Function 25 provides a cross reference between physical page numbers and segment addresses.

This function can also unmap physical pages, making them inaccessible for reading or writing. You unmap a physical page by setting its associated logical page to FFFFh.

You might unmap an entire set of mapped pages, for example, before loading and executing a program. Doing so ensures the loaded program, if it accesses expanded memory, won't access the pages your program has mapped. However, you must save the mapping context before you unmap the physical pages. This enables you to restore it later so you can access the memory you mapped there. To save the mapping context, use Function 8, 15, or 16. To restore the mapping context, use Function 9, 15, or 16.

The handle determines what type of pages are being mapped. Logical pages allocated by Function 4 and Function 27 (Allocate Standard Pages subfunction) are referred to as pages and are 16K bytes long. Logical pages allocated by Function 27 (Allocate Raw Pages subfunction) are referred to as raw pages and might not be the same size as logical pages.

CALLING PARAMETERS

AH = 44h
Contains the Map Handle Page function.

AL = physical_page_number
Contains the number of the physical page into which the logical page number is to be mapped. Physical pages are numbered zero relative.

BX = logical_page_number
Contains the number of the logical page to be mapped at the physical page within the page frame. Logical pages are numbered zero relative. The logical page must be in the range zero through (number of pages allocated to the EMM handle - 1). However, if BX contains logical page number FFFFh, the physical page specified in AL will be unmapped (be made inaccessible for reading or writing).

DX = emm_handle
Contains the EMM handle your program received from Function 4 (Allocate Pages).

REGISTERS MODIFIED

AX

STATUS

AH = 0 SUCCESSFUL
The manager has mapped the page. The page is ready to be accessed.

AH = 80h NON-RECOVERABLE
The manager detected a malfunction in the memory manager software.

AH = 81h NON-RECOVERABLE

The manager detected a malfunction in the expanded memory hardware.

AH = 83h NON-RECOVERABLE

The memory manager couldn't find the EMM handle your program specified.

AH = 84h NON-RECOVERABLE

The function code passed to the memory manager isn't defined.

AH = 8Ah RECOVERABLE

The logical page is out of the range of logical pages which are allocated to the EMM handle. This status is also returned if a program attempts map a logical page when no logical pages are allocated to the handle.

AH = 8Bh RECOVERABLE

The physical page number is out of the range of allowable physical pages. The program can recover by attempting to map into memory at a physical page which is within the range of allowable physical pages.

EXAMPLE

```
emm_handle              DW  ?
logical_page_number         DW  ?
physical_page_number         DB  ?

MOV  DX, emm_handle           ; load EMM handle
MOV  BX, logical_page_number    ; load logical page number
MOV  AL, physical_page_number   ; load physical page number
MOV  AH, 44h                  ; load function code
INT  67h                    ; call the memory manager
OR   AH, AH                   ; check EMM status
JNZ  emm_err_handler          ; jump to error handler on error
```


FUNCTION 6 DEALLOCATE PAGES

PURPOSE

Deallocate Pages deallocates the logical pages currently allocated to an EMM handle. Only after the application deallocates these pages can other applications use them. When a handle is deallocated, it name is set to all ASCII nulls (binary zeros).

Note.....

A program must perform this function before it exits to DOS. If it doesn't, no other programs can use these pages or the EMM handle. This means that a program using expanded memory should trap critical errors and control-break if there is a chance that the program will have allocated pages when either of these events occur.

Calling Parameters

AH = 45h
Contains the Deallocate Pages function.

DX = handle
Contains the EMM handle returned by Function 4 (Allocate Pages).

REGISTERS MODIFIED

AX

STATUS

AH = 0 SUCCESSFUL
The manager has deallocated the pages previously allocated to the EMM handle.

AH = 80h NON-RECOVERABLE
The manager detected a malfunction in the memory manager software.

AH = 81h NON-RECOVERABLE
The manager detected a malfunction in the expanded memory hardware.

AH = 83h NON-RECOVERABLE
The manager couldn't find the specified EMM handle.

AH = 84h NON-RECOVERABLE
The function code passed to the memory manager is not defined.

AH = 86h RECOVERABLE

The memory manager detected a save or restore page mapping context error (Function 8 or 9). There is a page mapping register state in the save area for the specified EMM handle. Save Page Map (Function 8) placed it there and a subsequent Restore Page Map (Function 9) has not removed

it. If you have saved the mapping context, you must restore it before you deallocate the EMM handle's pages.

EXAMPLE

```
emm_handle                  DW  ?

MOV  DX, emm_handle             ; load EMM handle
MOV  AH, 45h                    ; load function code
INT  67h                    ; call the memory manager
OR   AH, AH                     ; check EMM status
JNZ  emm_err_handler            ; jump to error handler on error
```


FUNCTION 7 GET VERSION

PURPOSE

The Get Version function returns the version number of the memory manager software.

CALLING PARAMETERS

AH = 46h

Contains the Get Version function.

RESULTS

These results are valid only if the status returned is zero.

AL = version number

Contains the memory manager's version number in binary coded decimal (BCD) format. The upper four bits contain the integer digit of the version number. The lower four bits contain the fractional digit of version number. For example, version 4.0 is represented like this:

```
0100 0000
 / \                          4 . 0
```

When checking for a version number, an application should check for a version number or greater. Vendors may use the fractional digit to indicate enhancements or corrections to their memory managers. Therefore, to allow for future versions of memory managers, an application shouldn't depend on an exact version number.

REGISTERS MODIFIED

AX

STATUS

AH = 0 SUCCESSFUL
The manager is present in the system and the hardware is working correctly.

AH = 80h NON-RECOVERABLE
The manager detected a malfunction in the memory manager software.

AH = 81h NON-RECOVERABLE
The manager detected a malfunction in the expanded memory hardware.

AH = 84h NON-RECOVERABLE
The function code passed to the memory manager is not defined.

EXAMPLE

24

```
emm_version                DB  ?

MOV  AH, 46h                ; load function code
INT  67h                   ; call the memory manager
OR   AH, AH                 ; check EMM status
JNZ  emm_err_handler        ; jump to error handler on error
MOV  emm_version, AL        ; save version number
```

25

FUNCTION 8 SAVE PAGE MAP

PURPOSE

Save Page Map saves the contents of the page mapping registers on all expanded memory boards in an internal save area. The function is typically used to save the memory mapping context of the EMM handle that was active when a software or hardware interrupt occurred. (See Function 9, Restore Page Map, for the restore operation.) If you're writing a resident program, an interrupt service, or a device driver that uses expanded memory, you must save the state of the mapping hardware. You must save this state because application software using expanded memory may be running when your program is invoked by a hardware interrupt, a software interrupt, or DOS.

The Save Page Map function requires the EMM handle that was assigned to your resident program, interrupt service routine, or device driver at the time it was initialized. This is not the EMM handle that the application software was using when your software interrupted it.

The Save Page Map function saves the state of the map registers for only the 64K-byte page frame defined in versions 3.x of this specification. Since all applications written to LIM versions 3.x require saving the map register state of only this 64K-byte page frame, saving the entire mapping state for a large number of mappable pages would be inefficient use of memory. Applications that use a mappable memory region outside the LIM 3.x page frame should use Function 15 or 16 to save and restore the state of the map registers.

CALLING PARAMETERS

AH = 47h

Contains the Save Page Map function.

DX = handle

Contains the EMM handle assigned to the interrupt service routine that's servicing the software or hardware interrupt. The interrupt service routine needs to save the state of the page mapping hardware before mapping any pages.

REGISTERS MODIFIED

AX

STATUS

AH = 0 SUCCESSFUL

The manager has saved the state of the page mapping hardware.

AH = 80h NON-RECOVERABLE

The manager detected a malfunction in the memory manager software.

AH = 81h NON-RECOVERABLE

The manager detected a malfunction in the expanded memory hardware.

AH = 83h NON-RECOVERABLE

The memory manager couldn't find the EMM handle your program specified.

AH = 84h NON-RECOVERABLE

The function code passed to the memory manager is not defined.

AH = 8Ch NON-RECOVERABLE

There is no room in the save area to store the state of the page mapping registers. The state of the map registers has not been saved.

AH = 8Dh CONDITIONALLY-RECOVERABLE

The save area already contains the page mapping register state for the EMM handle your program specified.

EXAMPLE

```
emm_handle                  DW  ?

MOV  DX, emm_handle             ; load EMM handle
MOV  AH, 47h                    ; load function code
INT  67h                    ; call the memory manager
OR   AH, AH                     ; check EMM status
JNZ  emm_err_handler            ; jump to error handler on error
```


FUNCTION 9 RESTORE PAGE MAP

PURPOSE

The Restore Page Map function restores the page mapping register contents on the expanded memory boards for a particular EMM handle. This function lets your program restore the contents of the mapping registers its EMM handle saved. (See Function 8, Save Page Map for the save operation.)

If you're writing a resident program, an interrupt service routine, or a device driver that uses expanded memory, you must restore the mapping hardware to the state it was in before your program took over. You must save this state because application software using expanded memory might have been running when your program was invoked.

The Restore Page Map function requires the EMM handle that was assigned to your resident program, interrupt service routine, or device driver at the time it was initialized. This is not the EMM handle that the application software was using when your software interrupted it.

The Restore Page Map function restores the state of the map registers for only the 64K-byte page frame defined in versions 3.x of this specification. Since all applications written to LIM versions 3.x require restoring the map register state of only this 64K-byte page frame, restoring the entire mapping state for a large number of mappable pages would be inefficient use of memory. Applications that use a mappable memory region outside the LIM 3.x page frame should use Function 15 or 16 to save and restore the state of the map registers.

CALLING PARAMETERS

AH = 48h
: Contains the Restore Page Map function.

DX = emm_handle
: Contains the EMM handle assigned to the interrupt service routine that's servicing the software or hardware interrupt. The interrupt service routine needs to restore the state of the page mapping hardware.

REGISTERS MODIFIED

AX

STATUS

AH = 0 SUCCESSFUL
: The manager has restored the state of the page mapping registers.

AH = 80h NON-RECOVERABLE

The manager detected a malfunction in the memory manager software.

AH = 81h NON-RECOVERABLE

The manager detected a malfunction in the expanded memory hardware.

AH = 83h NON-RECOVERABLE

The memory manager couldn't find the EMM handle your program specified.

AH = 84h NON-RECOVERABLE

The function code passed to the memory manager is not defined.

AH = 8Eh CONDITIONALLY-RECOVERABLE

There is no page mapping register state in the save area for the specified EMM handle. Your program didn't save the contents of the page mapping hardware, so Restore Page Map can't restore it.

EXAMPLE

```
emm_handle                  DW  ?

MOV  DX, emm_handle             ; load EMM handle
MOV  AH, 48h                    ; load function code
INT  67h                    ; call the memory manager
OR   AH, AH                     ; check EMM status
JNZ  emm_err_handler            ; jump to error handler on error
```


FUNCTION 10 RESERVED

In earlier versions of the Lotus/Intel/Microsoft Expanded Memory Specification the use of this function was documented. This function is now documented as "RESERVED" (i.e. not documented) in the public versions of the specification and new programs should not use it.

Existing programs that use this function may still work correctly if the hardware is capable of supporting them. Since IBM Microchannel hardware is not capable of supporting this function, it cannot be implemented in the IBM Microchannel architecture. However, programs that use Functions 16 through 30 in Version 4.0 of this specification must not use Functions 10 and 11. These functions won't work correctly if a program attempts to mix the use of the new functions (Functions 16 through 30) and functions 10 and 11. Functions 10 and 11 are specific to the hardware on Intel expanded memory boards and will not work correctly on all vendors expanded memory boards.

30

FUNCTION 11 RESERVED

In earlier versions of the Lotus/Intel/Microsoft Expanded Memory Specification the use of this function was documented. This function is now documented as "RESERVED" (i.e. not documented) in the public versions of the specification and new programs should not use it.

Existing programs that use this function may still work correctly if the hardware is capable of supporting them. Since IBM Microchannel hardware is not capable of supporting this function, it cannot be implemented in the IBM Microchannel architecture. However, programs that use Functions 16 through 30 in Version 4.0 of this specification must not use Functions 10 and 11. These functions won't work correctly if a program attempts to mix the use of the new functions (Functions 16 through 30) and functions 10 and 11. Functions 10 and 11 are specific to the hardware on Intel expanded memory boards and will not work correctly on all vendors expanded memory boards.

31

FUNCTION 12 GET HANDLE COUNT

PURPOSE

The Get Handle Count function returns the number of open EMM handles (including the operating system handle 0) in the system.

CALLING PARAMETERS

AH = 4Bh

Contains the Get Handle Count function.

RESULTS

These results are valid only if the status returned is zero.

BX = total_open_emm_handles
Contains the number of open EMM handles [including the operating system handle (0)]. This number will not exceed 255.

REGISTERS MODIFIED

AX, BX

STATUS

AH = 0 SUCCESSFUL
The manager has returned the number of active EMM handles.

AH = 80h NON-RECOVERABLE
The manager detected a malfunction in the memory manager software.

AH = 81h NON-RECOVERABLE
The manager detected a malfunction in the expanded memory hardware.

AH = 84h NON-RECOVERABLE
The function code passed to the memory manager is not defined.

32

EXAMPLE

```
total_open_emm_handles          DW  ?

MOV  AH, 4Bh                    ; load function code
INT  67h                        ; call the memory manager
OR   AH, AH                     ; check EMM status
JNZ  emm_err_handler            ; jump to error handler on error
MOV  total_open_emm_handles, BX    ; save total active handle count
```

33

FUNCTION 13 GET HANDLE PAGES

PURPOSE

The Get Handle Pages function returns the number of pages allocated to a specific EMM handle.

CALLING PARAMETERS

AH = 4Ch
Contains the Get Handle Pages function.

DX = emm_handle
Contains the EMM handle.

RESULTS

These results are valid only if the status returned is zero.

BX = num_pages_alloc_to_emm_handle
Contains the number of logical pages allocated to the specified EMM handle. This number never exceeds 2048 because the memory manager allows a maximum of 2048 pages (32M bytes) of expanded memory.

REGISTERS MODIFIED

AX, BX

STATUS

AH = 0 SUCCESSFUL
The manager has returned the number of pages allocated to the EMM handle.

AH = 80h NON-RECOVERABLE
The manager detected a malfunction in the memory manager software.

AH = 81h NON-RECOVERABLE
The manager detected a malfunction in the expanded memory hardware.

AH = 83h NON-RECOVERABLE
The memory manager couldn't find the EMM handle your program specified.

AH = 84h NON-RECOVERABLE
The function code passed to the memory manager is not defined.

EXAMPLE

```
emm_handle                  DW  ?
pages_alloc_to_handle       DW  ?

MOV  DX, emm_handle             ; load EMM handle
MOV  AH, 4Ch                    ; load function code
INT  67h                        ; call the memory manager
OR   AH, AH                     ; check EMM status
JNZ  emm_err_handler            ; jump to error handler on error
MOV  pages_alloc_to_handle, BX  ; save number of pages allocated
                                ; to specified handle
```

35

FUNCTION 14 GET ALL HANDLES PAGES

PURPOSE

The Get All Handle Pages function returns an array of the open emm handles and the number of pages allocated to each one.

CALLING PARAMETERS

AH = 4Dh

Contains the Get All Handle Pages function.

```
handle_page_struct      STRUC
  emm_handle            DW ?
  pages_alloc_to_handle DW ?
handle_page_struct      ENDS
```

ES:DI = pointer to handle_page

Contains a pointer to an array of structures where a copy of all open EMM handles and the number of pages allocated to each will be stored. Each structure has these two members:

.emm_handle

The first member is a word which contains the value of the open EMM handle. The values of the handles this function returns will be in the range of 0 to 255 decimal (0000h to 00FFh). The uppermost byte of the handle is always zero.

.pages_alloc_to_handle

The second member is a word which contains the number of pages allocated to the open EMM handle.

RESULTS

These results are valid only if the status returned is zero.

BX = total_open_emm_handles
Contains the number of open EMM handles (including the operating system handle [0]). The number cannot be zero because the operating system handle is always active and cannot be deallocated. This number will not exceed 255.

REGISTERS MODIFIED

AX, BX

STATUS

AH = 0 SUCCESSFUL
The manager has returned the array.

AH = 80h NON-RECOVERABLE
The manager detected a malfunction in the memory manager software.

AH = 81h NON-RECOVERABLE
The manager detected a malfunction in the expanded memory hardware.

AH = 84h NON-RECOVERABLE
The function code passed to the memory manager isn't defined.

EXAMPLE

```
handle_page                 handle_page_struct 255 DUP (?)
total_open_handles            DW  ?

MOV  AX, SEG handle_page
MOV  ES, AX
LEA  DI, handle_page          ; ES:DI points to handle_page
MOV  AH, 4Dh                  ; load function code
INT  67h                      ; call the memory manager
OR   AH, AH                   ; check EMM status
JNZ  emm_err_handler          ; jump to error handler on error
MOV  total_open_handles, BX   ; save total open handle count
```


FUNCTION 15 GET/SET PAGE MAP

GET PAGE MAP SUBFUNCTION

PURPOSE

The Get Page Map subfunction saves the mapping context for all mappable memory regions (conventional and expanded) by copying the contents of the mapping registers from each expanded memory board to a destination array. The application must pass a pointer to the destination array. This subfunction doesn't require an EMM handle. Use this function instead of Functions 8 and 9 if you need to save or restore the mapping context but don't want (or have) to use a handle.

CALLING PARAMETERS

AX = 4E00h
: Contains the Get Page Map subfunction.

ES:DI = dest_page_map
: Contains a pointer to the destination array address in segment:offset format. Use the Get Size of Page Map Save Array subfunction to determine the size of the desired array.

RESULTS

These results are valid only if the status returned is zero.

dest_page_map
: The array contains the state of all the mapping registers on all boards in the system. It also contains any additional information necessary to restore the boards to their original state when the program invokes a Set subfunction.

REGISTERS MODIFIED

AX

STATUS

AH = 0 SUCCESSFUL
: The manager has returned the array.

AH = 80h NON-RECOVERABLE
: The manager detected a malfunction in the memory manager software.

AH = 81h NON-RECOVERABLE

The manager detected a malfunction in the expanded memory hardware.

AH = 84h NON-RECOVERABLE

The function code passed to the memory manager is not defined.

AH = 8Fh NON-RECOVERABLE

The subfunction parameter is invalid.

EXAMPLE

```
dest_page_map                DB  ?  DUP  (?)

MOV  AX, SEG dest_page_map
MOV  ES, AX
LEA  DI, dest_page_map          ; ES:DI points to dest_page_map
MOV  AX, 4E00h                  ; load function code
INT  67h                        ; call the memory manager
OR   AH, AH                     ; check EMM status
JNZ  emm_err_handler            ; jump to error handler on error
```


FUNCTION 15 GET/SET PAGE MAP (CONTINUED)

SET PAGE MAP SUBFUNCTION

PURPOSE

The Set Page Map subfunction restores the mapping context for all mappable memory regions (conventional and expanded) by copying the contents of a source array into the mapping registers on each expanded memory board in the system. The application must pass a pointer to the source array. This subfunction doesn't require an EMM handle. Use this function instead of Functions 8 and 9 if you need to save or restore the mapping context but don't want (or have) to use a handle.

CALLING PARAMETERS

AX = 4E01h

Contains the Set Page Map subfunction.

DS:SI = source_page_map
Contains a pointer to the source array address in segment:offset format. The application must point to an array which contains the mapping register state. Use the Get Size of Page Map Save Array subfunction to determine the size of the desired array.

REGISTERS MODIFIED

AX

STATUS

AH = 0 SUCCESSFUL
The manager has passed the array.

AH = 80h NON-RECOVERABLE
The manager detected a malfunction in the memory manager software.

AH = 81h NON-RECOVERABLE
The manager detected a malfunction in the expanded memory hardware.

AH = 84h NON-RECOVERABLE
The function code passed to the memory manager is not defined.

AH = 8Fh NON-RECOVERABLE
The subfunction parameter is invalid.

AH = A3h NON-RECOVERABLE
The contents of the source array have been corrupted, or the pointer passed to the subfunction is invalid.

EXAMPLE

```
source_page_map                DB  ?  DUP  (?)
```



```
MOV  AX, SEG source_page_map
MOV  DS, AX
LEA  SI, source_page_map          ; DS:SI points to source_page_map
MOV  AX, 4E01h                    ; load function code
INT  67h                          ; call the memory manager
OR   AH, AH                       ; check EMM status
JNZ  emm_err_handler              ; jump to error handler on error
```

FUNCTION 15 GET/SET PAGE MAP (CONTINUED)

GET & SET PAGE MAP SUBFUNCTION

PURPOSE

The Get & Set subfunction simultaneously saves a current mapping context and restores a previous mapping context for all mappable memory regions (both conventional and expanded). It first copies the contents of the mapping registers from each expanded memory board in the system into a destination array. (The application must pass a pointer to the destination array.) Then, the subfunction copies the contents of a source array into the mapping registers on each of the expanded memory boards. (The application must pass a pointer to the source array.)

Use this function instead of Functions 8 and 9 if you need to save or restore the mapping context but don't want (or have) to use a handle.

CALLING PARAMETERS

AX = 4E02h
: Contains the Get & Set Page Map subfunction.

ES:DI = dest_page_map
: Contains a pointer to the destination array address in segment:offset format. The current contents of the map registers will be saved in this array.

DS:SI = source_page_map
: Contains a pointer to the source array address in segment:offset format. The contents of this array will be copied into the map registers. The application must point to an array which contains the mapping register state. This address is required only for the Set or Get and Set subfunctions.

RESULTS

These results are valid only if the status returned is zero.

dest_page_map
: The array contains the mapping state. It also contains any additional information necessary to restore the original state when the program invokes a Set subfunction.

REGISTERS MODIFIED

AX

STATUS

AH = 0 SUCCESSFUL
The manager has returned and passed both arrays.

42

AH = 80h NON-RECOVERABLE
The manager detected a malfunction in the memory manager software.

AH = 81h NON-RECOVERABLE
The manager detected a malfunction in the expanded memory hardware.

AH = 84h NON-RECOVERABLE the function code passed to the memory manager is not defined.

AH = 8Fh NON-RECOVERABLE
The subfunction parameter is invalid.

AH = A3h NON-RECOVERABLE
The contents of the source array have been corrupted, or the pointer passed to the subfunction is invalid.

EXAMPLE

```
dest_page_map                DB  ?  DUP  (?)
source_page_map               DB  ?  DUP  (?)

MOV  AX, SEG dest_page_map
MOV  ES, AX
MOV  AX, SEG source_page_map
MOV  DS, AX
LEA  DI, dest_page_map          ; ES:DI points to dest_page_map
LEA  SI, source_page_map        ; DS:SI points to source_page_map
MOV  AX, 4E02h                  ; load function code
INT  67h                        ; call the memory manager
OR   AH, AH                     ; check EMM status
JNZ  emm_err_handler            ; jump to error handler on error
```

43

FUNCTION 15 GET/SET PAGE MAP (CONTINUED)

GET SIZE OF PAGE MAP SAVE ARRAY SUBFUNCTION

PURPOSE

The Get Size of Page Map Save Array subfunction returns the storage requirements for the array passed by the other three subfunctions. This subfunction doesn't require an EMM handle.

CALLING PARAMETERS

AX = 4E03h
: Contains the Get Size of Page Map Save Array subfunction. The size of this array depends on how the expanded memory system is configured and how the expanded memory manager is implemented. Therefore, the size must be determined after the memory manager is loaded.

RESULTS

These results are valid only if the status returned is zero.

AL = size_of_array
: Contains the number of bytes that will be transferred to the memory area an application supplies whenever a program requests the Get, Set, or Get and Set subfunctions.

REGISTERS MODIFIED

AX

STATUS

AH = 0 SUCCESSFUL
: The manager has returned the array size.

AH = 80h NON-Recoverable the manager detected a malfunction in the memory manager software.

AH = 81h NON-Recoverable the manager detected a malfunction in the expanded memory hardware.

AH = 84h NON-Recoverable the function code passed to the memory manager is not defined.

AH = 8Fh NON-RECOVERABLE
: The subfunction parameter is invalid.

EXAMPLE

```
size_of_array                DB  ?

MOV  AX, 4E03h                  ; load function code
INT  67h                     ; call the memory manager
OR   AH, AH                     ; check EMM status
JNZ  emm_err_handler            ; jump to error handler on error
```

44

```
MOV  size_of_array, AL          ; save array size
```

45

FUNCTION 16 GET/SET PARTIAL PAGE MAP

GET PARTIAL PAGE MAP SUBFUNCTION

PURPOSE

The Get Partial Page Map subfunction saves a partial mapping context for specific mappable memory regions in a system. Because this function saves only a subset of the entire mapping context, it uses much less memory for the save area and may be potentially faster than Function 15. The subfunction does this by copying the contents of selected mapping registers from each expanded memory board to a destination array.

The application must pass a pair of pointers. The first points to a structure which specifies which mappable segments to save; the second points to the destination array.

Use this function instead of Functions 8 and 9 if you need to save or restore the mapping context but don't want (or have) to use a handle.

CALLING PARAMETERS

AX = 4F00h
Contains the Get Partial Page Map subfunction.

```
partial_page_map_struct     STRUC
  mappable_segment_count   DW  ?
  mappable_segment         DW  (?)  DUP  (?)
partial_page_map_struct     ENDS
```

DS:SI = partial_page_map

Contains a pointer to a structure which specifies only those mappable memory regions which are to have their mapping context saved. The structure members are described below.

.mappable_segment_count

The first member is a word which specifies the number of members in the word array which immediately follows it. This number should not exceed the number of mappable segments in the system.

.mappable_segment

The second member is a word array which contains the segment addresses of the mappable memory regions whose mapping contexts are to be saved. The segment address must be a mappable segment. Use Function 25 to determine which segments are mappable.

ES:DI = dest_array

Contains a pointer to the destination array address in Segment:Offset format. To determine the size of the required array, see the Get Size of Partial Page Map Save Array subfunction.

RESULTS

These results are valid only if the status returned is zero.

dest_array

The array contains the partial mapping context and any additional information necessary to restore this context to its original state when the program invokes a Set subfunction.

REGISTERS MODIFIED

AX

STATUS

AH = 0 SUCCESSFUL

The manager has saved the partial map context.

AH = 80h NON-RECOVERABLE
The manager detected a malfunction in the memory manager software.

AH = 81h NON-RECOVERABLE
The manager detected a malfunction in the expanded memory hardware.

AH = 84h NON-RECOVERABLE
The function code passed to the memory manager is not defined.

AH = 8Bh NON-RECOVERABLE
One of the specified segments is not a mappable segment.

AH = 8Fh NON-RECOVERABLE
The subfunction parameter is invalid.

AH = A3h NON-RECOVERABLE
The contents of the partial page map structure have been corrupted, the pointer passed to the subfunction is invalid, or the mappable_segment_count exceeds the number of mappable segments in the system.

EXAMPLE

```
partial_page_map            partial_page_map_struct <>
dest_array                  DB ? DUP (?)

MOV  AX, SEG partial_page_map
MOV  DS, AX
LEA  SI, partial_page_map       ; DS:SI points to
MOV  AX, SEG dest_array          ; partial_page_map
MOV  ES, AX
LEA  DI, dest_array             ; ES:DI points to dest_array
MOV  AX, 4F00h                   ; load function code
INT  67h                        ; call the memory manager
OR   AH, AH                     ; check EMM status
JNZ  emm_err_handler            ; jump to error handler on error
```


FUNCTION 16 GET/SET PARTIAL PAGE MAP (CONTINUED)

SET PARTIAL PAGE MAP SUBFUNCTION

PURPOSE

The Set Partial Page Map subfunction provides a mechanism for restoring the mapping context for a partial mapping context for specific mappable memory regions in a system. Because this function restores only a subset of the entire mapping context and not the entire systems mapping context, it uses much less memory for the save area and is potentially faster than Function 15. The subfunction does this by copying the contents of the source array to selected mapping registers on each expanded memory board. The application passes a pointer to the source array. Use this function instead of Functions 8 and 9 if you need to save or restore the mapping context but don't want (or have) to use a handle.

CALLING PARAMETERS

AX = 4F01h
Contains the Set Partial Page Map subfunction.

source_array DB ? DUP (?)

DS:SI = source_array
Contains a pointer to the source array in segment:offset format. The application must point to an array which contains the partial mapping register state. To determine the size of the required array, see the Get Size of Partial Page Map Save Array subfunction.

REGISTERS MODIFIED

AX

STATUS

AH = 0 SUCCESSFUL
The manager has restored the partial mapping context.

AH = 80h NON-RECOVERABLE
The manager detected a malfunction in the memory manager software.

AH = 81h NON-RECOVERABLE
The manager detected a malfunction in the expanded memory hardware.

AH = 84h NON-RECOVERABLE
The function code passed to the memory manager is not defined.

AH = 8Fh NON-RECOVERABLE

The subfunction parameter is invalid.

AH = A3h NON-RECOVERABLE

The contents of the source array have been corrupted, or the pointer passed to the subfunction is invalid.

EXAMPLE

```
MOV  AX, SEG source_array
MOV  DS, AX
LEA  SI, source_array          ; DS:SI points to source_array
MOV  AX, 4F01h                 ; load function code
INT  67h                     ; call the memory manager
OR   AH, AH                    ; check EMM status
JNZ  emm_err_handler           ; jump to error handler on error
```


FUNCTION 16 GET/SET PARTIAL PAGE MAP (CONTINUED)

GET SIZE OF PARTIAL PAGE MAP SAVE ARRAY SUBFUNCTION

PURPOSE

The Return Size subfunction returns the storage requirements for the array passed by the other two subfunctions. This subfunction doesn't require an EMM handle.

CALLING PARAMETERS

AX = 4F02h

Contains the Get Size of Partial Page Map Save Array subfunction. The size of this array depends on the expanded memory system configuration and the implementation of the expanded memory manager. Therefore, it will vary between hardware configurations and implementations and must be determined after a specific memory manager is loaded.

BX = number of pages in the partial array

Contains the number of pages in the partial map to be saved by the Get/Set Partial Page Map subfunctions. This number should be the same as the mappable_segment_count in the Get Partial Page Map subfunction.

Results

These results are valid only if the status returned is zero.

AL = size_of_partial_save_array
Contains the number of bytes that will be transferred to the memory area supplied by an application whenever a program requests the Get or Set subfunction.

REGISTERS MODIFIED

AX

STATUS

AH = 0 SUCCESSFUL
The manager has returned the array size.

AH = 80h NON-RECOVERABLE
The manager detected a malfunction in the memory manager software.

AH = 81h NON-RECOVERABLE
The manager detected a malfunction in the expanded memory hardware.

AH = 84h NON-RECOVERABLE
The function code passed to the memory manager is not defined.

AH = 8Bh NON-RECOVERABLE
The number of pages in the partial array is outside the range of physical pages in the system.

AH = 8Fh NON-RECOVERABLE
The subfunction parameter is invalid.

EXAMPLE

```
number_of_pages_to_map          DW  ?
size_of_partial_save_array      DB  ?
```

```
MOV  BX, number_of_pages_to_map
MOV  AX, 4F02h                  ; load function code
INT  67h                        ; call the memory manager
OR   AH, AH                     ; check EMM status
JNZ  emm_err_handler            ; jump to error handler on error
MOV  size_of_partial_save_array, AL; save array size
```

51

FUNCTION 17 MAP/UNAMP MULTIPLE HANDLE PAGES

PURPOSE

This function can, in a single invocation, map (or unmap) logical pages into as many physical pages as the system supports. Consequently, it has less execution overhead than mapping pages one at a time. For applications which do a lot of page mapping, this is the preferred mapping method.

MAPPING MULTIPLE PAGES

The handle passed to this function determines what type of logical pages are being mapped. Logical pages that Function 4 and Function 27 (Allocate Standard Pages subfunction) allocate are referred to as pages and are 16K bytes. Logical pages that Function 27 (Allocate Raw Pages subfunction) allocates are referred to as raw pages and might not be the same size as the pages Function 4 and Function 27 (Allocate Standard Pages subfunction) allocate.

UNMAPPING MULTIPLE PAGES

This function can make specific physical pages unavailable for reading or writing. A logical page which is unmapped from a specific physical page cannot be read or written from that physical page. The logical page which is unavailable (unmapped) can be made available again by mapping it, or a new logical page, at the physical page that was unmapped. Unmapping a physical page is accomplished by setting the logical page it is associated with to FFFFh.

You might unmap an entire set of mapped pages, for example, before loading and executing a program. This ensures that the loaded program won't be able to access the pages your program has mapped. However, you must save the mapping context before you unmap the physical pages. This enables you to restore it later so that you may access the memory you had mapped there. You can save the mapping context with Functions 8, 15, or 16. You can restore the mapping context with Functions 9, 15, or 16.

MAPPING AND UNMAPPING MULTIPLE PAGES SIMULTANEOUSLY

Both mapping and unmapping pages can be done in the same invocation.

Mapping or unmapping no pages is not considered an error. If a request to map or unmap zero pages is made, nothing is done and no error is returned.

ALTERNATE MAPPING AND UNMAPPING METHODS

You can map or unmap pages using two methods. Both methods produce identical results.

1 The first method specifies both a logical page and a physical page at which the logical page is to be mapped. This method is an extension of Function 5 (Map Handle Page).

2 The second method specifies both a logical page and a corresponding segment address at which the logical page is to be mapped. While this is functionally the same as the first method, it may be easier to use the actual segment address of a physical page than to use a number which only represents its location. The memory manager verifies whether the specified segment address falls on the boundary of a mappable physical page. The manager then translates the segment address passed to it into the necessary internal representation to map the pages.

MAP/UNMAP MULTIPLE HANDLE PAGES

LOGICAL PAGE/PHYSICAL PAGE METHOD

CALLING PARAMETERS

AX = 5000h

Contains the Map/Unmap Multiple Handle Pages subfunction using the logical page/physical page method.

```
log_to_phys_map_struct   STRUC
  log_page_number       DW  ?
  phys_page_number      DW  ?
log_to_phys_map_struct   ENDS
```

DX = handle

Contains the EMM handle.

CX = log_to_phys_map_len

Contains the number of entries in the array. For example, if the array contained four pages to map or unmap, then CX would contain 4. The number in CX should not exceed the number of mappable pages in the system.

DS:SI = pointer to log_to_phys_map array

Contains a pointer to an array of structures that contains the information necessary to map the desired pages. The array is made up of the following two elements:

.log_page_number

The first member is a word which contains the number of the logical page which is to be mapped. Logical pages are numbered zero relative, so the number for a logical page can only range from zero to (maximum number of logical pages allocated to the handle - 1).

If the logical page number is set to FFFFh, the physical page associated with it is unmapped rather than mapped. Unmapping a physical page makes it inaccessible for reading or writing.

.phys_page_number

The second member is a word which contains the number of the physical page at which the logical page is to be mapped. Physical pages are numbered zero relative, so the number for a physical page can only range from zero to

(maximum number of physical pages supported in the system - 1).

REGISTERS MODIFIED

AX

STATUS

AH = 0 SUCCESSFUL

The logical pages have been mapped, or unmapped, at the specified physical pages.

AH = 80h NON-RECOVERABLE

The manager has detected a malfunction in the memory manager software.

AH = 81h NON-RECOVERABLE

The manager has detected a malfunction in the expanded memory hardware.

AH = 83h NON-RECOVERABLE

The manager couldn't find the specified EMM handle. The manager doesn't currently have any information pertaining to the specified EMM handle. The program has probably corrupted its EMM handle.

AH = 84h NON-RECOVERABLE

The function code passed to the manager is not defined.

AH = 8Ah RECOVERABLE

One or more of the mapped logical pages is out of the range of logical pages allocated to the EMM handle. The program can recover by attempting to map a logical page which is within the bounds for the specified EMM handle. When this error occurs, the only pages mapped were the ones valid up to the point that the error occurred.

AH = 8Bh RECOVERABLE

One or more of the physical pages is out of the range of mappable physical pages, or the log_to_phys_map_len exceeds the number of mappable pages in the system. The program can recover from this condition by attempting to map into memory at a physical page which is in the range of the physical page numbers supported by the system. When this error occurs, the only pages mapped were the ones valid up to the point that the error occurred.

AH = 8Fh NON-RECOVERABLE

The subfunction parameter is invalid.

EXAMPLE

```
log_to_phys_map          log_to_phys_map_struct ? DUP (?)
emm_handle               DW ?
```

```
MOV  AX, SEG log_to_phys_map       ; DS:SI points to
```

54

```
MOV  DS, AX                        ; log_to_phys_map
LEA  SI, log_to_phys_map
MOV  CX, LENGTH log_to_phys_map    ; set length field
MOV  DX, emm_handle                ; load handle
MOV  AX, 5000h                     ; load function code
INT  67h                           ; call the memory manager
OR   AH, AH                        ; check EMM status
JNZ  emm_err_handler               ; jump to error handler on error
```

55

MAP/UNMAP MULTIPLE HANDLE PAGES

LOGICAL PAGE/SEGMENT ADDRESS METHOD

CALLING PARAMETERS

AX = 5001h

Contains the Map/Unmap Multiple Handle Pages subfunction using the logical page/segment address method.

```
log_to_seg_map_struct        STRUC
  log_page_number            DW  ?
  mappable_segment_address   DW  ?
log_to_seg_map_struct        ENDS
```

DX = handle

Contains the EMM handle.

CX = log_to_segment_map_len

Contains the number of entries in the array. For example, if the array contained four pages to map or unmap, then CX would contain four.

DS:SI = pointer to log_to_segment_map array

Contains a pointer to an array of structures that contains the information necessary to map the desired pages. The array is made up of the following elements:

.log_page_number

The first member is a word which contains the number of the logical pages to be mapped. Logical pages are numbered zero relative, so the number for a logical page can range from zero to (maximum number of logical pages allocated to the handle - 1).

If the logical page number is set to FFFFh, the physical page associated with it is unmapped rather than mapped. Unmapping a physical page makes it inaccessible for reading or writing.

.mappable_segment_address

The second member is a word which contains the segment address at which the logical page is to be mapped. This segment address must correspond exactly to a mappable segment address. The mappable segment addresses are available with Function 25 (Get Mappable Physical Address Array)

REGISTERS MODIFIED

AX

STATUS

AH = 0 SUCCESSFUL

The logical pages have been mapped (or unmapped), at the specified physical pages.

AH = 80h NON-RECOVERABLE

The manager has detected a malfunction in the memory manager software.

AH = 81h NON-RECOVERABLE

The manager has detected a malfunction in the expanded memory hardware.

AH = 83h NON-RECOVERABLE

The manager could not find the specified EMM handle. The manager doesn't have any information pertaining to the specified EMM handle. The program has probably corrupted its EMM handle.

AH = 84h NON-RECOVERABLE

The function code passed to the manager is not defined.

AH = 8Ah RECOVERABLE

One or more of the logical pages to be mapped is out of the range of logical pages allocated to the EMM handle. The program can recover from this condition by mapping a logical page which is within the bounds for the specified EMM handle. When this error occurs, the only pages mapped or unmapped were the ones valid up to the point that the error occurred.

AH = 8Bh RECOVERABLE

One or more of the mappable segment addresses specified is not mappable, the segment address doesn't fall exactly on a mappable address boundary, or the log_to_segment_map_len exceeds the number of mappable segments in the system. The program can recover from this condition by mapping into memory on an exact mappable segment address. When this error occurs, the only pages mapped were the ones valid up to the point that the error occurred.

AH = 8Fh NON-RECOVERABLE

The subfunction parameter is invalid.

EXAMPLE

```
log_to_seg_map                log_to_seg_map_struct 4 DUP (?)
emm_handle                    DW ?

MOV  AX, SEG log_to_seg_map
MOV  DS, AX
LEA  SI, log_to_seg_map          ; DS:SI points to log_to_seg_map
MOV  CX, LENGTH log_to_seg_map   ; load # of segments to
                                 ; map/unmap
MOV  DX, emm_handle              ; load handle
MOV  AX, 5001h                   ; load function code
INT  67h                         ; call the memory manager
OR   AH, AH                      ; check EMM status
```

57

```
JNZ  emm_err_handler             ; jump to error handler on error
```

58

FUNCTION 18 REALLOCATE PAGES

PURPOSE

This function allows an application program to increase or decrease (reallocate) the number of logical pages allocated to an EMM handle. There are four reallocation cases of interest:

1 A reallocation count of zero. The handle assigned to application remains assigned and is still available for use by the application. The memory manager won't reassign the handle to any other application. However, the handle will have any currently allocated pages returned to the memory manager. The application must invoke the Deallocate Pages function (Function 6) before returning to DOS, or the handle will remain assigned and no other application will be able to use it.

2 A reallocation count equal to the current allocation count. This is not treated as an error, and a successful status is returned.

3 A reallocation count greater than the current allocation count. The memory manager will attempt to add new pages to those pages already allocated to the specified EMM handle. The number of new pages added is the difference between the reallocation count and the current allocation count. The sequence of logical pages allocated to the EMM handle remains continuous after this operation. The newly allocated pages have logical page numbers which begin where the previously allocated pages ended, and continue in ascending sequence.

4 A reallocation count less than the current allocation count. The memory manager will attempt to subtract some of the currently allocated pages and return them to the memory manager. The number of old pages subtracted is the difference between the current allocation count and the re-allocation count. The pages are subtracted from the end of the sequence of pages currently allocated to the specified EMM handle. The sequence of logical pages allocated to the EMM handle remains continuous after this operation.

The handle determines what type of logical pages are being reallocated. Logical pages which were originally allocated with Function 4 or Function 27 (Allocate Standard Pages subfunction) are called pages and are 16K bytes long. Logical pages which were allocated with Function 27 (Allocate Raw Pages subfunction) are called raw pages and might not be the same size as pages allocated with Function 4 or Function 27 (Allocate Standard Pages subfunction).

CALLING PARAMETERS

AH = 51h
Contains the Reallocate Handle Pages function.

DX = handle
Contains the EMM handle.

BX = reallocation_count
Contains the total number of pages this handle should have allocated to it after this function is invoked.

RESULTS

These results are valid only if the status returned is zero.

BX = number of pages allocated to handle after reallocation
Contains the number of pages now allocated to the EMM handle after the pages have been added or subtracted. If the status returned is not zero, the value in BX is equal to the number of pages allocated to the handle prior to the invocation of this function. This information can be used to verify that the request generated the expected results.

REGISTERS MODIFIED

AX, BX

STATUS

AH = 00h SUCCESSFUL
The pages specified have been added or subtracted to or from the handle specified.

AH = 80h NON-Recoverable the manager has detected a malfunction in the memory manager software.

AH = 81h NON-Recoverable the manager has detected a malfunction in the expanded memory hardware.

AH = 83h NON-Recoverable the manager could not find the specified EMM handle. The manager doesn't have any information pertaining to the specified EMM handle. The program may have corrupted its EMM handle.

AH = 84h NON-Recoverable the function code passed to the manager is not defined.

AH = 87h Recoverable the number of pages that are available in the system is insufficient for the new allocation request. The program can recover from this condition by specifying fewer pages be allocated to the EMM handle.

AH = 88h Recoverable the number of unallocated pages is insufficient for the new allocation request. The program can recover from this condition by either requesting again when additional pages are available or specifying fewer pages.

EXAMPLE

```
emm_handle                  DW   ?
realloc_count               DW   ?
current_alloc_page_count    DW   ?

MOV  DX, emm_handle          ; specify EMM handle
```



```
MOV  BX, realloc_count       ; specify count
MOV  AH, 51h                 ; load function code
INT  67h                     ; call the memory manager
OR   AH, AH                  ; check EMM status
JNZ  emm_err_handler         ; jump to error handler on error
MOV  current_alloc_page_count, BX
```


FUNCTION 19 GET/SET HANDLE ATTRIBUTE

DESIGN CONSIDERATIONS

This function is an option which will probably not be available in a typical expanded memory manager, system, or memory board. Most personal computer systems disable memory refresh signals for a considerable period during a warm boot. This can corrupt some of the data in memory boards, even though there is no problem with the design of the memory board, its operation, or the memory chips. This memory refresh deficiency is present in the software design of the ROM BIOS in most personal computer systems.

The majority of memory board designs, chip types, or personal computer systems won't be able to support the non-volatility feature. The reason that this ROM BIOS deficiency is not evident in the conventional or extended memory area is that the ROM BIOS always initializes this area during a warm boot. Memory data integrity is not a problem with the conventional or extended memory region, because it isn't physically possible to have data retained there across a warm boot event -- the ROM BIOS sets it to zero.

Consequently, expanded memory board manufacturers should not supply this function unless their board can guarantee the integrity of data stored in the board's memory during a warm boot. Generally, this means the memory board has an independent memory refresh controller which does not depend on the system board's memory refresh.

If the expanded memory manager, system, or memory board cannot support this feature, it should return the not supported status described in the function.

FUNCTION 19 GET/SET HANDLE ATTRIBUTE (CONTINUED)

GET HANDLE ATTRIBUTE SUBFUNCTION

PURPOSE

This subfunction returns the attribute associated with a handle. The attributes are volatile or non-volatile. Handles with non-volatile attributes enable the memory manager to save the contents of a handle's pages between warm boots. However, this function may be disabled with a user option or may not be supported by the memory board or system hardware.

If the handle's attribute has been set to non-volatile, the handle, its name (if it is assigned one), and the contents of the pages allocated to the handle are all maintained after a warm boot.

CALLING PARAMETERS

AX = 5200h
Contains the Get Handle Attribute subfunction.

DX = handle
Contains the EMM handle.

RESULTS

These results are valid only if the status returned is zero.

AL = handle attribute

Contains the EMM handle's attribute. The only attributes a handle may have are volatile or non-volatile. A value of zero indicates the handle is volatile. A value of one indicates that the handle is non-volatile.

REGISTERS MODIFIED

AX

STATUS

AH = 0 SUCCESSFUL

The handles attribute has been obtained.

AH = 80h NON-RECOVERABLE

The manager has detected a malfunction in the memory manager software.

AH = 81h NON-RECOVERABLE

The manager has detected a malfunction in the expanded memory hardware.

AH = 83h NON-RECOVERABLE

The manager couldn't find the specified EMM handle. The manager doesn't have any information pertaining to the specified EMM handle. The program may have corrupted its EMM handle.

AH = 84h NON-RECOVERABLE

The function code passed to the manager is not defined.

AH = 8Fh NON-RECOVERABLE

The subfunction parameter is invalid.

AH = 91h NON-RECOVERABLE

This feature is not supported.

EXAMPLE

```
emm_handle              DW   ?
handle_attrib           DB   ?
```

```
MOV  DX, emm_handle          ; specify EMM handle
MOV  AX, 5200h               ; load function code
INT  67h                     ; call the memory manager
OR   AH, AH                  ; check EMM status
JNZ  emm_err_handler         ; jump to error handler on error
MOV  handle_attrib, AL       ; save handle attribute
```


FUNCTION 19 GET/SET HANDLE ATTRIBUTE (CONTINUED)

SET HANDLE ATTRIBUTE SUBFUNCTION

PURPOSE

This subfunction can be used to modify the attribute which a handle has associated with it. The attributes which a handle may have are volatile or non-volatile. The non-volatile attribute enables the EMM to save the contents of a handle's pages between warm boots. However, this function may be disabled with a user option or may not be supported by the memory board or system hardware.

If the handle's attribute has been set to non-volatile, the handle, its name (if it is assigned one), and the contents of the pages allocated to the handle are all maintained after a warm boot.

CALLING PARAMETERS

AX = 5201h
: Contains the Set Handle Attribute function.

DX = handle
: Contains the EMM handle.

BL = new handle attribute
: Contains the handle's new attribute. A value of zero indicates that the handle should be made volatile. A value of one indicates that the handle should be made non-volatile.

  A volatile handle attribute instructs the memory manager to deallocate both the handle and the pages allocated to it after a warm boot. If all handles have the volatile attribute (the default attribute) at warm boot, the handle directory will be empty and all of expanded memory will be initialized to zero immediately after a warm boot.

REGISTERS MODIFIED

AX

STATUS

AH = 0 SUCCESSFUL
The handles attribute has been modified.

AH = 80h NON-RECOVERABLE
The manager has detected a malfunction in the memory manager software.

AH = 81h NON-RECOVERABLE
The manager has detected a malfunction in the expanded memory hardware.

AH = 83h NON-RECOVERABLE
The manager could not find the specified EMM handle. The

manager doesn't have any information pertaining to the specified EMM handle. The program may have corrupted its EMM handle.

AH = 84h NON-RECOVERABLE
The function code passed to the manager is not defined.

AH = 8Fh NON-RECOVERABLE
The subfunction parameter is invalid.

AH = 90h NON-RECOVERABLE
The attribute type is undefined.

AH = 91h NON-RECOVERABLE
This feature is not supported.

EXAMPLE

```
emm_handle          DW  ?
new_handle_attrib   DB  ?
```

```
MOV  DX, emm_handle              ; specify EMM handle
MOV  BL, new_handle_attrib       ; specify the set attribute
MOV  AX, 5201h                   ; load function code
INT  67h                         ; call the memory manager
OR   AH, AH                      ; check EMM status
JNZ  emm_err_handler             ; jump to error handler on error
```

66

FUNCTION 19 GET/SET HANDLE ATTRIBUTE (CONTINUED)

GET ATTRIBUTE CAPABILITY SUBFUNCTION

PURPOSE

This subfunction can be used to determine whether the memory manager can support the non-volatile attribute.

CALLING PARAMETERS

AX = 5202h
Contains the Get Attribute Capability subfunction.

RESULTS

These results are valid only if the status returned is zero.

AL = attribute capability
Contains the attribute capability. A value of zero indicates that the memory manager and hardware supports only volatile handles. A value of one indicates that the memory manager/hardware supports both volatile and non-volatile handles.

REGISTERS MODIFIED

AX

STATUS

AH = 0 SUCCESSFUL
The attribute capability has been returned.

AH = 80h NON-RECOVERABLE
The manager has detected a malfunction in the memory manager software.

AH = 81h NON-RECOVERABLE
The manager has detected a malfunction in the expanded memory hardware.

AH = 84h NON-RECOVERABLE
The function code passed to the manager is not defined.

AH = 8Fh NON-RECOVERABLE
The subfunction parameter is invalid.

EXAMPLE

```
attrib_capability           DB  ?

MOV  AX, 5202h                ; load function code
INT  67h                    ;  call the memory manager
OR   AH, AH                   ; check EMM status
JNZ  emm_err_handler          ; jump to error handler on error
MOV  attrib_capability, AL    ; save attribute capability
```


FUNCTION 20 GET/SET HANDLE NAME

GET HANDLE NAME SUBFUNCTION

PURPOSE

This subfunction gets the eight character name currently assigned to a handle. There is no restriction on the characters which may be used in the handle name (that is, anything from 00h through FFh).

The handle name is initialized to ASCII nulls (binary zeros) three times: when the memory manager is installed, when a handle is allocated, and when a handle is deallocated. A handle with a name which is all ASCII nulls, by definition, has no name. When a handle is assigned a name, at least one character in the name must be a non-null character in order to distinguish it from a handle without a name.

CALLING PARAMETERS

AX = 5300h
Contains the Get Handle Name function.

DX = handle number
Contains the EMM handle.

ES:DI = pointer to handle_name array
Contains a pointer to an eight-byte array into which the name currently assigned to the handle will be copied.

RESULTS

These results are valid only if the status returned is zero.

handle_name array
Contains the name associated with the specified handle.

REGISTERS MODIFIED

AX

STATUS

AH = 0 SUCCESSFUL
The handle name has been returned.

AH = 80h NON-RECOVERABLE
The manager has detected a malfunction in the memory manager software.

AH = 81h NON-RECOVERABLE
The manager has detected a malfunction in the expanded memory hardware.

AH = 83h NON-RECOVERABLE
The manager couldn't find the specified EMM handle. The

manager doesn't have any information on the specified EMM handle. The program may have corrupted its EMM handle.

AH = 84h NON-RECOVERABLE
The function code passed to the manager is not defined.

AH = 8Fh NON-RECOVERABLE
The subfunction parameter is invalid.

EXAMPLE

```
handle_name             DB 8 DUP (?)
emm_handle              DW  ?
```

```
MOV  AX, SEG handle_name
MOV  ES, AX
LEA  DI, handle_name            ; ES:DI points to handle_name
MOV  DX, emm_handle             ; specify EMM handle
MOV  AX, 5300h                  ; load function code
INT  67h                        ; call the memory manager
OR   AH, AH                     ; check EMM status
JNZ  emm_err_handler            ; jump to error handler on error
```


FUNCTION 20 GET/SET HANDLE NAME

SET HANDLE NAME SUBFUNCTION

PURPOSE

This subfunction assigns an eight character name to a handle. There is no restriction on the characters which may be used in the handle name. The full range of values may be assigned to each character in a name (that is, 00h through FFh).

At installation, all handles have their name initialized to ASCII nulls (binary zeros). A handle with a name consisting of all ASCII nulls has no name. When a handle is assigned a name, at least one character in the name must be a non-null character in order to distinguish it from a handle without a name. No two handles may have the same name.

A handle can be renamed at any time by setting the handles name to a new value. A handle can have its name removed by setting the handle's name to all ASCII nulls. When a handle is deallocated, its name is removed (set to ASCII nulls).

CALLING PARAMETERS

AX = 5301h
: Contains the Set Handle Name function.

DX = handle number
: Contains the EMM handle.

DS:SI = pointer to handle_name
: Contains a pointer to a byte array which contains the name that is to be assigned to the handle. The handle name must be padded with nulls if the name is less than eight characters long.

REGISTERS MODIFIED

AX

STATUS

AH = 0 SUCCESSFUL
The handle name has been assigned.

AH = 80h NON-RECOVERABLE
The manager has detected a malfunction in the memory manager software.

AH = 81h NON-RECOVERABLE
The manager has detected a malfunction in the expanded memory hardware.

AH = 83h NON-RECOVERABLE
The manager couldn't find the specified EMM handle. The manager doesn't currently have any information pertaining

to the specified EMM handle. The program may have corrupted its EMM handle.

AH = 84h NON-RECOVERABLE
The function code passed to the manager is not defined.

AH = 8Fh NON-RECOVERABLE
The subfunction parameter is invalid.

AH = A1h RECOVERABLE
A handle with this name already exists. The specified handle was not assigned a name.

EXAMPLE

```
handle_name             DB "AARDVARK"
emm_handle              DW  ?
```

```
MOV  AX, SEG handle_name
MOV  DS, AX
LEA  SI, handle_name            ; DS:SI points to handle_name
MOV  DX, emm_handle             ; specify EMM handle
MOV  AX, 5301h                  ; load function code
INT  67h                        ; call the memory manager
OR   AH, AH                     ; check EMM status
JNZ  emm_err_handler            ; jump to error handler on error
```


FUNCTION 21 GET HANDLE DIRECTORY

GET HANDLE DIRECTORY SUBFUNCTION

PURPOSE

This function returns an array which contains all active handles and the names associated with each. Handles which have not been assigned names have a default name of all ASCII nulls (binary zeros). When a handle is first allocated, or when all the pages belonging to a handle are deallocated (that is, an open handle is closed), its default name is set to ASCII nulls. It takes a subsequent assignment of a name for a handle to have a name after it has been opened. The full range of values may be assigned to each character in a name (that is, 00h through FFh).

The number of bytes required by the array is:

10 bytes * total number of handles

The maximum size of this array is:

(10 bytes/entry) * 255 entries = 2550 bytes.

CALLING PARAMETERS

AX = 5400h
Contains the Get Handle Directory function.

```
handle_dir_struct   STRUC
  handle_value     DW  ?
  handle_name      DB  8  DUP  (?)
handle_dir_struct   ENDS
```

ES:DI = pointer to handle_dir

Contains a pointer to an area of memory into which the memory manager will copy the handle directory. The handle directory is an array of structures. There are as many entries in the array as there are open EMM handles. The structure consists of the following elements:

.handle_value

The first member is a word which contains the value of the open EMM handle.

.handle_name

The second member is an 8 byte array which contains the ASCII name associated with the EMM handle. If there is no name currently associated with the handle, it has a value of all zeros (ASCII nulls).

RESULTS

These results are valid only if the status returned is zero.

handle_dir

Contains the handle values and handle names associated with each handle value.

AL = number of entries in the handle_dir array

Contains the number of entries in the handle directory array. This is also the same as the number of open handles. For example, if only one handle is active, AL will contain a one.

REGISTERS MODIFIED

AX

STATUS

AH = 0 SUCCESSFUL

The handle directory has been returned.

AH = 80h NON-RECOVERABLE

The manager has detected a malfunction in the memory manager software.

AH = 81h NON-RECOVERABLE

The manager has detected a malfunction in the expanded memory hardware.

AH = 84h NON-RECOVERABLE

The function code passed to the manager is not defined.

AH = 8Fh NON-RECOVERABLE

The subfunction parameter is invalid.

EXAMPLE

```
handle_dir                  handle_dir_struct 255 DUP (?)
num_entries_in_handle_dir   DB ?

MOV  AX, SEG handle_dir
MOV  ES, AX
LEA  DI, handle_dir         ; ES:DI points to handle_dir
MOV  AX, 5400h              ; load function code
INT  67h                    ; call the memory manager
OR   AH, AH                 ; check EMM status
JNZ  emm_err_handler        ; jump to error handler on error
MOV  num_entries_in_handle_dir, AL ; save number of entries
```


FUNCTION 21 GET HANDLE DIRECTORY

SEARCH FOR NAMED HANDLE SUBFUNCTION

PURPOSE

This subfunction searches the handle name directory for a handle with a particular name. If the named handle is found, this subfunction returns the handle number associated with the name. At the time of installation, all handles have their names initialized to ASCII nulls. A handle with a name which is all ASCII nulls has, by definition, no name. When a handle is assigned a name, at least one character in the name must be a non-null character in order to distinguish it from a handle without a name.

CALLING PARAMETERS

AX = 5401h

Contains the Search for Named Handle subfunction.

DS:SI = handle_name
Contains a pointer to an 8-byte string that contains the name of the handle be searched for.

RESULTS

These results are valid only if the status returned is zero.

DX = value of named handle
The value of the handle which matches the handle name specified.

REGISTERS MODIFIED

AX, DX

STATUS

AH = 0 SUCCESSFUL
The handle value for the named handle has been found.

AH = 80h NON-RECOVERABLE
The manager has detected a malfunction in the memory manager software.

AH = 81h NON-RECOVERABLE
The manager has detected a malfunction in the expanded memory hardware.

AH = 84h NON-RECOVERABLE
The function code passed to the manager is not defined.

AH = 8Fh NON-RECOVERABLE
The subfunction parameter is invalid.

AH = A0h RECOVERABLE
No corresponding handle could be found for the handle name specified.

AH = A1h NON-RECOVERABLE
A handle found had no name (all ASCII nulls).

EXAMPLE

```
named_handle        DB  'AARDVARK'
named_handle_value      DW  ?
```

```
MOV  AX, SEG named_handle
MOV  DS, AX
LEA  SI, named_handle            ; DS:SI points to named_handle
MOV  AX, 5401h                   ; load function code
INT  67h                         ; call the memory manager
OR   AH, AH                      ; check EMM status
JNZ  emm_err_handler             ; jump to error handler on error
MOV  named_handle_value, DX      ; save value of named handle
```

75

FUNCTION 21 GET HANDLE DIRECTORY

GET TOTAL HANDLES SUBFUNCTION

PURPOSE

This subfunction returns the total number of handles that the memory manager supports, including the operating system handle (handle value 0).

CALLING PARAMETERS

AX = 5402h
Contains the Get Total Handles subfunction.

RESULTS

These results are valid only if the status returned is zero.

BX = total_handles
The value returned represents the maximum number of handles which a program may request the memory manager to allocate memory to. The value returned includes the operating system handle (handle value 0).

REGISTERS MODIFIED

AX, BX

STATUS

AH = 0 SUCCESSFUL
The total number of handles supported has been returned.

AH = 80h NON-RECOVERABLE
The manager has detected a malfunction in the memory manager software.

AH = 81h NON-RECOVERABLE

The manager has detected a malfunction in the expanded memory hardware.

AH = 84h NON-RECOVERABLE

The function code passed to the manager is not defined.

AH = 8Fh NON-RECOVERABLE

The subfunction parameter is invalid.

EXAMPLE

```
total_handles                DW     ?

MOV  AX, 5402h                ; load function code
INT  67h                     ; call the memory manager
OR   AH, AH                   ; check EMM status
JNZ  emm_err_handler          ; jump to error handler on error
MOV  total_handles, BX        ; save total handle count
```

76

FUNCTION 22 ALTER PAGE MAP & JUMP

PURPOSE

This function alters the memory mapping context and transfers control to the specified address. It is analogous to the FAR JUMP in the 8086 family architecture. The memory mapping context which existed before the invocation of this function is lost.

Mapping no pages and jumping is not considered an error. If a request to map zero pages and jump is made, control is transferred to the target address, and this function performs a far jump.

CALLING PARAMETERS

AH = 55h

Contains the Alter Page Map & Jump function.

```
log_phys_map_struct       STRUC
  log_page_number        DW ?
  phys_page_number_seg   DW ?
log_phys_map_struct       ENDS
```

```
map_and_jump_struct     STRUC
target_address          DD ?
log_phys_map_len        DB ?
log_phys_map_ptr        DD ?
map_and_jump_struct     ENDS
```

AL = physical page number/segment selector

Contains a code which indicates whether the value contained in the .log_phys_map.phys_page_number_seg are physical page numbers or are the segment address representation of the physical page numbers. A zero in AL indicates that the values are physical page numbers. A one in AL indicates that the values in these members are the segment address representations of the physical page numbers.

DX = handle number

Contains the EMM handle.

DS:SI = pointer to map_and_jump structure

Contains a pointer to a structure that contains the information necessary to map the desired physical pages and jump to the target address. The structure consists of the following elements:

.target_address

The first member is a far pointer which contains the target address to which control is to be transferred. The address is represented in segment:offset format. The offset portion of the address is stored in the low portion of the double word.

.log_phys_map_len

The second member is a byte which contains the number of entries in the array of structures which immediately

follows it. The array is as long as the application developer needs to map the desired logical pages into physical pages. The number of entries cannot exceed the number of mappable pages in the system.

.log_phys_map_ptr

The third member is a pointer to an array of structures which contain the logical page numbers and Alternate DMA register sets physical page numbers or segment addresses at which they are to be mapped. Each entry in the array of structures contains the following two elements:

.log_page_number

The first member of this structure is a word which contains the number of the logical page to be mapped.

.phys_page_number_seg

The second member of this structure is a word which contains either the physical page number or the segment address representation of the physical page number at which the previous logical page number is to be mapped. The value passed in AL determines the type of representation.

REGISTERS MODIFIED

AX

Note.....

Values in registers which don't contain required parameters maintain the values across the jump. The values in registers (with the exception of AX) and the flag state at the beginning of the function are still in the registers and flags when the target address is reached.

STATUS

AH = 0 SUCCESSFUL

Control has been transferred to the target address.

AH = 80h NON-RECOVERABLE

The manager has detected a malfunction in the memory manager software.

AH = 81h NON-RECOVERABLE

The manager has detected a malfunction in the expanded memory hardware.

AH = 83h NON-RECOVERABLE

The manager could not find the specified EMM handle. The manager does not currently have any information pertaining to the specified EMM handle. The program may have corrupted its EMM handle.

AH = 84h NON-RECOVERABLE

The function code passed to the manager is not defined.

78

AH = 8Ah RECOVERABLE

One or more of the logical pages to map into a corresponding physical page is out of the range of logical pages which are allocated to the EMM handle. The program can recover from this condition by mapping a logical page which is within the bounds for the EMM handle.

AH = 8Bh RECOVERABLE

One or more of the physical pages is out of the range of allowable physical pages, or the log_phys_map_len exceeds the number of mappable pages in the system. Physical page numbers are numbered zero relative. The program can recover from this condition by mapping into memory at a physical page which is in the range of supported physical pages.

AH = 8Fh RECOVERABLE

The subfunction parameter is invalid.

EXAMPLE

```
log_phys_map            log_phys_map_struct (?) DUP (?)
map_and_jump             map_and_jump_struct (?)
emm_handle              DW ?
phys_page_or_seg_mode       DB ?
```

```
MOV  AX, SEG map_and_jump
MOV  DS, AX
LEA  SI, map_and_jump            ; DS:SI points to map_and_jump
MOV  DX, emm_handle
MOV  AH, 55h                     ; load function code
MOV  AL, phys_page_or_seg_mode   ; specify physical page or seg
                                 ; mode
INT  67h                         ; call memory manager
OR   AH, AH                      ; check EMM status
JNZ  emm_err_handler             ; jump to error handler on error
```


FUNCTION 23 ALTER PAGE MAP & CALL

ALTER PAGE MAP & CALL SUBFUNCTION

PURPOSE

This subfunction saves the current memory mapping context, alters the specified memory mapping context, and transfers control to the specified address. It is analogous to the FAR CALL in the 8086 family architecture. Just as a return from a FAR CALL restores the original value in the code segment register, this subfunction restores the state of the specified mapping context after the return.

There is no explicit expanded memory subfunction which emulates a return from a FAR CALL. However, this facility is implicitly available through the standard return from a FAR CALL. The following paragraphs describe how this works:

After this function is invoked, unless an error is detected, the memory manager will transfer control to the address specified. If an error occurs, the memory manager returns immediately with the error code in the AH register. Otherwise, the memory manager pushes on the stack information which enables it to restore the mapping context after the return.

When the called procedure wants to return to the calling procedure, it simply issues a standard FAR RETURN. The memory manager traps this return, restores the specified mapping context, and returns to the calling procedure. The memory manager also returns a status from a successful return just as it does for all functions.

Developers using this subfunction must make allowances for the additional stack space this subfunction will use.

CALLING PARAMETERS

AH = 56h
Contains the Alter Page Map & Call function.

```
log_phys_map_struct       STRUC
  log_page_number         DW ?
  phys_page_number_seg    DW ?
log_phys_map_struct       ENDS

map_and_call_struct       STRUC
  target_address          DD ?
  new_page_map_len        DB ?
  new_page_map_ptr        DD ?
  old_page_map_len        DB ?
  old_page_map_ptr        DD ?
  reserved                DW  4 DUP (?)
map_and_call_struct       ENDS
```

AL = physical page number/segment selector
Contains a code which indicates whether the value contained in the

.new_page_map.phys_page_number_seg/.old_page_map.phys_page_number_seg members are physical pages numbers or are the segment address representation of the physical page numbers. A value of zero in AL indicates that the values in these members are physical page numbers. A value of one in AL indicates that the values in these members are the segment address representations of the physical page numbers.

DX = handle number
Contains the EMM handle.

DS:SI = pointer to map_and_call structure
Contains a pointer to a structure which contains the information necessary to map the desired physical pages and call the target address. The structure members are described here:

.target_address
: The first member is a far pointer which contains the target address to which control is to be transferred. The address is represented in segment:offset format. The offset portion of the address is stored in the low portion of the pointer. The application must supply this value.

.new_page_map_len
: The second member is a byte which contains the number of entries in the new mapping context to which new_page_map_ptr points. This number cannot exceed the number of mappable pages in the system.

.new_page_map_ptr
: The third member is a far pointer that points to an array of structures which contains a list of the logical page numbers and the physical page numbers/segments at which they are to be mapped immediately after the call. The contents of the new array of structures are described at the end of the map_and_call structure.

.old_page_map_len
: The fourth member is a byte which contains the number of entries in the old mapping context to which old_page_map_ptr points. This number cannot exceed the number of mappable pages in the system.

.old_page_map_ptr
: The fifth member is a far pointer that points to an array of structures which contains a list of the logical page numbers and the physical page numbers/segments at which they are to be mapped immediately after the return. The contents of the old array of structures are described at the end of the map_and_call structure.

.reserved
: The sixth member is reserved for use by the memory manager.

Each entry in the old and new array of structures contains two elements:

.log_page_number

The first member of this structure is a word which contains a logical page number which is to be mapped at the succeeding physical page number/segment immediately after the CALL (in the case of the new array of structures) or after the RETURN (in the case of the old array of structures).

.phys_page_number_seg

The second member of this structure is a word which contains either the physical page number or the segment address representation of the physical page number/segment at which the preceding logical page is to be mapped immediately after the CALL (in the case of the new array of structures) or after the RETURN (in the case of the old array of structures).

REGISTERS MODIFIED

AX

Note.....

Values in registers which don't contain required parameters maintain the values across the call. The values in registers (with the exception of AX) and the flag state at the beginning of the function are still in the registers and flags when the target address is reached.

STATUS

AH = 0 SUCCESSFUL

Control has been transferred to the target address.

AH = 80h NON-RECOVERABLE

The manager has detected a malfunction in the memory manager software.

AH = 81h NON-RECOVERABLE

The manager has detected a malfunction in the expanded memory hardware.

AH = 83h NON-RECOVERABLE

The manager couldn't find the specified EMM handle. The manager doesn't have any information pertaining to the specified EMM handle. The program may have corrupted its EMM handle.

AH = 84h NON-RECOVERABLE

The function code passed to the manager is not defined.

AH = 8Ah RECOVERABLE

One or more of the logical pages to map into a corresponding physical page is out of the range of logical pages which are allocated to the EMM handle. The program can recover from this condition by mapping a logical page which is within the bounds for the EMM handle.

AH = 8Bh RECOVERABLE

One or more of the physical pages is out of the range of allowable physical pages, or you've specified more physical pages than exist in the system. Physical page numbers are numbered zero relative. The program can recover from this condition by mapping a physical page which is in the range from zero to three.

AH = 8Fh NON-RECOVERABLE

The subfunction parameter is invalid.

EXAMPLE

```
new_page_map            log_phys_map_struct (?) DUP (?)
old_page_map            log_phys_map_struct (?) DUP (?)
map_and_call            map_and_call_struct (?)
emm_handle              DW ?
phys_page_or_seg_mode   DB ?

MOV  AX, SEG map_and_call
MOV  DS, AX
LEA  SI, map_and_call           ; DS:SI points to map_and_call
MOV  DX, emm_handle             ; specify EMM handle
MOV  AH, 56h                    ; load function code
MOV  AL, phys_page_or_seg_mode  ; specify physical page or seg
                                ; mode
INT  67h                        ; control is actually
                                ; transferred to the called
                                ; procedure at this point
OR   AH, AH                     ; check EMM status
JNZ  emm_err_handler            ; jump to error handler on error
```

FUNCTION 23 ALTER PAGE MAP & CALL

GET PAGE MAP STACK SPACE SIZE SUBFUNCTION

PURPOSE

Since the Alter Page Map & Call function pushes additional information onto the stack, this subfunction returns the number of bytes of stack space the function requires.

CALLING PARAMETERS

AX = 5602h
: Contains the Get Page Map Stack Space Size subfunction.

RESULTS

These results are valid only if the status returned is zero.

BX = stack space required
: Contains the number of bytes which the Alter Page Map & Call function will require. In other words, BX contains the number (including the return address) which has to be added to the stack pointer to remove all elements from the stack.

REGISTERS MODIFIED

AX, BX

STATUS

AH = 0 SUCCESSFUL
: The size of the array has been returned.

AH = 80h NON-RECOVERABLE
: The manager has detected a malfunction in the memory manager software.

AH = 81h NON-RECOVERABLE
: The manager has detected a malfunction in the expanded memory hardware.

AH = 84h NON-RECOVERABLE
: The function code passed to the manager is not defined.

AH = 8Fh NON-RECOVERABLE
: The subfunction parameter is invalid.

EXAMPLE

```
stack_space_reqd        DW ?

MOV  AX, 5602h          ; load function code
INT  67h                ; call the memory manager
OR   AH, AH             ; check EMM status
JNZ  emm_err_handler    ; jump to error handler on error
```

84

```
MOV  stack_space_reqd, BX   ; save required stack size count
```

85

FUNCTION 24 MOVE/EXCHANGE MEMORY REGION

MOVE MEMORY REGION SUBFUNCTION

PURPOSE

This subfunction copies a region of memory in the following memory source/destination combinations.

- o conventional memory to conventional memory
- o conventional memory to expanded memory
- o expanded memory to conventional memory
- o expanded memory to expanded memory

You do not have to save and restore the expanded memory mapping context to perform these move operations. The current mapping context is maintained throughout this operation.

The length of the region is limited by the amount of expanded memory allocated to the handles specified.

However, in most practical applications, the region length will be considerably smaller. A region length of zero is not an error, and no move will be performed.

A region length which exceeds 16K bytes is not an error. In this case the function assumes that a group of logical pages is the target for the move. The logical page specified represents the first logical page in which the move will take place. If the region length exceeds 16K bytes, or if the region is less than 16K bytes but spans logical pages, there must be sufficient logical pages remaining after the first logical page for the entire region to fit.

If your application needs to save a region of conventional memory in expanded memory, you can move it without having to perform a save or restore of the current mapping context.

The memory manager maintains the context. A move of up to 1M byte may be performed, although practical lengths are substantially less than this value.

If the source and destination handles are identical, the source and destination regions are tested for overlap before the move. If they overlap, the move direction is chosen so that the destination region receives an intact copy of the source region. A status will be returned indicating that this overlap has occurred.

CALLING PARAMETERS

AX = 5700h
Contains the Move Memory Region function.

```
move_source_dest_struct      STRUC
  region_length            DD  ?

                                        86

  source_memory_type       DB  ?
  source_handle            DW  ?
  source_initial_offset    DW  ?
  source_initial_seg_page  DW  ?
  dest_memory_type         DB  ?
  dest_handle              DW  ?
  dest_initial_offset      DW  ?
  dest_initial_seg_page    DW  ?
move_source_dest_struct      ENDS
```

DS:SI = pointer to exchange_source_dest structure
Contains a pointer to a data structure which contains the source and destination information for the exchange. The structure members are described here:

.region_length

The first member is a double word which specifies the length of the memory region (in bytes) to be moved.

.source_memory_type

The second member is a byte which specifies the type of memory where the source region resides. A value of zero indicates that the source region resides in conventional memory (excluding the page frame segment). A value of one indicates that the source region resides in expanded memory.

.source_handle

If the source region resides in expanded memory, the third member is a word which specifies the handle number associated with the source memory region. If the source region resides in conventional memory, this variable has no meaning and should be set to zero for future compatibility.

.source_initial_offset

The fourth member is a word which specifies the offset within the source region from which to begin the move. If the source region resides in expanded memory, the source_initial_offset is relative to the beginning of the 16K logical page. Because the offset is relative to the beginning of a 16K expanded memory page, it may only take on values between 0000h and 3FFFh.

If the source region resides in conventional memory, the source_initial_offset is a word which specifies the offset, relative to the beginning of the source segment, from which to begin the move. Because the offset is relative to the beginning of a 64K-byte conventional memory segment, it may take on values between 0000h and FFFFh.

.source_initial_seg_page

The fifth member is a word which specifies the initial segment or logical page number within the source region from which to begin the move. If the source region resides in expanded memory, the value specifies the logical page within the source region from which to begin the move. If the source region resides in conventional memory, the

source_initial_seg_page specifies the initial segment address within conventional memory from which to begin the move.

.dest_memory_type

The sixth member is a byte which specifies the type of memory where the destination region resides. A value of zero indicates conventional memory; a value of one indicates expanded memory.

.dest_handle

If the destination region resides in expanded memory, the seventh member is a word which specifies the handle number associated with the destination memory region. If the destination region resides in conventional memory, this variable has no meaning and should be set to zero for future compatibility.

.dest_initial_offset

The eighth member is a word which specifies the offset within the destination region from which to begin the move.

If the destination region resides in expanded memory, the dest_initial_offset is relative to the beginning of the 16K-byte logical page. Because the offset is relative to the beginning of a 16K-byte expanded memory page, it may take on values between 0000h and 3FFFh.

If the destination region resides in conventional memory, the dest_initial_offset is a word which specifies the offset, relative to the beginning of the destination segment, to begin the move. Because the offset is relative to the beginning of a 64K conventional memory segment, it may take on values between 0000h and FFFFh.

.dest_initial_seg_page

The ninth member is a word which specifies the initial segment or logical page number within the destination region from which to begin the move.

If the destination region resides in expanded memory then the value specifies the logical page within the destination region from which to begin the exchange.

If the destination region resides in conventional memory, the dest_initial_seg_page specifies the initial segment address within conventional memory from which to begin the move.

REGISTERS MODIFIED

AX

STATUS

AH = 0 SUCCESSFUL
The memory regions have been moved.

88

AH = 80h NON-RECOVERABLE
The manager has detected a malfunction in the memory manager software.

AH = 81h NON-RECOVERABLE
The manager has detected a malfunction in the expanded memory hardware.

AH = 83h NON-RECOVERABLE
The manager couldn't find either the source or destination EMM handles. The memory manager doesn't have any information on the handles specified. The program may have corrupted its EMM handles.

AH = 84h NON-RECOVERABLE
The function code passed to the manager is not defined.

AH = 8Ah NON-RECOVERABLE
One or more of the logical pages is out of the range of logical pages allocated to the source/destination handle.

AH = 8Fh NON-RECOVERABLE
The subfunction parameter is invalid.

AH = 92h SUCCESSFUL

The source and destination expanded memory regions have the same handle and overlap. This is valid for a move. The move has been completed and the destination region has a full copy of the source region. However, at least a portion of the source region has been overwritten by the move. Note that the source and destination expanded memory regions with different handles will never physically overlap because the different handles specify totally different regions of expanded memory.

AH = 93h CONDITIONALLY-RECOVERABLE

The length of the source or destination expanded memory region specified exceeds the length of the expanded memory region allocated either the source or destination handle. Insufficient pages are allocated to this handle to move a region of the size specified. The program can recover from this condition by allocating additional pages to the destination or source handle and attempting to execute the function again. However, if the application program allocated as much expanded memory as it thought it needed, this may be a program error and is not recoverable.

AH = 94h NON-RECOVERABLE

The conventional memory region and expanded memory region overlap. This is invalid, the conventional memory region cannot overlap the expanded memory region.

AH = 95h NON-RECOVERABLE

The offset within the logical page exceeds the length of the logical page. The initial source or destination offsets within an expanded memory region must be between 0000h and 3FFFh (16383 or (length of a logical page - 1)).

AH = 96h NON-RECOVERABLE

Region length exceeds 1M byte.

AH = 98h NON-RECOVERABLE

The memory source and destination types are undefined.

AH = A2h NON-RECOVERABLE

An attempt was made to wrap around the 1M-byte address space of conventional memory during the move. The combination of source/destination starting address and length of the region to be moved exceeds 1M byte. No data was moved.

EXAMPLE

```
move_source_dest                move_source_dest_struct (?)

MOV  AX, SEG move_source_dest
MOV  DS, AX
LEA  SI, move_source_dest         ; DS:SI points to
                                  ; move_source_dest
MOV  AX, 5700h                    ; load function code
INT  67h                          ; call the memory manager
OR   AH, AH                       ; check EMM status
JNZ  emm_err_handler              ; jump to error handler on error
```

90

FUNCTION 24 MOVE/EXCHANGE MEMORY REGION

EXCHANGE MEMORY REGION SUBFUNCTION

PURPOSE

This subfunction exchanges (using a string move) a region of memory in any of the following memory source/destination combinations.

- conventional memory to conventional memory
- conventional memory to expanded memory
- expanded memory to conventional memory
- expanded memory to expanded memory

The term expanded memory region refers only to the area of memory above 640K bytes (9FFFFh). If a system provides mappable conventional memory, this function treats the mappable conventional memory regions as ordinary conventional memory. The contents of the source region and the destination region are exchanged.

The exchange operation can be performed without having to save and restore the expanded memory mapping context. The current mapping context is maintained throughout this operation. The length of the region is limited to the amount of expanded memory allocated to the specified EMM handles. A length of zero is not an error; however, no exchange will be performed. A region length which exceeds 16K bytes is not an error. In this case the function assumes that a group of logical pages is the target for the exchange. The logical page specified represents the first logical page in which the exchange will take place. If the region length exceeds 16K bytes, or if the region is less than 16K bytes but spans logical pages, there must be sufficient logical pages remaining after the first logical page for the entire region to fit.

If your application needs to exchange a region of conventional memory with expanded memory, you can simply exchange it with the region of interest without having to perform a save or restore of the current mapping context.

An exchange of up to 1M byte may be performed, although practical lengths are obviously below that value. Checking is done before starting the exchange to prevent the possibility of overlap during the exchange operation. Overlapping source and destination regions for a exchange are invalid, and the exchange will not take place.

CALLING PARAMETERS

AX = 5701h
Contains the Exchange Memory Region function.

```
xchg_source_dest_struct      STRUC
  region_length            DD ?
  source_memory_type       DB ?
  source_handle            DW ?
```



```
  source_initial_offset    DW ?
  source_initial_seg_page  DW ?
  dest_memory_type         DB ?
  dest_handle              DW ?
  dest_initial_offset      DW ?
  dest_initial_seg_page    DW ?
xchg_source_dest_struct      ENDS
```

DS:SI = pointer to move_source_dest structure

Contains a pointer to the data structure which contains the source & destination information for the exchange. The structure members are described here:

.region_length

The first member is a double word which specifies the length of the memory region to be exchanged.

.source_memory_type

The second member is a byte which specifies the type of memory where the source region resides. A value of zero indicates that the source region resides in conventional memory. A value of one indicates that the source region resides in expanded memory.

.source_handle

If the source region resides in expanded memory, the third member is a word which specifies the handle number associated with the source memory region. If the source region resides in conventional memory, this variable has no meaning and should be set to zero for future compatibility.

.source_initial_offset

The fourth member is a word which specifies the offset within the source region from which to begin the exchange.

If the source region resides in expanded memory, the source_initial_offset is relative to the beginning of the 16K-byte logical page. Because the offset is relative to the beginning of a 16K-byte expanded memory page, it may only take on values between 0000h and 3FFFh.

If the source region resides in conventional memory, the source_initial_offset is a word which specifies the offset, relative to the beginning of the source segment, to begin the exchange at. Because the offset is relative to the beginning of a 64K-byte conventional memory segment, it may only take on values between 0000h and FFFFh.

.source_initial_seg_page

The fifth member is a word which specifies the initial segment or logical page number within the source region from which to begin the exchange.

If the source region resides in expanded memory then the value specifies the logical page within the source region from which to begin the exchange.

If the source region resides in conventional memory, the source_initial_seg_page specifies the initial segment address within conventional memory from which to begin the exchange.

.dest_memory_type

The sixth member is a byte which specifies the type of memory where the destination region resides. A value of zero indicates that the destination region resides in conventional memory (excluding the page frame segment). A value of one indicates that the destination region resides in expanded memory.

.dest_handle

If the destination region resides in expanded memory, the seventh member is a word which specifies the handle number associated with the destination memory region. If the destination region resides in conventional memory , then this variable has no meaning and should be set to zero for future compatibility.

.dest_initial_offset

The eighth member is a word which specifies the offset within the destination region from which to begin the exchange.

If the destination region resides in expanded memory, the dest_initial_offset is relative to the beginning of the 16K-byte logical page. Because the offset is relative to the beginning of a 16K-byte expanded memory page, it may only take on values between 0000h and 3FFFh.

If the destination region resides in conventional memory, the dest_initial_offset is a word which specifies the offset, relative to the beginning of the destination segment, to begin the exchange at. Because the offset is relative to the beginning of a 64K-byte conventional memory segment, it may only take on values between 0000h and FFFFh.

.dest_initial_seg_page

The ninth member is a word which specifies the initial segment or logical page number within the destination region from which to begin the exchange.

If the destination region resides in expanded memory, the value specifies the logical page within the destination region from which to begin the exchange.

If the destination region resides in conventional memory, the dest_initial_seg_page specifies the initial segment address within conventional memory from which to begin the exchange.

REGISTERS MODIFIED

AX

STATUS

AH = 0 SUCCESSFUL

The memory regions have been exchanged.

AH = 80h NON-RECOVERABLE

The manager has detected a malfunction in the memory manager software.

AH = 81h NON-RECOVERABLE

The manager has detected a malfunction in the expanded memory hardware.

AH = 83h NON-RECOVERABLE

The manager could not find either the source or destination EMM handles. The memory manager does not currently have any information pertaining to the handles specified. The program may have corrupted its EMM handles.

AH = 84h NON-RECOVERABLE

The function code passed to the manager is not defined.

AH = 8Ah NON-RECOVERABLE

One or more of the logical pages is out of the range of logical pages allocated to the source/destination handle.

AH = 8Fh NON-RECOVERABLE

The subfunction parameter is invalid.

AH = 93h CONDITIONALLY RECOVERABLE

The length of the source or destination expanded memory region specified, exceeds the length of the expanded memory region allocated to the source or destination specified EMM handle. There are insufficient pages allocated to this handle to exchange a region of the size specified. The program can recover from this condition by attempting to allocate additional pages to the destination or source handle and attempting to execute the function again. However, if the application program was allocated as much expanded memory as it thought it needed, this may be a program error and is therefore not recoverable.

AH = 94h NON-RECOVERABLE

The conventional memory region and expanded memory region overlap. This is invalid, the conventional memory region cannot overlap the expanded memory region.

AH = 95h NON-RECOVERABLE

The offset within the logical page exceeds the length of the logical page. The initial source or destination offsets within an expanded memory region must be between 0000h and 3FFFh (16383 or (length of a logical page - 1)).

AH = 96h NON-RECOVERABLE

Region length exceeds 1M Byte limit.

AH = 97h NON-RECOVERABLE

The source and destination expanded memory regions have the same handle and overlap. This is invalid, the source and destination expanded memory regions cannot have the same handle and overlap when they are being exchanged. Note that the source and destination expanded memory regions which have different handles will never physically overlap because the different handles specify totally different regions of expanded memory.

AH = 98h NON-RECOVERABLE

The memory source and destination types are undefined.

AH = A2h NON-RECOVERABLE

An attempt was made to wrap around the 1M-byte address space of conventional memory during the exchange. The source starting address together with the length of the region to be exchanged exceeds 1M byte. No data was exchanged.

EXAMPLE

```
xchg_source_dest                xchg_source_dest_struct (?)

MOV  AX, SEG xchg_source_dest
MOV  DS, AX
LEA  SI, xchg_source_dest         ; DS:SI points to
                                  ; xchg_source_dest
MOV  AX, 5701h                    ; load function code
INT  67h                          ; call the memory manager
OR   AH, AH                       ; check EMM status
JNZ  emm_err_handler              ; jump to error handler on error
```


FUNCTION 25 GET MAPPABLE PHYSICAL ADDRESS ARRAY

GET MAPPABLE PHYSICAL ADDRESS ARRAY SUBFUNCTION

PURPOSE

This subfunction returns an array containing the segment address and physical page number for each mappable physical page in a system. The contents of this array provide a cross reference between physical page numbers and the actual segment addresses for each mappable page in the system. The array is sorted in ascending segment order. This does not mean that the physical page numbers associated with the segment addresses are also in ascending order.

CALLING PARAMETERS

AX = 5800h

Contains the Get Mappable Physical Address Array subfunction.

```
mappable_phys_page_struct   STRUC
  phys_page_segment       DW ?
  phys_page_number        DW ?
mappable_phys_page_struct   ENDS
```

ES:DI = mappable_phys_page

Contains a pointer to an application-supplied memory area where the memory manager will copy the physical address array. Each entry in the array is a structure containing two members:

.phys_page_segment

The first member is a word which contains the segment address of the mappable physical page associated with the physical page number following it. The array entries are sorted in ascending segment address order.

.phys_page_number

The second member is a word which contains the physical page number which corresponds to the previous segment address. The physical page numbers are not necessarily in ascending order.

EXAMPLE 1

An expanded memory board has its page frame starting at address C0000h and has no mappable conventional memory. For this configuration, physical page 0 corresponds to segment address C000h, physical page 1 corresponds to segment address C400h, etc. The array would contain the following data (in this order):

C000h, 00h
C400h, 01h
C800h, 02h
CC00h, 03h

EXAMPLE 2

An expanded memory board has a large page frame starting at address C0000h and has mappable conventional memory from 90000h through 9FFFFh. For this configuration, physical page 0 corresponds to segment address C000h, physical page 1 corresponds to segment address C400h, etc. The array would contain the following data in the order specified. Note that the expanded memory region always has the lowest numerically valued physical page numbers.

9000h, 0Ch
9400h, 0Dh
9800h, 0Eh
9C00h, 0Fh
C000h, 00h
C400h, 01h
C800h, 02h
CC00h, 03h
D000h, 04h
D400h, 05h
D800h, 06h
DC00h, 07h
E000h, 08h
E400h, 09h
E800h, 0Ah
EC00h, 0Bh

RESULTS

These results are valid only if the status returned is zero.

CX = number of entries in the mappable_phys_page
Multiply this number by (SIZE mappable_phys_page_struct) to determine the number of bytes the physical page address array requires.

REGISTERS MODIFIED

AX, CX

STATUS

AH = 0 SUCCESSFUL
The hardware configuration array has been returned.

AH = 80h NON-RECOVERABLE
The manager has detected a malfunction in the memory manager software.

AH = 81h NON-RECOVERABLE
The manager has detected a malfunction in the expanded memory hardware.

AH = 84h NON-RECOVERABLE
The function code passed to the manager is not defined.

AH = 8Fh NON-RECOVERABLE
The subfunction parameter is invalid.

97

EXAMPLE

```
mappable_phys_page              mappable_phys_page_struct (?)

MOV  AX, SEG mappable_phys_page
MOV  ES, AX
LEA  DI, mappable_phys_page       ; ES:DI points to
                                  ; mappable_phys_page
MOV  AX, 5800h                    ; load function code
INT  67h                          ; call the memory manager
OR   AH, AH                       ; check EMM status
JNZ  emm_err_handler              ; jump to error handler on error
MOV  mappable_page_entry_count, CX ; save mappable page entry count
```

98

FUNCTION 25 GET MAPPABLE PHYSICAL ADDRESS ARRAY

GET MAPPABLE PHYSICAL ADDRESS ARRAY ENTRIES SUBFUNCTION

PURPOSE

This subfunction gets the number of entries which will be required for the array the first subfunction returns.

CALLING PARAMETERS

AX = 5801h
Contains the Get Physical Page Address Array Entry Count subfunction. This subfunction returns a word which represents the number of entries in the array returned by the previous subfunction. This number also represents the number of mappable physical pages in a system.

RESULTS

These results are valid only if the status returned is zero.

CX = number of entries in the mappable_phys_page
Multiply this number by (SIZE mappable_phys_page_struct) to determine the number of bytes the physical page address array will require.

REGISTERS MODIFIED

AX, CX

STATUS

AH = 0 SUCCESSFUL
The physical page address array has been returned.

AH = 80h NON-RECOVERABLE
The manager has detected a malfunction in the memory manager software.

AH = 81h NON-RECOVERABLE
The manager has detected a malfunction in the expanded memory hardware.

AH = 84h NON-RECOVERABLE
The function code passed to the manager is not defined.

AH = 8Fh NON-RECOVERABLE
The subfunction parameter is invalid.

EXAMPLE

```
mappable_page_entry_count       DW ?

MOV  AX, 5801h                  ; load function code
INT  67h                        ; call memory manager
OR   AH, AH                     ; check EMM status
```

99

```
JNZ  emm_err_handler            ; jump to error handler on error
MOV  mappable_page_entry_count, CX ; save mappable page entry count
```

100

FUNCTION 26 GET EXPANDED MEMORY HARDWARE INFORMATION

GET HARDWARE CONFIGURATION ARRAY SUBFUNCTION

Note.....

This function is for use by operating systems only. This function can be disabled at any time by the operating system. Refer to Function 30 for a description of how an operating system does this.

PURPOSE

This subfunction returns an array containing expanded memory hardware configuration information for use by an operating system/environment.

CALLING PARAMETERS

AX = 5900h

Contains the Get Hardware Configuration Array subfunction.

```
hardware_info_struct        STRUC
  raw_page_size             DW ?
  alternate_register_sets   DW ?
  context_save_area_size    DW ?
  DMA_register_sets         DW ?
  DMA_channel_operation     DW ?
hardware_info_struct        ENDS
```

ES:DI = hardware_info

Contains a pointer to a memory area that the operating system supplies where the memory manager will copy expanded memory hardware information. The structure contains these five members:

.raw_page_size

The first member is a word which contains the size of a raw mappable physical page in paragraphs (16 bytes). LIM standard pages are always 16K bytes. However, other implementations of expanded memory boards do not necessarily comply with this standard and can emulate a 16K-byte page by mapping in multiple smaller pages. This member specifies the size of a mappable physical page viewed from the hardware implementation level.

.alternate_register_sets

The second member is a word which specifies the number of alternate mapping register sets. The additional mapping register sets are termed alternate mapping register sets in this document.

All expanded memory boards have at least one set of hardware registers to perform the logical to physical page mapping. Some expanded memory boards have more than one set of these mapping registers. This member specifies how many of these alternate mapping register sets exist (beyond the one set that all expanded memory boards have) on the expanded memory boards in the system. If an expanded

memory card has only one set of mapping registers (that is, no alternate mapping register sets) this member has a value of zero.

.context_save_area_size

The third member is a word which contains the storage requirements for the array required to save a mapping context. The value returned in this member is exactly the same as that returned by Function 15 (Get Size of Page Map Save Array subfunction).

.DMA_channels

The fourth member is a word which contains the number of register sets that can be assigned to DMA channels. These DMA register sets, although similar in use to alternate register sets, are for DMA mapping and not task mapping.

If the expanded memory hardware does not support DMA register sets, care must be taken when DMA is taking place.

In a multitasking operating system, when one task is waiting for DMA to complete, it is useful to be able to switch to another task. However, if the DMA is taking place in memory that the second task will need to remap, remapping would be disastrous.

If the expanded memory hardware can detect when DMA is occurring, the OS/E should allow task switches and remapping during DMA. If no special support for DMA is available, no remapping should be done when DMA is in progress.

.DMA_channel_operation

The fifth member is a word which specifies a special case for the DMA register sets. A value of zero specifies that the DMA register sets behave as described in Function 28. A value of one specifies that the expanded memory hardware has only one DMA register set. In addition, if any channel is mapped through this register set, then all channels are mapped through it. For LIM standard boards, this value is zero.

RESULTS

These results are valid only if the status returned is zero.

hardware_info
Contains the expanded memory hardware specific information described above.

REGISTERS MODIFIED

AX

STATUS

AH = 0 SUCCESSFUL
The hardware configuration array has been returned.

AH = 80h NON-RECOVERABLE
The manager has detected a malfunction in the memory manager software.

AH = 81h NON-RECOVERABLE
The manager has detected a malfunction in the expanded memory hardware.

AH = 84h NON-RECOVERABLE
The function code passed to the manager is not defined.

AH = 8Fh NON-RECOVERABLE
The subfunction parameter is invalid.

AH = A4h NON-RECOVERABLE
Access to this function has been denied by the operating system. The function cannot be used at this time.

EXAMPLE

```
hardware_info                hardware_info_struct (?)

MOV  AX, SEG hardware_info
MOV  ES, AX
LEA  DI, hardware_info          ; ES:DI points to hardware_info
MOV  AX, 5900h                  ; load function code
INT  67h                    ; call the memory manager
OR   AH, AH                     ; check EMM status
JNZ  emm_err_handler            ; jump to error handler on error
```

FUNCTION 26 GET EXPANDED MEMORY HARDWARE INFORMATION

GET UNALLOCATED RAW PAGE COUNT SUBFUNCTION

PURPOSE

The Get Unallocated Raw Page Count subfunction returns the number of unallocated non-standard length mappable pages as well as the total number of non-standard length mappable pages in expanded memory to the operating system.

One variety of expanded memory board has a page size which is a sub-multiple of 16K bytes. An expanded memory page which is a sub-multiple of 16K is termed a raw page. An operating system may deal with mappable physical page sizes which are sub-multiples of 16K bytes.

If the expanded memory board supplies pages in exact multiples of 16K bytes, the number of pages this function returns is identical to the number Function 3 (Get Unallocated Page Count) returns. In this case, there is no difference between a page and a raw page.

CALLING PARAMETERS

AX = 5901h
: Contains the Get Unallocated Raw Page Count subfunction.

RESULTS

These results are valid only if the status returned is zero.

BX = unallocated raw pages
: The number of raw pages that are currently available for use.

DX = total raw pages
: The total number of raw pages in expanded memory.

REGISTERS MODIFIED

AX, BX, DX

STATUS

AH = 0 SUCCESSFUL
: The manager has returned the number unallocated raw pages and the number of total raw pages in expanded memory.

AH = 80h NON-RECOVERABLE

The manager detected a malfunction in the memory manager software.

AH = 81h NON-RECOVERABLE

The manager detected a malfunction in the expanded memory hardware.

104

AH = 84h NON-RECOVERABLE

The function code passed to the memory manager is not defined.

AH = 8Fh NON-RECOVERABLE

The subfunction parameter is invalid.

EXAMPLE

```
unalloc_raw_pages               DW ?
total_raw_pages                 DW ?

MOV  AX, 5901h                  ; load function code
INT  67h                        ; call the memory manager
OR   AH, AH                     ; check EMM status
JNZ  emm_err_handler            ; jump to error handler on error
MOV  unalloc_raw_pages, BX      ; save unallocated raw page
                                ; count
MOV  total_raw_pages, DX        ; save total raw page count
```

105

FUNCTION 27 ALLOCATE STANDARD/RAW PAGES

ALLOCATE STANDARD PAGES SUBFUNCTION

PURPOSE

The Allocate Standard Pages subfunction allocates the number of standard size (16K bytes) pages that the operating system requests and assigns a unique EMM handle to these pages. The EMM handle owns these pages until the operating system deallocates them. This subfunction allows you to allocate zero pages to a handle, unlike Function 4 (Allocate Pages).

Note.....

This note affects expanded memory manager implementers and operating system developers only. Applications should not use the following characteristic of the memory manager. An application violating this rule will be incompatible with future versions of Microsoft's operating systems and environments.

To be compatible with this specification, an expanded memory manager will provide a special handle which is available to the operating system only. This handle will have a value of 0000h and will have a set of pages allocated to it when the expanded memory manager driver installs. The pages that the memory manager will automatically allocate to handle 0000h are those that backfill conventional memory. Typically, this backfill occurs between addresses 40000h (256K) and 9FFFFh (640K). However, the range can extend below and above this limit if the hardware and memory manager have the capability.

An operating system won't have to invoke Function 27 to obtain this handle because it can assume the handle already exists and is available for use immediately after the expanded memory device driver installs. When an operating system wants to use this handle, it uses the special handle value of 0000h. The operating system will be able to invoke any EMM function using this special handle value. To allocate pages to this handle, the operating system need only invoke Function 18 (Reallocate pages).

There are two special cases for this handle:

1. Function 27 (Allocate Standard Pages). This function must never return zero as a handle value. Applications must always invoke Function 27 to allocate pages and obtain a handle which identifies the pages which belong to it. Since Function 27 never returns a handle value of zero, an application will never gain access to this special handle.

2. Function 6 (Deallocate Pages). If the operating system uses it to deallocate the pages which are allocated to this handle, the pages the handle owns will be returned to the manager for use. But the handle will not be available for reassignment. The manager should treat a deallocate pages function request for this handle the same as a reallocate pages function request, where the number of pages to reallocate to this handle is zero.

CALLING PARAMETERS

AX = 5A00h

Contains the Allocate Standard Pages subfunction.

BX = num_of_standard_pages_to_alloc

Contains the number of standard pages the operating system wants to allocate.

RESULTS

These results are valid only if the status returned is zero.

DX = handle

Contains a unique EMM handle. The operating system must use this EMM handle as a parameter in any function that requires it. Up to 255 handles may be obtained. (Both Function 27 and Function 4 must share the same 255 handles).

For all functions using this handle, the length of the physical and logical pages allocated to it are standard length (that is, 16K bytes).

REGISTERS MODIFIED

AX, DX

STATUS

AH = 0 SUCCESSFUL

The manager has allocated the pages to an assigned EMM raw handle.

AH = 80h NON-RECOVERABLE

The manager detected a malfunction in the memory manager software.

AH = 81h NON-RECOVERABLE

The manager detected a malfunction in the expanded memory hardware.

AH = 84h NON-RECOVERABLE

The function code passed to the memory manager is not defined.

AH = 85h RECOVERABLE

All EMM handles are being used.

AH = 87h RECOVERABLE

There aren't enough expanded memory pages present in the system to satisfy the operating system's request.

AH = 88h RECOVERABLE

There aren't enough unallocated pages to satisfy the operating system's request.

AH = 8Fh NON-RECOVERABLE

The subfunction parameter is invalid.

EXAMPLE

```
num_of_standard_pages_to_alloc     DW ?
emm_handle                         DW ?

MOV  BX, num_of_standard_pages_to_alloc
MOV  AX, 5A00h                      ; load function code
INT  67h                          ; call the memory manager
OR   AH, AH                         ; check EMM status
JNZ  emm_err_handler                ; jump to error handler on error
MOV  emm_handle, DX                 ; save handle
```


FUNCTION 27 ALLOCATE STANDARD/RAW PAGES

ALLOCATE RAW PAGES SUBFUNCTION

PURPOSE

The Allocate Raw Pages subfunction allocates the number of non-standard size pages that the operating system requests and assigns a unique EMM handle to these pages. The EMM handle owns these pages until the operating system deallocates them. This function allows you to allocate zero pages to a handle, unlike Function 4 (Allocate Pages).

A hardware vendor may design an expanded memory board that has a page size which is a sub-multiple of 16K bytes. A physical page which is a sub-multiple of 16K is termed a raw page. The operating system may deal with page sizes which are sub-multiples of 16K bytes. The memory manager must treat any function using a handle with raw pages allocated to it by Allocate Raw Pages differently than it does a handle that has normal 16K-byte pages allocated to it.

Handles which are assigned using Function 4 (Allocate pages) or Function 27 (Allocate Standard Pages subfunction) must have pages which are 16K bytes -- this is the length of a standard expanded memory page. If the expanded memory board hardware is not able to supply 16K-byte pages, the memory manager must emulate pages which are 16K bytes combining multiple non-standard size pages to form a single 16K-byte page.

Handles which are assigned using Function 27 (Allocate Raw Pages) are called raw handles. All logical pages allocated to a raw handle may have a non-standard length (that is, not 16K bytes). However, once the operating system has allocated a number of raw pages to a handle, it is the responsibility of the memory manager to recognize that raw handle as one that has non-standard size pages allocated to it. The memory manager must identify these handles and treat all functions which use handles which have non-standard page lengths differently. The logical page length becomes the length of the non-standard size page for any raw handle that Function 27 assigns.

Note.....

This note affects expanded memory manager implementers and operating system developers only. Applications should not use the following characteristic of the memory manager. An application violating this rule will be incompatible with future versions of Microsoft's operating systems and environments.

To be compatible with this specification, an expanded memory manager will provide a special handle which is available to the operating system only. This handle will have a value of 0000h and will have a set of pages allocated to it when the expanded memory manager driver installs. The pages that the memory manager will automatically allocate to handle 0000h are those that backfill conventional memory. Typically, this backfill occurs between addresses 40000h (256K) and 9FFFFh (640K). However, the range can extend below and above this limit if the hardware and memory manager have the capability.

An operating system won't have to invoke Function 27 to obtain this handle because it can assume the handle already exists and is available for use immediately after the expanded memory device driver installs. When an operating system wants to use this handle, it uses the special handle value of 0000h. The operating system will be able to invoke any EMM function using this special handle value. To allocate pages to this handle, the operating system need only invoke Function 18 (Reallocate pages).

There are two special cases for this handle:

1 Function 27 (Allocate Raw Pages). This function must never return zero as a handle value. Applications must always invoke Function 27 to allocate pages and obtain a handle which identifies the pages which belong to it. Since Function 27 never returns a handle value of zero, an application will never gain access to this special handle.

2 Function 6 (Deallocate Pages). If the operating system uses it to deallocate the pages which are allocated to this special handle, the pages the handle owns will be returned to the manager for use. But the handle will not be available for reassignment. The manager should treat a deallocate pages function request for this handle the same as a reallocate pages function request, where the number of pages to reallocate to this handle is zero.

CALLING PARAMETERS

AX = 5A01h
Contains the Allocate Raw Pages subfunction.

BX = num_of_raw_pages_to_alloc
Contains the number of raw pages the operating system wants to allocate.

RESULTS

These results are valid only if the status returned is zero.

DX = raw handle
Contains a unique EMM raw handle. The operating system must use this EMM raw handle as a parameter in any function that requires it. Up to 255 handles may be obtained. (Both Function 4 and Function 27 must share the same 255 handles).

For all functions using this raw handle, the length of the physical and logical pages allocated to it may be non-standard (that is, not 16K bytes).

REGISTERS MODIFIED

AH = 0 SUCCESSFUL
The manager has allocated the raw pages to an assigned EMM raw handle.

AH = 80h NON-RECOVERABLE
The manager detected a malfunction in the memory manager software.

AH = 81h NON-RECOVERABLE
The manager detected a malfunction in the expanded memory hardware.

AH = 84h NON-RECOVERABLE
The function code passed to the memory manager is not defined.

AH = 85h RECOVERABLE
All EMM handles are being used.

AH = 87h RECOVERABLE
There aren't enough expanded memory raw pages present in the system to satisfy the operating system's request.

AH = 88h RECOVERABLE
There aren't enough unallocated raw pages to satisfy the operating system's request.

AH = 8Fh NON-RECOVERABLE
The subfunction parameter is invalid.

EXAMPLE

```
num_of_raw_pages_to_alloc       DW ?
emm_raw_handle                  DW ?
```

```
MOV  BX, num_of_raw_pages_to_alloc
MOV  AX, 5A01h                  ; load function code
INT  67h                        ; call the memory manager
OR   AH, AH                     ; check EMM status
JNZ  emm_err_handler            ; jump to error handler on error
MOV  emm_raw_handle, DX         ; save raw handle
```


FUNCTION 28 ALTERNATE MAP REGISTER SET

Note.....

This function is for use by operating systems only. The operating system can disable this function at any time. Refer to Function 30 for a description of how an operating system can enable or disable this function.

DESIGN CONSIDERATIONS

The hardware support for the entire set of subfunctions described is generally not present on every expanded memory board from every vendor of expanded memory board products. For some of the subfunctions, software emulation is provided. For other subfunctions, a certain protocol in their use must be observed. The subfunctions for which this is most crucial are those which address system DMA capabilities.

System DMA Capabilities & Expanded Memory Support of DMA

In a multitasking operating system, when one task is waiting for DMA to complete, it is useful to be able to switch to another task. This specification describes a capability which may be designed into expanded memory boards to provide DMA into memory regions which may be mapped out while the DMA is occurring. For expanded memory boards that do not provide this, it is crucial to understand that while DMA is in progress into a region of mappable memory, the memory mapping context cannot be changed. That is, all DMA action must be complete before any remapping of pages can be done.

Expanded Memory Support of DMA Register Sets

Expanded memory boards which have DMA registers sets could support DMA into a region of mappable memory while the memory mapping context is being switched. It is important to realize that these DMA register sets are separate from the alternate map register sets. An example of how an OS/E might use DMA registers sets follows:

EXAMPLE 1

1. Allocate a DMA register set.
2. Get current register set.
3. Set the DMA register set.
4. Map in the memory desired.
5. Get the DMA register set.
6. Set the original register set.
7. Assign the desired DMA channel to the DMA register set.

The preceding set of calls makes all DMA accesses for the desired DMA channel get mapped through the current DMA register set regardless of the current register set. In other words, the DMA register set

112

overrides the current mapping register set for DMA operations on the DMA channel specified. A DMA channel that is not assigned to a DMA register set has all its DMA operations mapped through the current mapping register set.

113

FUNCTION 28 ALTERNATE MAP REGISTER SET

GET ALTERNATE MAP REGISTER SET SUBFUNCTION

Note..ss.....

This function is for use by operating systems only. The operating system can disable this function at any time. Refer to Function 30 for a description of how an operating system can enable or disable this function.

PURPOSE

The subfunction does one of two things depending on the map register set which is active at the time this function is invoked:

1. If the preceding Set Alternate Map Register Set call was done with the alternate map register set equal to zero (BL = 0), these points apply:

a. The context save area pointer saved within EMM by the Set Alternate Map Register Set subfunction) is returned by this call. This pointer is always returned for boards which do not supply alternate mapping register sets.

b. If the context save area pointer returned is not equal to zero, this subfunction copies the contents of the mapping registers on each expanded memory board in the system into the save area specified by the pointer. The format of this save area is the same as that returned by Function 15 (Get Page Map subfunction). This is intended to simulate getting an alternate map register set. Note that the memory manager does not allocate the space for the context: the operating system must do so.

c. If the context save area pointer returned is equal to zero, this subfunction does not copy the contents of the mapping registers on each expanded memory board in the system into the save area specified by the pointer.

d. The context save area pointer must have been initialized by a previous Set Alternate Map Register Set call. Note that the value of the context save area pointer saved within EMM is zero immediately after installation.

e. The context save area must be initialized by a previous Get Page Map call (Function 15).

2. If the preceding Set Alternate Map Register Set call was done with the alternate map register set greater than zero (BL > 0), then the number of the alternate map register set which is in use at the time that this function is invoked is returned. The context save area pointer is not returned in this case.

CALLING PARAMETERS

AX = 5B00h

Contains the Get Alternate Map Register Set subfunction.

RESULTS

These results are valid only if the status returned is zero.

IF (BL <> 0) current active alternate map register set number
Contains the alternate map register set which was active at the time that this function was invoked.

ES:DI
Unaffected.

IF (BL = 0)
Indicates that a pointer to an area which contains the state of all the map registers on all boards in the system, and any additional information necessary to restore the boards to their original state, has been returned.

ES:DI = pointer to a map register context save area
Contains a pointer to an operating system supplied context save area. The pointer is in standard segment:offset format. This pointer is always returned if the expanded memory hardware does not supply alternate mapping register sets.

The operating system first passes this pointer to the memory manager whenever it invokes a Set Alternate Map Register Set subfunction (the description follows). If the OS/E invokes this function before invoking a Set Alternate Map Register Set subfunction, this function returns a pointer value of zero. The OS/E must have allocated the space for the save area. However, the OS must request that the memory manager initialize the contents of this save area before it contains any useful information.

The OS/E must initialize the save area it has allocated by invoking Function 15 (Get Page Map subfunction). After the OS/E has done this, the save area will contain the state of all the map registers on all boards in the system. The save area will also contain any additional information necessary to restore the boards to their original state when the operating system invokes a Set Alternate Map Register Set subfunction.

REGISTERS MODIFIED

AX, BX, ES:DI

STATUS

AH = 0 SUCCESSFUL
The manager got the alternate map register set.

AH = 80h NON-RECOVERABLE
The manager detected a malfunction in the memory manager software.

AH = 81h NON-RECOVERABLE
The manager detected a malfunction in the expanded memory hardware.

AH = 84h NON-RECOVERABLE
The function code passed to the memory manager is not defined.

AH = 8Fh NON-RECOVERABLE
The subfunction parameter is invalid.

AH = A4h NON-RECOVERABLE
The operating system denied access to this function. The function cannot be used at this time.

EXAMPLE

```
alt_map_reg_set                 DB ?
context_save_area_ptr_seg       DW ?
context_save_area_ptr_offset    DW ?

MOV  AX, 5B00h                  ; load function code
INT  67h                        ; call the memory manager
OR   AH, AH                     ; check EMM status
JNZ  emm_err_handler            ; jump to error handler on error
MOV  alt_map_reg_set, BL
TEST BL, BL
JNZ  no_ptr_returned
MOV  context_save_area_ptr_seg, ES ; save pointer values
MOV  context_save_area_ptr_offset,  DI

no_ptr_returned:
```


FUNCTION 28 ALTERNATE MAP REGISTER SET

SET ALTERNATE MAP REGISTER SET SUBFUNCTION

Notes.....

This function is for use by operating systems only. The operating system can disable this function at any time. Refer to Function 30 for a description of how an operating system can enable or disable this function.

PURPOSE

The subfunction does one of two things, depending on the map register set specified:

1 If the alternate map register set specified is zero, map register set zero is activated. If the map register context restore area pointer is not equal to zero, the contents of the restore area pointed to by ES:DI are copied into register set zero on each expanded memory board in the system. If the pointer is equal to zero, the contents are not copied.

   Regardless of its value, the map register context restore area pointer is saved within the memory manager. It will be used during the Get Alternate Map Register Set subfunction.

   The operating system must supply the pointer to the area. This subfunction is intended to simulate setting an alternate map register set. Note that the operating system must allocate the space for the context. The memory manager saves the context save area pointer internally.

2 If the alternate map register set specified is not zero, the alternate map register set specified is activated. The restore area, which the operating system is pointing to, is not used.

CALLING PARAMETERS

AX = 5B01h
   Contains the Set Alternate Map subfunction.

BL = new alternate map register set number
   Contains the number of the alternate map register set which is to be activated.

If BL <> 0
   A pointer to a map register context restore area is not required and the contents of ES:DI are unaffected and ignored. The alternate map register set specified in BL is activated if the board supports it.

If BL = 0

A pointer to an area which contains the state of all the map registers on all boards in the system, and any additional information necessary to restore the boards to their original state, has been passed in ES:DI.

ES:DI = pointer to a map register context restore area

Contains a pointer to an OS/E supplied map register context restore area. The pointer is in standard segment:offset format. This pointer must always be passed if the expanded memory hardware does not supply alternate mapping register sets.

The memory manager must save this pointer whenever the OS/E invokes this function. The OS/E must have allocated the space for the restore area. Additionally, the contents of this restore area must have been initialized by the memory manager before it will contain any useful information. The OS/E initializes the restore area it has allocated by invoking Function 15 (Get Page Map subfunction). After the OS/E has done this, the restore area will contain the state of the map registers on all boards in the system, and any additional information necessary to restore the boards to their original state when the operating system invokes a Set Alternate Map Register Set subfunction.

REGISTERS MODIFIED

AX

STATUS

AH = 0 SUCCESSFUL

The manager set the alternate map register set.

AH = 80h NON-RECOVERABLE

The manager detected a malfunction in the memory manager software.

AH = 81h NON-RECOVERABLE

The manager detected a malfunction in the expanded memory hardware.

AH = 84h NON-RECOVERABLE
The function code passed to the memory manager is not defined.

AH = 8Fh NON-RECOVERABLE
The subfunction parameter is invalid.

AH = 9Ah NON-RECOVERABLE
Alternate map register sets are supported, but the alternate map register set specified is not supported.

AH = 9Ch NON-RECOVERABLE
Alternate map register sets are not supported, and the alternate map register set specified is not zero.

AH = 9Dh NON-RECOVERABLE
Alternate map register sets are supported, but the alternate map register set specified is either not defined or not allocated.

AH = A3h NON-RECOVERABLE
The contents of the source array have been corrupted, or the pointer passed to the subfunction is invalid.

AH = A4h NON-RECOVERABLE
The operating system has denied access to this function. The function cannot be used at this time.

EXAMPLE

```
alt_map_reg_set                 DB ?
context_restore_area_ptr_seg    DW ?
context_restore_area_ptr_offset  DW ?

MOV  AX, 5B01h                  ; load function code
MOV  BL, alt_map_reg_set
TEST BL, BL
JZ   no_ptr_passed
MOV  ES, context_restore_area_ptr_seg
MOV  DI, context_restore_area_ptr_offset

no_ptr_passed:
```

```
INT  67h                    ; call the memory manager
OR   AH, AH                   ; check EMM status
JNZ  emm_err_handler            ; jump to error handler on error
```

119

FUNCTION 28 ALTERNATE MAP REGISTER SET

GET ALTERNATE MAP SAVE ARRAY SIZE SUBFUNCTION

Note.....

This function is for use by operating systems only. The operating system can disable this function at any time. Refer to Function 30 for a description of how an operating system can enable or disable this function.

PURPOSE

This subfunction returns the storage requirements for the map register context save area referenced by the other subfunctions.

CALLING PARAMETERS

AX = 5B02h
Contains the Get Alternate Map Save Array Size subfunction.

RESULTS

These results are valid only if the status returned is zero.

DX = size_of_array
Contains the number of bytes that will be transferred to the memory area supplied by an operating system whenever an operating system requests the Get, Set, or Get and Set subfunction.

REGISTERS MODIFIED

AX, DX

STATUS

AH = 0 SUCCESSFUL
The manager has returned the array size.

AH = 80h NON-RECOVERABLE
The manager detected a malfunction in the memory manager software.

AH = 81h NON-RECOVERABLE

The manager detected a malfunction in the expanded memory hardware.

AH = 84h NON-RECOVERABLE

The function code passed to the memory manager is not defined.

AH = 8Fh NON-RECOVERABLE

The subfunction parameter is invalid.

AH = A4h NON-RECOVERABLE

The operating system has denied access to this function. The function cannot be used at this time.

EXAMPLE

```
size_of_array                DW ?

MOV  AX, 5B02h                  ; load function code
INT  67h                     ; call the memory manager
OR   AH, AH                    ; check EMM status
JNZ  emm_err_handler             ; jump to error handler on error
MOV  size_of_array, DX          ; save size of array
```


FUNCTION 28 ALTERNATE MAP REGISTER SET

ALLOCATE ALTERNATE MAP REGISTER SET SUBFUNCTION

Note.....

This function is for use by operating systems only. The operating system can disable this function at any time. Refer to Function 30 for a description of how an operating system can enable or disable this function.

PURPOSE

The Allocate Alternate Map Register Set subfunction gets the number of an alternate map register set for an operating system if an alternate map register set is currently available for use. If the hardware does not support alternate map register sets, an alternate map register set number of zero will be returned.

The alternate map register set allocated may be referred to by this number when using the Get or Set Alternate Map Register Set subfunctions. The operating system can use these subfunctions to switch map contexts very rapidly on expanded memory boards with alternate map register sets.

This subfunction copies the currently active alternate map register sets contents into the newly allocated alternate map register set's mapping registers. This is done so that when the OS/E performs a Set Alternate Map Register Set subfunction the memory mapped before the allocation of the new alternate map will be available for reading and writing. This function does not actually change the alternate map register set in use, but in addition to allocating a new alternate map register set, it prepares the new alternate map register set for a subsequent Set Alternate Map Register Set subfunction.

CALLING PARAMETERS

AX = 5B03h
: Contains the Allocate Alternate Map Register Set subfunction.

RESULTS

These results are valid only if the status returned is zero.

BL = alternate map register set number
: Contains the number of an alternate map register set. If there are no alternate map register sets supported by the hardware, a zero will be returned. In this case, the Get Alternate Map function (Function 28) should be invoked in order to obtain a pointer to a map register context save area. The OS/E must supply this save area. The save area is necessary because the hardware doesn't support alternate map register sets.

REGISTERS MODIFIED

AX, BX

STATUS

AH = 0 SUCCESSFUL
: The manager has returned the array size.

AH = 80h NON-RECOVERABLE
The manager detected a malfunction in the memory manager software.

AH = 81h NON-RECOVERABLE
The manager detected a malfunction in the expanded memory hardware.

AH = 84h NON-RECOVERABLE
The function code passed to the memory manager is not defined.

AH = 8Fh NON-RECOVERABLE
The subfunction parameter is invalid.

AH = 9Bh NON-RECOVERABLE
Alternate map register sets are supported. However, all alternate map register sets are currently allocated.

AH = A4h NON-RECOVERABLE
The operating system has denied access to this function. The function cannot be used at this time.

EXAMPLE

```
alt_map_reg_num                 DB ?

MOV  AX, 5B03h                  ; load function code
INT  67h                        ; call the memory manager
OR   AH, AH                     ; check EMM status
JNZ  emm_err_handler            ; jump to error handler on error
MOV  alt_map_reg_num, BL        ; save # of alternate map reg
                                ; set
```


FUNCTION 28 ALTERNATE MAP REGISTER SET

DEALLOCATE ALTERNATE MAP REGISTER SET SUBFUNCTION

Note.....

This function is for use by operating systems only. The operating system can disable this function at any time. Refer to Function 30 for a description of how an operating system can enable or disable this function.

PURPOSE

The Deallocate Alternate Map Register Set subfunction returns the alternate map register set to the memory manager for future use. The memory manager may reallocate the alternate map register set when needed.

This subfunction also makes the mapping context of the alternate map register specified unavailable for reading or writing (unmapping). This protects the pages previously mapped in an alternate map register set by making them inaccessible. Note that the current alternate map register set cannot be deallocated. This makes all memory which was currently mapped into conventional and expanded memory inaccessible.

CALLING PARAMETERS

AX = 5B04h
: Contains the Deallocate Alternate Map Register Set subfunction.

BL = alternate register set number
: Contains the number of the alternate map register set to deallocate. Map register set zero cannot be allocated or deallocated. However, if alternate map register set zero is specified and this subfunction is invoked, no error will be returned. The function invocation is ignored in this case.

REGISTERS MODIFIED

AX

STATUS

AH = 0 SUCCESSFUL
: The manager has returned the array size.

AH = 80h NON-RECOVERABLE
: The manager detected a malfunction in the memory manager software.

AH = 81h NON-RECOVERABLE
: The manager detected a malfunction in the expanded memory hardware.

AH = 84h NON-RECOVERABLE
The function code passed to the memory manager is not defined.

AH = 8Fh NON-RECOVERABLE
The subfunction parameter is invalid.

AH = 9Ch NON-RECOVERABLE
Alternate map register sets are not supported and the alternate map register set specified is not zero.

AH = 9Dh NON-RECOVERABLE
Alternate map register sets are supported, but the alternate map register set specified is either not defined or not allocated.

AH = A4h NON-RECOVERABLE
The operating system has denied access to this function. The function cannot be used at this time.

EXAMPLE

```
alternate_map_reg_set           DB ?

MOV  AL, alternate_map_reg_set    ; specify alternate map reg set
MOV  AX, 5B04h                    ; load function code
INT  67h                          ; call the memory manager
OR   AH, AH                       ; check EMM status
JNZ  emm_err_handler              ; jump to error handler on error
```


FUNCTION 28 ALTERNATE MAP REGISTER SET

ALLOCATE DMA REGISTER SET SUBFUNCTION

Note.....

This function is for use by operating systems only. The operating system can disable this function at any time. Refer to Function 30 for a description of how an operating system can enable or disable this function.

PURPOSE

The Allocate DMA Register Set subfunction gets the number of a DMA register set for an OS/E, if a DMA register set is currently available for use. If the hardware does not support DMA register sets, a DMA register set number of zero will be returned.

In a multitasking operating system, when one task is waiting for DMA to complete, it is useful to be able to switch to another task. However, if the DMA is being mapped through the current register set, the switching cannot occur. That is, all DMA action must be complete before any remapping of pages can be done.

The operating system would initiate a DMA operation on a specific DMA channel using a specific alternate map register set. This alternate map register set would not be used again, by the operating system or an application, until after the DMA operation is complete. The operating system guarantees this by not changing the contents of the alternate map register set, or allowing an application to change the contents of the alternate map register set, for the duration of the DMA operation.

CALLING PARAMETERS

AX = 5B05h
: Contains the Allocate DMA Register Set subfunction.

RESULTS

These results are valid only if the status returned is zero.

BL = DMA register set number
: Contains the number of a DMA register set. If there are no DMA register sets supported by the hardware, a zero will be returned.

REGISTERS MODIFIED

AX, BX

STATUS

AH = 0 SUCCESSFUL
: The manager has allocated the DMA register set.

AH = 80h NON-RECOVERABLE
: The manager detected a malfunction in the memory manager software.

AH = 81h NON-RECOVERABLE
The manager detected a malfunction in the expanded memory hardware.

AH = 84h NON-RECOVERABLE
The function code passed to the memory manager is not defined.

AH = 8Fh NON-RECOVERABLE
The subfunction parameter is invalid.

AH = 9Bh NON-RECOVERABLE
DMA register sets are supported. However, all DMA register sets are currently allocated.

AH = A4h NON-RECOVERABLE
Access to this function has been denied by the operating system. The function cannot be used at this time.

EXAMPLE

```
DMA_reg_set_number              DB ?

MOV  AX, 5B05h                  ; load function code
INT  67h                        ; call memory manager
OR   AH, AH                     ; check EMM status
JNZ  emm_err_handler            ; jump to error handler on error
MOV  DMA_reg_set_number, BL     ; save number of DMA register
                                ; set
```


FUNCTION 28 ALTERNATE MAP REGISTER SET

ENABLE DMA ON ALTERNATE MAP REGISTER SET SUBFUNCTION

Note.....

This function is for use by operating systems only. The operating system can disable this function at any time. Refer to Function 30 for a description of how an operating system can enable or disable this function.

PURPOSE

This subfunction allows DMA accesses on a specific DMA channel to be associated with a specific alternate map register set. In a multitasking operating system, when a task is waiting for the completion of DMA, it is useful to be able to switch to another task until the DMA operation completes.

Any DMA on the specified channel will go through the specified DMA register set regardless of the current register set. If a DMA channel is not assigned to a DMA register set, DMA for that channel will be mapped through the current register set.

CALLING PARAMETERS

AX = 5B06h
Contains the Enable DMA on Alternate Map Register Set subfunction.

BL = DMA register set number
Contains the number of the alternate map register set to be used for DMA operations on the DMA channel specified by DL. If the alternate map register set specified is zero, no special action will be taken on DMA accesses for the DMA channel specified.

DL = DMA channel number
Contains the DMA channel which is to be associated with the DMA map register set specified in BL.

REGISTERS MODIFIED

AX

STATUS

AH = 0 SUCCESSFUL
The manager has enabled DMA on the DMA register set and the DMA channel specified.

AH = 80h NON-RECOVERABLE
The manager detected a malfunction in the memory manager software.

AH = 81h NON-RECOVERABLE
The manager detected a malfunction in the expanded memory hardware.

AH = 84h NON-RECOVERABLE
The function code passed to the memory manager is not defined.

AH = 8Fh NON-RECOVERABLE
The subfunction parameter is invalid.

AH = 9Ah NON-RECOVERABLE
Alternate DMA register sets are supported, but the alternate DMA register set specified is not supported.

AH = 9Ch NON-RECOVERABLE
Alternate DMA register sets are not supported, and the DMA register set specified is not zero.

AH = 9Dh NON-RECOVERABLE
DMA register sets are supported, but the DMA register set specified is either not defined or not allocated.

AH = 9Eh NON-RECOVERABLE
Dedicated DMA channels are not supported.

AH = 9Fh NON-RECOVERABLE
Dedicated DMA channels are supported. But the DMA channel specified is not supported.

AH = A4h NON-RECOVERABLE
The operating system has denied access to this function. The function cannot be used at this time.

EXAMPLE

```
alt_map_reg_set             DB ?
DMA_channel_num             DB ?

MOV  BL, alt_map_reg_set
MOV  DL, DMA_channel_num
MOV  AX, 5B06h              ; load function code
INT  67h                    ; call the memory manager
OR   AH, AH                 ; check EMM status
JNZ  emm_err_handler        ; jump to error handler on error
```


FUNCTION 28 ALTERNATE MAP REGISTER SET

DISABLE DMA ON ALTERNATE MAP REGISTER SET SUBFUNCTION

Note.....

This function is for use by operating systems only. The operating system can disable this function at any time. Refer to Function 30 for a description of how an operating system can enable or disable this function.

PURPOSE

This subfunction disables DMA accesses for all DMA channels which were associated with a specific alternate map register set.

CALLING PARAMETERS

AX = 5B07h
Contains the Disable DMA on Alternate Map Register Set subfunction.

BL = alternate register set number
Contains the number of the DMA register set for which all operations are to be disabled. If the alternate map register set specified is zero, no action will be taken.

REGISTERS MODIFIED

AX

STATUS

AH = 0 SUCCESSFUL
The manager has returned the array size.

AH = 80h NON-RECOVERABLE
The manager detected a malfunction in the memory manager software.

AH = 81h NON-RECOVERABLE
The manager detected a malfunction in the expanded memory hardware.

AH = 84h NON-RECOVERABLE
The function code passed to the memory manager is not defined.

AH = 8Fh NON-RECOVERABLE
The subfunction parameter is invalid.

AH = 9Ah NON-RECOVERABLE
Alternate DMA register sets are supported, but the
alternate DMA register set specified is not supported.

130

AH = 9Ch NON-RECOVERABLE
Alternate DMA register sets are not supported, and the DMA
register set specified is not zero.

AH = 9Dh NON-RECOVERABLE
DMA register sets are supported, but the DMA register set
specified is either not defined or not allocated.

AH = 9Eh NON-RECOVERABLE
Dedicated DMA channels are not supported.

AH = 9Fh NON-RECOVERABLE
Dedicated DMA channels are supported, but the DMA channel
specified is not supported.

AH = A4h NON-RECOVERABLE
The operating system has denied access to this function.
The function cannot be used at this time.

EXAMPLE

```
DMA_reg_set                 DB ?

MOV  BL, DMA_reg_set
MOV  AX, 5B07h                ; load function code
INT  67h                    ; call the memory manager
OR   AH, AH                   ; check EMM status
JNZ  emm_err_handler          ; jump to error handler on error
```

131

FUNCTION 28 ALTERNATE MAP REGISTER SET

DEALLOCATE DMA REGISTER SET SUBFUNCTION

Note.....

This function is for use by operating systems only. The operating system can disable this function at any time. Refer to Function 30 for a description of how an operating system can enable or disable this function.

PURPOSE

The Deallocate DMA Register Set subfunction deallocates the specified DMA register set.

CALLING PARAMETERS

AX = 5B08h
Contains the Deallocate DMA Register Set subfunction.

BL = DMA register set number
Contains the number of the DMA register set which should not be used for DMA operations any longer. The DMA register set would have been previously allocated and enabled for DMA operations on a specific DMA channel. If the DMA register set specified is zero, no action will be taken.

REGISTERS MODIFIED

AX

STATUS

AH = 0 SUCCESSFUL
The manager has deallocated the DMA register set.

AH = 80h NON-RECOVERABLE
The manager detected a malfunction in the memory manager software.

AH = 81h NON-RECOVERABLE
The manager detected a malfunction in the expanded memory hardware.

AH = 84h NON-RECOVERABLE
The function code passed to the memory manager is not defined.

AH = 8Fh NON-RECOVERABLE
The subfunction parameter is invalid.

AH = 9Ch NON-RECOVERABLE
DMA register sets are not supported, and the DMA register set specified is not zero.

AH = 9Dh NON-RECOVERABLE

DMA register sets are supported, but the DMA register set specified is either not defined or not allocated.

AH = A4h NON-RECOVERABLE

The operating system has denied access to this function. The function cannot be used at this time.

EXAMPLE

```
DMA_reg_set_num                 DB  ?

MOV  DMA_reg_set_num, BL
MOV  AX, 5B08h                  ; load function code
INT  67h                        ; call memory manager
OR   AH, AH                     ; check EMM status
JNZ  emm_err_handler            ; jump to error handler on error
```


FUNCTION 29 PREPARE EXPANDED MEMORY HARDWARE FOR WARM BOOT

PURPOSE

This function prepares the expanded memory hardware for an impending warm boot. This function assumes that the next operation that the operating system performs is a warm boot of the system. In general, this function will effect the current mapping context, the alternate register set in use, and any other expanded memory hardware dependencies which need to be initialized at boot time. If an application decides to map memory below 640K, the application must trap all possible conditions leading to a warm boot and invoke this function before performing the warm boot itself.

CALLING PARAMETERS

AH = 5Ch

Contains the Prepare for Warmboot function.

REGISTERS MODIFIED

AX

STATUS

AH = 0 SUCCESSFUL

The manager has prepared the expanded memory hardware for a warm boot.

AH = 80h NON-RECOVERABLE

The manager detected a malfunction in the memory manager software.

AH = 81h NON-RECOVERABLE

The manager detected a malfunction in the expanded memory hardware.

AH = 84h NON-RECOVERABLE

The function code passed to the memory manager is not defined.

EXAMPLE

```
MOV  AH, 5Ch                ; load function code
INT  67h                    ; call the memory manager
OR   AH, AH                 ; check EMM status
JNZ  emm_err_handler        ; jump to error handler on error
```


FUNCTION 30 ENABLE/DISABLE OS/E FUNCTION SET FUNCTIONS

ENABLE OS/E FUNCTION SET SUBFUNCTION

Note.....

This function is for use by operating systems only. The operating system can disable this function at any time.

PURPOSE

This subfunction provides an OS/E with the ability to enable all programs or device drivers to use the OS/E specific functions. The capability is provided only for an OS/E which manages regions of mappable conventional memory and cannot permit programs to use any of the functions which affect mappable conventional memory regions, but must be able to use these functions itself. When an OS/E disables these functions and a program attempts to use them, the memory manager returns a status to the program indicating that the OS/E has denied the program access to the function. In other words, the functions will not work when disabled. However, all programs may use them when enabled. The OS/E (Operating System/Environment) functions this subfunction enables are:

Function 26. Get Expanded Memory Hardware Information.

Function 28. Alternate Map Register Sets.

Function 30. Enable/Disable Operating System Functions.

It appears contradictory that the OS/E can re-enable these functions when the function which re-enables them is itself disabled. An overview of the process follows.

The memory manager enables all the OS/E specific functions, including this one, when it is loaded. The OS/E gets exclusive access to these functions by invoking either of the Enable/Disable OS/E Function Set subfunctions before any other software does.

On the first invocation of either of these subfunctions, the memory manager returns an access_key which the OS/E must use in all future invocations of either of these subfunctions. The memory manager does not require the access_key on the first invocation of the Enable/Disable OS/E Function Set subfunctions.

ENABLE OS/E FUNCTION SET SUBFUNCTION

On all subsequent invocations, the access_key is required for either the Enable or Disable OS/E Function Set subfunctions. Since the access_key is returned only on the first invocation of the Enable/Disable OS/E Function Set subfunctions, and presumably the OS/E is the first software to invoke this function, only the OS/E obtains a copy of this key. The memory manager must return an access key with a random value, a fixed value key defeats the purpose of providing this level of security for an OS/E.

CALLING PARAMETERS

AX = 5D00h
Contains the Enable OS/E Function Set subfunction.

BX,CX = access_key
Required on all function invocations after the first. The access_key value returned by the first function invocation is required.

RESULTS

These results are valid only if the status returned is zero.

BX,CX = access_key

Returned only on the first function invocation, the memory manager returns a random valued key which will be required thereafter for the execution of this function. On all invocations after the first, this key is not returned. Neither BX or CX is affected after the first time this function is invoked.

REGISTERS MODIFIED

AX, BX, CX

STATUS

AH = 0 SUCCESSFUL

The operating system function set has been enabled.

AH = 80h NON-RECOVERABLE

The manager detected a malfunction in the memory manager software.

AH = 81h NON-RECOVERABLE

The manager detected a malfunction in the expanded memory hardware.

AH = 84h NON-RECOVERABLE

The function code passed to the memory manager is not defined.

AH = 8Fh NON-RECOVERABLE

The subfunction parameter is invalid.

AH = A4h NON-RECOVERABLE

The operating system has denied access to this function. The function cannot be used at this time. The value of the key which was passed to this function does not entitle the program to execute this function.

EXAMPLE

First invocation

```
access_key                DW 2 DUP (?)

MOV  AX, 5D00h                ; load function code
```

```
INT  67h                    ; call the memory manager
OR   AH, AH                   ; check EMM status
JNZ  emm_err_handler              ; jump to error handler on error
MOV  access_key[0], BX
MOV  access_key[2], CX
```

All invocations after the first

```
access_key                  DW 2 DUP (?)

MOV  BX, access_key[0]
MOV  CX, access_key[2]
MOV  AX, 5D00h                ; load function code
INT  67h                    ; call the memory manager
OR   AH, AH                   ; check EMM status
JNZ  emm_err_handler              ; jump to test handler on error
```

137

FUNCTION 30 ENABLE/DISABLE OS/E FUNCTION SET FUNCTIONS

DISABLE OS/E FUNCTION SET SUBFUNCTION

Note.....

This function is for use by operating systems only. The operating system can disable this function at any time.

PURPOSE

This subfunction provides an OS/E with the ability to disable all programs or device drivers from using the OS/E specific functions. The capability is provided only for an OS/E which manages regions of mappable conventional memory and cannot permit programs to use any of the functions which would affect mappable conventional memory regions. When an OS/E disables these functions and a program attempts to use them, the memory manager returns a status to the program indicating that the OS/E has denied the program access to the function. In other words, the functions will not work when disabled.

The OS/E (Operating System) functions which are disabled by this subfunction are:

Function 26. Get Expanded Memory Hardware Information.

Function 28. Alternate Map Register Sets.

Function 30. Enable/Disable Operating System Functions.

CALLING PARAMETERS

AX = 5D01h
Contains the Disable OS/E Function Set subfunction.

BX,CX = access_key
Required on all function invocations after the first. The access_key value returned by the first function invocation is required.

RESULTS

These results are valid only if the status returned is zero.

BX,CX = access_key
Returned only on the first function invocation, the memory manager returns a random valued key which will be required thereafter for the execution of this function. On all invocations after the first, this key is not returned. Neither BX nor CX is affected after the first time this function is invoked.

REGISTERS MODIFIED

AX, BX, CX

STATUS

AH = 0 SUCCESSFUL
The operating system function set has been disabled.

AH = 80h NON-RECOVERABLE
The manager detected a malfunction in the memory manager software.

AH = 81h NON-RECOVERABLE
The manager detected a malfunction in the expanded memory hardware.

AH = 84h NON-RECOVERABLE
The function code passed to the memory manager is not defined.

AH = 8Fh NON-RECOVERABLE
The subfunction parameter is invalid.

AH = A4h NON-RECOVERABLE

The operating system has denied access to this function. The function cannot be used at this time. The value of the key which was passed to this function does not entitle the program to execute this function.

EXAMPLE

First Function invocation:

```
access_key                  DW 2 DUP (?)

MOV  AX, 5D01h                  ; load function code
INT  67h                    ; call the memory manager
OR   AH, AH                   ; check EMM status
JNZ  emm_err_handler            ; jump to error handler on error
MOV  access_key[0], BX
MOV  access_key[2], CX
```

All invocations after the first:

```
access_key                  DW 2 DUP (?)

MOV  BX, access_key[0]
MOV  CX, access_key[2]
MOV  AX, 5D01h                  ; load function code
INT  67h                    ; call the memory manager
OR   AH, AH                   ; check EMM status
JNZ  emm_err_handler            ; jump to error handler on error
```


FUNCTION 30 ENABLE/DISABLE OS/E FUNCTION SET FUNCTIONS

RETURN ACCESS KEY SUBFUNCTION

Note.....

This function is for use by operating systems only. The operating system can disable this function at any time.

PURPOSE

This subfunction provides an OS/E with the ability to return the access key to the memory manager. Returning the access key to the memory manager places the memory manager in the state it is in at installation time (regarding the use of the OS/E function set and the access key). That is, access to the OS/E function set is enabled. Upon execution of the next enable/disable OS/E function set subfunction, the access key will once again be returned.

CALLING PARAMETERS

AX = 5D02h
Contains the Return Access Key subfunction.

BX,CX = access_key
Required on all function invocations. The access_key value returned by the first function invocation of the enable or disable subfunctions is required.

REGISTERS MODIFIED

AX

STATUS

AH = 0 SUCCESSFUL
The access key has been returned to the memory manager.

AH = 80h NON-RECOVERABLE
The manager detected a malfunction in the memory manager software.

AH = 81h NON-RECOVERABLE
The manager detected a malfunction in the expanded memory hardware.

AH = 84h NON-RECOVERABLE
The function code passed to the memory manager is not defined.

AH = 8Fh NON-RECOVERABLE
The subfunction parameter is invalid.

AH = A4h NON-RECOVERABLE
Access to this function has been denied by the operating system. The function cannot be used at this time. The

value of the key which was passed to this function does not entitle the program to execute this function.

EXAMPLE

```
access_key                  DW 2 DUP (?)

MOV  BX, access_key[0]
MOV  CX, access_key[2]
MOV  AX, 5D02h                  ; load function code
INT  67h                        ; call the memory manager
OR   AH, AH                     ; check EMM status
JNZ  emm_err_handler            ; jump to error handler on error
```


APPENDIX A FUNCTION AND STATUS CODE CROSS REFERENCE TABLES

This appendix contains two cross reference tables: one lists this function codes and the status codes they return; the other lists the status codes and the functions that return them.

TABLE A-1. FUNCTION AND STATUS CODE CROSS REFERENCE

Function	Status	Description
40h	00h 80h 81h 84h	Get memory manager status
41h	00h 80h 81h 84h	Get Page Frame Segment Address
42h	00h 80h 81h 84h	Get Unallocated Page Count
43h	00h 80h 81h 84h 85h 87h 88h	Allocate Pages
44h	00h 80h 81h 83h 84h 8Ah 8Bh	Map/Unmap Handle Page
45h	00h 80h 81h 83h 84h 86h	Deallocate Pages
46h	00h 80h 81h 84h	Get EMM Version
47h	00h 80h 81h 83h 84h 8Ch 8Dh	Save Page Map

48h 00h 80h 81h 83h Restore Page Map
84h 8Eh

49h Reserved

4Ah Reserved

4Bh 00h 80h 81h 84h Get EMM Handle Count

4Ch 00h 80h 81h 83h Get EMM Handle Pages
84h

4Dh 00h 80h 81h 84h Get All EMM Handle Pages

4E00h 00h 80h 81h 84h Get Page Map
8Fh

4E01h 00h 80h 81h 84h Set Page Map
8Fh A3h

4E02h 00h 80h 81h 84h Get & Set Page Map
8Fh A3h

4E03h 00h 80h 81h 84h Get Size of Page Map Save Array
8Fh

4F00h 00h 80h 81h 84h Get Partial Page Map
8Bh 8Fh A3h

4F01h 00h 80h 81h 84h Set Partial Page Map
8Fh A3h

4F02h 00h 80h 81h 84h Get Size of Partial Page Map Array
8Bh 8Fh

5000h 00h 80h 81h 83h Map/Unmap Multiple Handle Pages
84h 8Ah 8Bh 8Fh (Physical page mode)

5001h 00h 80h 81h 83h Map/Unmap Multiple Handle Pages
84h 8Ah 8Bh 8Fh (Segment address mode)

51h 00h 80h 81h 83h Reallocate Pages
84h 87h 88h

5200h 00h 80h 81h 83h Get Handle Attribute
84h 8Fh 91h

5201h 00h 80h 81h 83h Set Handle Attribute
84h 8Fh 90h 91h

5202h 00h 80h 81h 84h Get Handle Attribute Capability
8Fh

5300h 00h 80h 81h 83h Get Handle Name
84h 8Fh

5301h 00h 80h 81h 83h Set Handle Name
84h 8FH A1h

5400h 00h 80h 81h 84h Get Handle Directory
8Fh

5401h 00h 80h 81h 84h Search for Named Handle
8Fh A0h A1h

5402h 00h 80h 81h 84h Get Total Handles
8Fh

5500h 00h 80h 81h 83h Alter Page Map & Jump
84h 8Ah 8Bh 8Fh (Physical page mode)

5501h 00h 80h 81h 83h Alter Page Map & Jump
84h 8Ah 8Bh 8Fh (Segment address mode)

5600h 00h 80h 81h 83h Alter Page Map & Call
84h 8Ah 8Bh 8Fh (Physical page mode)

5601h 00h 80h 81h 83h Alter Page Map & Call
84h 8Ah 8Bh 8Fh (Segment address mode)

5602h 00h 80h 81h 84h Get Alter Page Map & Call Stack Space Size
8Fh

5700h 00h 80h 81h 83h Move Memory Region
84h 8Ah 8Fh 92h
93h 94h 95h 96h
98h A2h

5701h 00h 80h 81h 83h Exchange Memory Region
84h 8Ah 8Fh 93h
94h 95h 96h 97h
98h A2h

5800h 00h 80h 81h 84h 8Fh Get Mappable Physical Address Array

5801h 00h 80h 81h 84h 8Fh Get Mappable Physical Address Array Entries

5900h 00h 80h 81h 84h 8Fh A4h Get Expanded Memory Hardware Information

5901h 00h 80h 81h 84h 8Fh Get Unallocated Raw Page Count

5A00h 00h 80h 81h 84h 85h 87h 88h 8Fh Allocate Standard Pages

5A01h 00h 80h 81h 84h 85h 87h 88h 8Fh Allocate Raw Pages

5B00h 00h 80h 81h 84h 8Fh A4h Get Alternate Map Register Set

5B01h 00h 80h 81h 84h 8Fh 9Ah 9Ch 9Dh A3h A4h Set Alternate Map Register Set

5B02h 00h 80h 81h 84h 8Fh A4h Get Alternate Map Save Array Size

5B03h 00h 80h 81h 84h 8Fh 9Bh A4h Allocate Alternate Map Register Set

5B04h 00h 80h 81h 84h 8Fh 9Ch 9Dh A4h Deallocate Alternate Map Register Set

5B05h 00h 80h 81h 84h 8Fh 9Bh A4h Allocate DMA Register Set

5B06h 00h 80h 81h 84h 8Fh 9Ah 9Ch 9Dh 9Eh 9Fh A4h Enable DMA on Alternate Map Register Set

5B07h 00h 80h 81h 84h 8Fh 9Ah 9Ch 9Dh 9Eh 9Fh A4h Disable DMA on Alternate Map Register Set

5B08h 00h 80h 81h 84h 8Fh 9Ch 9Dh A4h Deallocate DMA Register Set

144

5Ch 00h 80h 81h 84h Prepare Expanded Memory Hardware for Warmboot

5D00h 00h 80h 81h 84h Enable Operating System Function Set 8Fh A4h

5D01h 00h 80h 81h 84h Disable Operating System Function Set 8Fh A4h

5D02h 00h 80h 81h 84h Return Operating System Access Key 8Fh A4h

145

TABLE A-2. STATUS AND FUNCTION CODE CROSS REFERENCE

Status	Functions	Description
00h	All	The function completed normally.
80h	All	The memory manager has detected a malfunction in the expanded memory software.
81h	All	The memory manager has detected a malfunction in the expanded memory hardware.
82h	None	This error code is not returned in version 3.2 of the memory manager or above. In earlier versions of the memory manager this code meant a "busy" status. This status indicated that the memory manager was already processing an expanded memory request when the current request was made and is unable to process another request. In versions 3.2 of the memory manager and above, the memory manager is never "busy" and can always honor requests.

83h	44h	45h	47h	48h 4Ch 5000h 5001h 51h 5200h 5201h 5300h 5301h 5500h 5501h 5600h 5601h 5700h 5701h	The memory manager can not find the handle specified.
84h	All				The function code passed to the manager is not currently defined.
85h	43h	5A00h	5A01h		No handles are currently available. All assignable handles are currently in use.
86h	45h				A mapping context restoration error has been detected. This error occurs when a program attempts to return a handle and there is still a "mapping context" on the context stack for the indicated handle.
87h	43h	51h	5A00h	5A01h	The number of total pages that are available in the system is insufficient to honor the request.

88h	43h	51h	5A00h	5A01h	The number of unallocated pages currently available is insufficient to honor the allocation request.
89h	43h				Zero pages could not be assigned to a handle.
8Ah	44h	5000h 5001h 5500h 5501h 5600h 5601h 5700h 5701h			The logical page to map into memory is out of the range of logical pages which are allocated to the handle.
8Bh	44h	4F00h 4F02h 5000h 5001h 5600h 5601h 5500h 5501h			One or more of the physical pages is out of the range of allowable physical pages.
8Ch	47h				The mapping register context save area is full.

8Dh	47h		The mapping register context stack already has a context associated with the handle. The program has attempted to save the mapping register context when there was already a context for the handle on the stack.
8Eh	48h		The mapping register context stack does not have a context associated with the handle. The program has attempted to restore the mapping register context when there was no context for the handle on the stack.
8Fh	All		The subfunction parameter passed to the function requiring a subfunction code is not defined.
90h	5201h		The attribute type is undefined.
91h	5200h	5201h	The system configuration does not support non-volatility.
92h	5700h		The source and destination expanded memory regions have the same handle and overlap. This is valid for a move. The move has been completed and the destination region has a full copy of the source region. However, at least a portion of the source region has been overwritten by the move.
93h	5700h	5701h	The length of the specified source or destination expanded memory region exceeds the length of the

expanded memory region allocated to the specified source or destination handle. There are insufficient pages allocated to this handle to move/exchange a region of the size specified.

94h 5700h 5701h The conventional memory region and expanded memory region overlap. This is invalid, the conventional memory region cannot overlap the expanded memory region.

95h 5700h 5701h The offset within the logical page exceeds the length of the logical page. The initial source or destination offsets within an expanded memory region must be between 0 and the (length of a logical page - 1) or 16383 (3FFFh).

96h 5700h 5701h Region length exceeds 1M-byte limit.

97h 5701h The source and destination expanded memory regions have the SAME handle AND overlap. This is invalid; the source and destination expanded memory regions cannot have the same handle and overlap when they are being exchanged.

98h 5700h 5701h The memory source and destination types are undefined/not supported.

9Ah 5B01h 5B06h 5B07h Alternate map register sets are supported, but the alternate map register set specified is not supported.

9Bh 5B03h 5B05h Alternate map/DMA register sets are supported. However, all alternate map/DMA register sets are currently allocated.

9Ch	5B01h	5B04h	5B06h	5B07h	Alternate map/DMA register sets are not supported, and the alternate map/DMA register set specified is not zero.
	5B08h				
9Dh	5B01h	5B04h	5B06h	5B07h	Alternate map/DMA register sets are supported, but the alternate map register set specified is not defined, not allocated, or is the currently allocated map register set.
	5B08h				

9Eh	5B06h	5B07h	Dedicated DMA channels are not supported.
9Fh	5B06h	5B07h	Dedicated DMA channels are supported. But the DMA channel specified is not supported.
A0h	5401h		No corresponding handle value could be found for the handle name specified.
A1h	5301h	5401h	A handle with this name already exists. The specified handle was not assigned a name.
A2h	5700h	5701h	An attempt was made to "wrap around" the 1M-byte address space during the move/exchange. The source starting address together with the length of the region to be moved/exchanged exceeds 1M-byte. No data was moved/exchanged.

A3h	4E01h	4E02h	4F00h	4F01h	The contents of the data structure passed to the function have either been corrupted or are meaningless.
	5B01h				
A4h	5900h	5B00h	5B01h	5B02h	The operating system has denied access to this function. The function cannot be used at this time.
	5B03h	5B04h	5B05h	5B06h	
	5B07h	5B08h	5D00h	5D01h	
	5D02h				

APPENDIX B

TESTING FOR THE PRESENCE OF THE EXPANDED MEMORY MANAGER

Before an application program can use the Expanded Memory Manager, it must determine whether DOS has loaded the manager. This appendix describes two methods your program can use to test for the presence of the memory manager and how to choose the correct one for your situation.

The first method uses the DOS "open handle" technique; the second method uses the DOS "get interrupt vector" technique.

WHICH METHOD SHOULD YOUR PROGRAM USE?

The majority of application programs can use either the "open handle" or the "get interrupt vector" method. However, if your program is a device driver or if it interrupts DOS during file system operations, you must use only the "get interrupt vector" method.

Device drivers execute from within DOS and can't access the DOS file system; programs that interrupt DOS during file system operations have a similar restriction. During their interrupt processing procedures, they can't access the DOS file system because another program may be using the system.

Since the "get interrupt vector" method doesn't require the DOS file system, you must use it for these types of programs.

THE "OPEN HANDLE" TECHNIQUE

Most application programs can use the DOS "open handle" technique to test for the presence of the memory manager. This section describes how to use the technique and gives an example.

Caution.....

Don't use this technique if your program is a device driver or if it interrupts DOS during file system operations. Use the "get interrupt vector" technique described later in this chapter.

USING THE "OPEN HANDLE" TECHNIQUE

This section describes how to use the DOS "open handle" technique to test for the presence of the memory manager. Follow these steps in order:

1. Issue an "open handle" command (DOS function 3Dh) in "read only" access mode (register AL = 0). This function requires your program to point to an ASCII string which contains the path name of the file or device in which you're interested (register set DS:DX contains the pointer). In this case the file is actually the name of the memory manager.

   You should format the ASCII string as follows:

   ASCII_device_name DB "EMMXXXX0", 0

   The ASCII codes for the capital letters EMMXXXX0 are terminated by a byte containing a value of zero.

2. If DOS returns no error status code, skip Steps 3 and 4 and go to Step 5. If DOS returns a "Too many open files" error status code, go to Step 3. If DOS returns a "File/Path not found" error status code, skip Step 3 and go to Step 4.

3. If DOS returns a "Too many open files" (not enough handles), status code, your program should invoke the "open file" command before it opens any other files. This will guarantee that at least one file handle will be available to perform the function without causing this error.

   After the program performs the "open file" command, it should perform the test described in Step 6 and close the "file handle" (DOS function 3Eh). Don't keep the manager "open" after this status test is performed since "manager" functions are not available thru DOS. Go to Step 6.

4. If DOS returns a "File/Path not found", the memory manager is not installed. If your application requires the memory manager, the user will have to reboot the system with a disk containing the memory manager and the appropriate CONFIG.SYS file before proceeding.

5. If DOS doesn't return an error status code you can assume that either a device with the name EMMXXXX0 is resident in the system, or a file with this name is on disk in the current disk drive. Go to Step 6.

6. Issue an "I/O Control for Devices" command (DOS function 44h) with a "get device information" command (register AL = 0h). DOS function 44h determines whether EMMXXXX0 is a device or a file.

You must use the file handle (register BX) which you obtained in Step 1 to access the "EMM" device. This function returns the "device information" in a word (register DX). Go to step 7.

7. If DOS returns any error status code, you should assume that the memory manager device driver is not installed. If your application requires the memory manager, the user will have to reboot the system with a disk containing the memory manager and the appropriate CONFIG.SYS file before proceeding.

8. If DOS didn't return an error status, test the contents of bit 7 (counting from 0) of the "device information" word (register DX) the function returned. Go to Step 9.

9. If bit 7 of the "device information" word contains a zero, then EMMXXXX0 is a file, and the memory manager device driver is not present. If your application requires its presence, the user will have to reboot the system with a disk containing the memory manager and the appropriate CONFIG.SYS file before proceeding. If bit 7 contains a one, then EMMXXXX0 is a device. Go to Step 10.

10. Issue an "I/O Control for Devices" command (DOS function 44h) with a "get output status" command (register AL = 7h).

    You must use the file handle you obtained in Step 1 to access the "EMM" device (register BX). Go to Step 11.

11. If the expanded memory device driver is "ready," the memory manager passes a status value of "0FFh" in register AL. The status value is "00h" if the device driver is "not ready."

    If the memory manager device driver is "not ready" and your application requires its presence, the user will have to reboot the system with a disk containing the memory manager and the appropriate CONFIG.SYS file before proceeding.

    If the memory manager device driver is "ready", go to Step 12.

12. Issue a "Close File Handle" command (DOS function 3Eh) to close the expanded memory device driver. You must use the file handle you obtained in Step 1 to close the "EMM" device (register BX)

AN EXAMPLE OF THE "OPEN HANDLE" TECHNIQUE

The following procedure is an example of the "open handle" technique outlined in the previous section.

```
;-------------------------------------------------------------;
;   The following procedure tests for the presence of the EMM   ;
;   in the system.  It returns the CARRY FLAG SET if the EMM is ;
;   present.  If the EMM is not present, this procedure returns ;
;   the CARRY FLAG CLEAR.                                       ;
;-------------------------------------------------------------;

first_test_for_EMM  PROC  NEAR

PUSH DS
PUSH CS
POP  DS
MOV  AX, 3D00h                  ; issue device open in
LEA  DX, ASCII_device_name      ; read only mode
INT  21h
JC   first_test_for_EMM_error_exit ; test for error during open

MOV  BX, AX                     ; get file handle returned by DOS
MOV  AX, 4400h                  ; issue IOCTL get device info
INT  21h
JC   first_test_for_EMM_err_exit   ; test for get device info error

TEST DX, 0080h                  ; test to determine
JZ   first_test_for_EMM_err_exit   ; if ASCII_device_name
                                ; is a device or a file
MOV  AX, 4407h                  ; issue "IOCTL"
INT  21h
JC   first_test_for_EMM_error_exit ; test for IOCTL error
PUSH AX                         ; save "IOCTL" status
MOV  AH, 3Eh                    ; issue "close
INT  21h                        ; file handle"
```



```
POP  AX                         ; restore "IOCTL" status
CMP  AL, 0FFh                   ; test for "device ready" status
JNE  first_test_for_EMM_error_exit ; returned by the driver
first_test_for_EMM_exit:

POP  DS                         ; EMM is present in the system
STC
RET
```

```
first_test_for_EMM_error_exit:

POP  DS                         ; EMM is NOT present in the system
CLC
RET

ASCII_device_name               DB  "EMMXXXX0", 0

first_test_for_EMM ENDP
```


THE "GET INTERRUPT VECTOR" TECHNIQUE

Any type of program can use the DOS "get interrupt vector" technique to test for the presence of the memory manager. This section describes how to use the technique and gives an example.

Caution.....

Be sure to use this technique (and not the "open handle technique) if your program is a device driver or if it interrupts DOS during file system operations.

USING THE "GET INTERRUPT VECTOR" TECHNIQUE

This section describes how to use the DOS "get interrupt vector" technique to test for the presence of the memory manager. Follow these steps in order:

1. Issue a "get vector" command (DOS function 35h) to obtain the contents of interrupt vector array entry number 67h (addresses 0000:019C thru 0000:019F).

   The memory manager uses this interrupt vector to perform all manager functions. The Offset portion of this interrupt service routine address is stored in the word located at address 0000:019Ch; the Segment portion is stored in the word located at address 0000:019Eh.

2. Compare the "device name field" with the contents of the ASCII string which starts at the address specified by the segment portion of the contents of interrupt vector address 67h and a fixed offset of 000Ah. If DOS loaded the memory manager at boot time this name field will have the name of the device in it.

Since the memory manager is implemented as a character device driver, its program origin is 0000h. Device drivers are required to have a "device header" located at the program origin. Within the "device header" is an 8 byte "device name field." For a character mode device driver this name field is always located at offset 000Ah within the device header. The device name field contains the name of the device which DOS uses when it references the device.

If the result of the "string compare" in this technique is positive, the memory manager driver is present.

AN EXAMPLE OF THE "GET INTERRUPT VECTOR" TECHNIQUE

The following procedure is an example of the "get interrupt vector" technique outlined in the previous section.

```
;-------------------------------------------------------------;
;   The following procedure tests for the presence of the EMM   ;
;   in the system.  It returns the CARRY FLAG SET if the EMM is ;
;   present.  If the EMM is not present, this procedure returns ;
;    the CARRY FLAG CLEAR.                                      ;
;-------------------------------------------------------------;
```



```
second_test_for_EMM             PROC    NEAR

PUSH DS
PUSH CS
POP  DS
MOV  AX, 3567h                  ; issue "get interrupt vector"
INT  21h
MOV  DI, 000Ah                  ; use the SEGMENT in ES returned by
                        ; DOS, place the "device name
                        ; field" OFFSET in DI
LEA  SI, ASCII_device_name      ; place the OFFSET of the device
                        ; name string in SI, the SEGMENT
                        ; is already in DS
MOV  CX, 8                      ; compare the name strings
CLD
REPE CMPSB
JNE  second_test_for_EMM_error_exit

second_test_for_EMM_exit:
```

```
POP  DS                 ; EMM is present in the system
STC
RET

second_test_for_EMM_error_exit:

POP  DS                 ; EMM is NOT present in the system
CLC
RET

ASCII_device_name       DB "EMMXXXX0"

second_test_for_EMM      ENDP
```


APPENDIX C

EXPANDED MEMORY MANAGER IMPLEMENTATION GUIDELINES

In addition to the functional specification, the expanded memory manager should provide certain resources. The following guidelines are provided so required resources are present in expanded memory managers which comply with this version of the LIM specification.

- The amount of expanded memory supported:

  Up to a maximum of 32M bytes of expanded memory should be supported.

- The number of handles supported:

  The maximum number expanded memory handles provided should be 255, the minimum should be 64.

- Handle Numbering:

  Although a handle is a word quantity, there is a maximum of 255 handles, including the operating system handle. This specification defines the handle word as follows: the low byte of the word is the actual handle value, the high byte of the handle is set to 00 by the memory manager. Previous versions of this specification did not specify the value of handles.

- New handle type: Handles versus Raw Handles:

The difference between a raw handle and a regular handle is slight. If you use Function 27 to "Allocate raw pages to a handle", what is returned in DX is termed a raw handle. The raw handle does not necessarily refer to 16 Kbyte pages. Instead it refers to the "raw" page size, which depends on the expanded memory hardware.

An application program can use Function 26 to find the raw page size, and by using the raw handle Function 27 returns, it can access them with the finer resolution that a particular expanded memory board may allow.

On the other hand, applications which use Function 4 to "allocate pages to handle" receive a handle which always refers to 16K-byte pages. On expanded memory boards with smaller raw pages, the EMM driver will allocate and maintain the number of raw pages it takes to create a single composite 16K-byte page. The difference between the expanded memory boards raw page size and the 16K-byte LIM page size is transparent to the application when it is using a handle obtained with Function 4.

The memory manager must differentiate between pages allocated to handles and pages allocated to raw handles. The meaning of a call to the driver changes depending on whether a handle or a raw handle is passed to the memory manager. If, for example, a handle is passed to Function 18 (Reallocate), the memory manager will

increase or decrease the number of 16K-byte pages allocated to the handle. If Function 18 is passed a raw handle, the memory manager will increase or decrease the number of raw (non-16K-byte) pages allocated to the raw handle. For LIM standard boards, there is no difference between pages and raw pages.

- The system Raw Handle (Raw Handle = 0000):

  For expanded memory boards that can remap the memory in the lower 640K-byte address space, managing the pages of memory which are used to fill in the lower 640K can be a problem. To solve this problem, the memory manager will create a raw handle with a value of 0000 when DOS loads the manager. This raw handle is called the system handle.

At power up, the memory manager will allocate all of the pages that are mapped into the lower 640K bytes to the system handle. These pages should be mapped in their logical order. For example, if the system board supplies 256K bytes of RAM, and the 384K bytes above it is mappable, the system handle should have its logical page zero mapped into the first physical page at 256K, its logical page one mapped into the next physical page, and so on.

The system handle should deal with raw pages. To release some of these pages so application programs can use them, an operating system could decrease the number of pages allocated to the system handle with the "Reallocate" function. Invoking the "Deallocate" function would decrease the system handle to zero size, but it must not deallocate the raw handle itself. The "Deallocate" function treats the system handle differently than it treats other raw handles. If the operating system can ever be "exited" (for example, the way Windows can be exited), it must increase the size of the system handle back to what is needed to fill 640K and map these logical pages back into physical memory before returning to DOS.

There are two functional special cases for this handle:

- The first special case deals with Function 4 (Allocate Pages). This function must never return zero as a handle value. Applications must always invoke Function 4 to allocate pages and obtain a handle which identifies its pages. Since Function 4 will never return a handle value of zero, an application will never gain access to this special handle.

- The second special case deals with Function 6 (Deallocate Pages). If the operating system uses Function 6 to deallocate the pages which are allocated to the system handle, the pages will be returned to the manager for use, but the handle will not be available for reassignment. The manager should treat a "deallocate pages" function request for this handle the same as a "reallocate pages" function request, where the number of pages to reallocate to this handle is zero.

- Terminate and Stay Resident (TSR) Program Cooperation In order for TSR's to cooperate with each other and with other applications, TSR's must follow this rule: a program may only remap the DOS partition it lives in.

  This rule applies at all times, even when no expanded memory is present.

- Accelerator Cards: To support generic accelerator cards, the support of Function 34, as defined by AST, is encouraged.

APPENDIX D

OPERATING SYSTEM/ENVIRONMENT USE OF FUNCTION 28

All expanded memory boards have a set of registers that "remember" the logical to physical page mappings. Some boards have extra (or alternate) sets of these mapping registers. Because no expanded memory board can supply an infinite number of alternate map register sets, this specification provides a way to simulate them using Function 28 (Alternate Map Register Set).

EXAMPLES

For the examples in this section, assume the hardware supports alternate map register sets. First Windows is brought up, then "Reversi" is started. Then control is switched back to the MS-DOS Executive. For this procedure, here are the calls to the expanded memory manager:

EXAMPLE 1

Allocate alt reg set ; Start up the MS-DOS
(for the MS-DOS executive) ; Executive

Set alt reg set
(for MS-DOS Executive)

Allocate alt reg set ; Start up Reversi
(for Reversi)

Set alt reg set
(for Reversi)

Map pages
(for Reversi)

```
Set alt reg set             ; Switch back to MS-DOS
(for MS-DOS Executive)      ; Executive
```

Notice this procedure needed no "get" calls because the register set contained all the information needed to save a context. However, using "Get" calls would have no ill effects.

159

EXAMPLE 2

```
Allocate alt reg set        ; Start up MS-DOS
(for MS-DOS Executive)      ; Executive

Set alt reg set
(for MS-DOS Executive)

Get alt reg set
(for MS-DOS Executive)

Allocate alt reg set        ; Start up Reversi
(for Reversi)

Set alt reg set
(for Reversi)

Map pages
(for Reversi)

Get alt reg set
(for Reversi)

Set alt reg set             ; Switch back to MS-DOS
(for MS-DOS Executive)      ; Executive
```

The important point to follow is that a Set must always precede a Get. The model of Set then Get is the inverse of what interrupt handlers use, which is Get then Set (Get the old map context and Set the new one). Another crucial point is that an alternate map register set must have the current mapping when allocated; otherwise, the Set will create chaos.

What happens if this is simulated in software? The same Set and Get model applies. The main difference is where the context is saved.

Since the allocate call is dynamic and there is no limit on the number of sets allocated, the OS/E must supply the space required. Device drivers cannot allocate space dynamically, since the request would fail. If the Allocate register set call returns a status indicating the alternate map register sets aren't supported, the OS/E must allocate space for the context. It must also initialize the context using Function 15. At that point it can do the Set, passing a pointer to the map context space. On the Get call, the EMM driver is to return a pointer to the same context space.

EXAMPLE 3

```
Allocate alt reg set              ; Start up MS-DOS
(for the MS-DOS executive)        ;Executive

Get Page Map
(for the MS-DOS executive)

Set alt reg set
(for MS-DOS Executive)
```



```
Allocate alt reg set              ; Start up Reversi
(for Reversi)

Set alt reg set
(for Reversi)

Map pages
(for Reversi)

Get Page Map
(for Reversi)

Set alt reg set                   ; Switch back to MS-DOS
(for MS-DOS Executive)            ; Executive
```


GLOSSARY

The following terms are used frequently in this specification:

Allocate
: To reserve a specified amount of expanded memory pages.

Application Program
: An application program is the program you write and your customer uses. Some categories of application software are word processors, database managers, spreadsheet managers, and project managers.

Conventional Memory
: The memory between 0 and 640K bytes, address range 00000h thru 9FFFFh.

Deallocate
: To return previously allocated expanded memory to the memory manager.

EMM
: See Expanded Memory Manager.

Expanded Memory
: Expanded memory is memory outside DOS's 640K-byte limit (usually in the range of C0000h through EFFFFH).

Expanded Memory Manager (EMM)
: A device driver that controls the interface between DOS application programs and expanded memory.

Extended Memory
: The 15M-byte address range between 100000h thru FFFFFFh available on an 80286 processor when it is operating in protected virtual address mode.

Handle
: A value that the EMM assigns and uses to identify a block of memory requested by an application program. All allocated logical pages are associated with a particular handle.

Logical Page
: The EMM allocates expanded memory in units (typically 16K-bytes) called logical pages.

Mappable Segment
: A 16K-byte region of memory which can have a logical page mapped at it.

Map Registers
: The set of registers containing the current mapping context of the EMM hardware.

Mapping
: The process of making a logical page of memory appear at a physical page.

Mapping Context
: The contents of the mapping registers at a specific instant. This context represents a map state.

Page Frame
: A collection of 16K-byte contiguous physical pages from which an application program accesses expanded memory.

Page Frame Base Address
: A page frame base address is the location (in segment format) of the first byte of the page frame.

Physical Page
: A physical page is the range of memory addresses occupied by a single 16K-byte page.

Raw Page
: The smallest unit of mappable memory that an expanded memory board can supply.

Resident Application Program
: A resident application program is loaded by DOS, executes, and remains resident in the system after it returns control to DOS. This type of program occupies memory and is usually invoked by the operating system, an application program, or the hardware. Some examples of resident application programs are RAM disks, print spoolers, and "pop-up" desktop programs.

Status Code
: A code that an EMM function returns which indicates something about the result of running the function. Some status codes indicate whether the function worked correctly and others may tell you something about he expanded memory hardware or software.

Transient Application
: A transient application program is loaded Program by DOS, executes, and doesn't remain in the system after it returns control to DOS. After a transient application program returns control to DOS, the memory it used is available for other programs.

Unmap

To make a logical page inaccessible for reading or writing.

163LOTUS /INTEL /MICROSOFT

EXPANDED MEMORY SPECIFICATION

Version 4.0

300275-005

October, 1987

Copyright (c) 1987

Lotus Development Corporation
55 Cambridge Parkway
Cambridge, MA 02142

Intel Corporation
5200 NE Elam Young Parkway
Hillsboro, OR 97124

Microsoft Corporation
16011 NE 36TH Way
Box 97017
Redmond, WA 98073

This specification was jointly developed by Lotus Development Corporation, Intel Corporation, and Microsoft Corporation. Although it has been released into the public domain and is not confidential or proprietary, the specification is still the copyright and property of Lotus Development Corporation, Intel Corporation, and Microsoft Corporation.

DISCLAIMER OF WARRANTY

LOTUS DEVELOPMENT CORPORATION, INTEL CORPORATION, AND MICROSOFT CORPORATION EXCLUDE ANY AND ALL IMPLIED WARRANTIES, INCLUDING WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE. NEITHER LOTUS NOR INTEL NOR MICROSOFT MAKE ANY WARRANTY OF REPRESENTATION, EITHER EXPRESS OR IMPLIED, WITH RESPECT TO THIS SPECIFICATION, ITS QUALITY, PERFORMANCE, MERCHANTABILITY, OR FITNESS FOR A PARTICULAR PURPOSE. NEITHER LOTUS NOR INTEL NOR MICROSOFT SHALL HAVE ANY LIABILITY FOR SPECIAL, INCIDENTAL, OR CONSEQUENTIAL DAMAGES ARISING OUT OF OR RESULTING FROM THE USE OR MODIFICATION OF THIS SPECIFICATION.

This specification uses the following trademarks:

Intel is a trademark of Intel Corporation
Lotus is a trademark of Lotus Development Corporation
Microsoft is a trademark of Microsoft Corporation

CHAPTER 1

INTRODUCTION

Because even the maximum amount (640K bytes) of conventional memory isn't always enough for large application programs, Lotus Development Corporation, Intel Corporation, and Microsoft Corporation created the Lotus/Intel/Microsoft (LIM) Expanded Memory Specification.

The LIM Expanded Memory Specification defines the software interface between the Expanded Memory Manager (EMM) -- a device driver that controls and manages expanded memory -- and application programs that use expanded memory.

WHAT IS EXPANDED MEMORY?

Expanded memory is memory beyond DOS's 640K-byte limit. The LIM specification supports up to 32M bytes of expanded memory. Because the 8086, 8088, and 80286 (in real mode) microprocessors can physically address only 1M byte of memory, they access expanded memory through a window in their physical address range. The next section explains how this is done.

HOW EXPANDED MEMORY WORKS

Expanded memory is divided into segments called logical pages. These pages are typically 16K-bytes of memory. Your computer accesses logical pages through a physical block of memory called a page frame. The page frame contains multiple physical pages, pages that the microprocessor can address directly. Physical pages are also typically 16K bytes of memory.

This page frame serves as a window into expanded memory. Just as your computer screen is a window into a large spreadsheet, so the page frame is a window into expanded memory.

A logical page of expanded memory can be mapped into (made to appear in) any one of the physical pages in the page frame. Thus, a read or write to the physical page actually becomes a read or write to the associated logical page. One logical page can be mapped into the page frame for each physical page.

The page frame is located above 640K bytes. Normally, only video adapters, network cards, and similar devices exist between 640K and 1024K.

This specification also defines methods for operating systems and environments to access expanded memory through physical pages below 640K bytes. These methods are intended for operating system/environment developers only.

CHAPTER 2

WRITING PROGRAMS THAT USE EXPANDED MEMORY

This chapter describes what every program must do to use expanded memory and describes more advanced techniques of using expanded memory.

This chapter also lists programming guidelines you should follow when writing programs that use expanded memory and provides the listings of some example programs.

WHAT EVERY PROGRAM MUST DO

This section describes the steps every program must take to use expanded memory.

In order to use expanded memory, applications must perform these steps in the following order:

1. Determine if EMM is installed.
2. Determine if enough expanded memory pages exist for your application. (Function 3)
3. Allocate expanded memory pages. (Function 4 or 18)
4. Get the page frame base address. (Function 2)
5. Map in expanded memory pages. (Function 5 or 17)

6. Read/write/execute data in expanded memory, just as if it were conventional memory.

7. Return expanded memory pages to expanded memory pool before exiting. (Function 6 or 18)

Table 2-1 overviews the functions while Chapter 3 describes each of these functions in detail. Example programs at the end of this chapter illustrate using expanded memory.

TABLE 2-1. THE BASIC FUNCTIONS

Function	Description
1	The Get Status function returns a status code indicating whether the memory manager hardware is working correctly.
2	The Get Page Frame Address function returns the address where the 64K-byte page frame is located.
3	The Get Unallocated Page Count function returns the number of unallocated pages (pages available to your program) and the total number of pages in expanded memory.

4	The Allocate Pages function allocates the number of pages requested and assigns a unique EMM handle to these pages.
5	The Map/Unmap Handle Page function maps a logical page to a specific physical page anywhere in the mappable regions of system memory.
6	The Deallocate Pages deallocates the logical pages currently allocated to an EMM handle.
7	The Get Version function returns the version number of the memory manager software.

ADVANCED PROGRAMMING

In addition to the basic functions, the Lotus/Intel/Microsoft Expanded Memory Specification provides several advanced functions which enhance the capabilities of software that uses expanded memory.

The following sections describe the advanced programming capabilities and list the advanced EMM functions.

Note.....

Before using the advanced functions, programs should first call Function 7 (Get Version) to determine whether the installed version of EMM supports these functions.

SAVING THE STATE OF MAPPING HARDWARE

Some software (such as interrupt service routines, device drivers, and resident software) must save the current state of the mapping hardware, switch mapping contexts, manipulate sections of expanded memory, and restore the original context of the memory mapping hardware. Use Functions 8 and 9 or 15 and 16 to save the state of the hardware.

RETRIEVING HANDLE AND PAGE COUNTS

Some utility programs need to keep track of how expanded memory is being used; use Functions 12 through 14 to do this.

MAPPING AND UNMAPPING MULTIPLE PAGES

Mapping multiple pages reduces the overhead an application must perform during mapping. Function 17 lets a program map (or unmap) multiple pages at one time.

In addition, you can map pages using segment addresses instead of physical pages. For example, if the page frame base address is set to D000, you can map to either physical page 0 or segment D000. Function 25 (Get Mappable Physical Address Array) returns a cross reference between all expanded memory physical pages and their corresponding segment values.

REALLOCATING PAGES

Reallocating pages (Function 18) lets applications dynamically allocate expanded memory pages without acquiring another handle or obtain a handle without allocating pages. Reallocating pages is an efficient means for applications to obtain and release expanded memory pages.

USING HANDLES AND ASSIGNING NAMES TO HANDLES

This specification lets you associate a name with a handle, so a family of applications can share information in expanded memory. For example, a software package consisting of a word processor, spreadsheet, and print spooler can share the same data among the different applications. The print spooler could use a handle name to reference data that either the spreadsheet or word processor put in expanded memory and could check for data in a particular handle name's expanded memory pages.

Use Function 20 (Set Handle Name subfunction) to assign a handle name to an EMM handle or Function 21 (Search for Named Handle subfunction) to obtain the EMM handle associated with the handle name. In addition, you can use Function 14 (Get Handle Pages) to determine the number of expanded memory pages allocated to an EMM handle.

USING HANDLE ATTRIBUTES

In addition to naming a handle, you can use Function 19 to associate an attribute (volatile or non-volatile) with a handle name. A non-volatile attribute enables expanded memory pages to preserve their data through a warmboot.

With a volatile attribute, the data is not preserved. The default attribute for handles is volatile.

Because using this function depends on the capabilities of the expanded memory hardware installed in the system, you should use the Get Attribute Capability subfunction before attempting to assign an attribute to a handle's pages.

ALTERING PAGE MAPS AND JUMPING/CALLING

You can use Functions 22 (Alter Page Map and Jump) and 23 (Alter Page Map and Call) to map a new set of values into the map registers and transfer program control to a specified address within expanded memory. These functions can be used to load and execute code in expanded memory. An application using this feature can significantly reduce the amount of conventional memory it requires. Programs can load needed modules into expanded memory at run time and use Functions 22 and 23 to transfer control to these modules.

Using expanded memory to store code can improve program execution in many ways. For example, sometimes programs need to be divided into small overlays because of conventional memory size limitations. Overlays targeted for expanded memory can be very large because LIM EMS 4.0 supports up to 32M bytes of expanded memory. This method of loading overlays improves overall system performance by conserving

conventional memory and eliminating conventional memory allocation errors.

MOVING OR EXCHANGING MEMORY REGIONS

Using Function 24 (Move/Exchange Memory Region), you can easily move and exchange data between conventional and expanded memory. Function 24 can manipulate up to one megabyte of data with one function call. Although applications can perform this operation without this function, having the expanded memory manager do it reduces the amount of overhead for the application. In addition, this function checks for overlapping regions and performs all the necessary mapping, preserving the mapping context from before the exchange/move call.

GETTING THE AMOUNT OF MAPPABLE MEMORY

Function 25 enables applications to determine the total amount of mappable memory the hardware/system currently supports. Not all expanded memory boards supply the same number of physical pages (map registers).

The Get Mappable Physical Address Array Entries subfunction returns the total number of physical pages the expanded memory hardware/system is capable of supporting. The Get Mappable Physical Address Array subfunction returns a cross reference between physical page numbers and the actual segment address for each of the physical pages.

OPERATING SYSTEM FUNCTIONS

In addition to the functions for application programs, this specification defines functions for operating systems/environments. These functions can be disabled at any time by the operating system/environment, so programs should not depend on their presence. Applications that avoid this warning and use these functions run a great risk of being incompatible with other programs, including the operating system.

TABLE 2-2. THE ADVANCED FUNCTIONS

Function Description

---

8 The Save Page Map saves the contents of the page mapping registers from all expanded memory boards in an internal save area.

9 The Restore Page Map function restores (from an internal save area) the page mapping register contents on the expanded memory boards for a particular EMM handle.

10 Reserved.

11 Reserved.

12 The Get Handle Count function returns the number of open EMM handles in the system.

13 The Get Handle Pages function returns the number of pages allocated to a specific EMM handle.

14 The Get All Handle Pages function returns an array of the active EMM handles and the number of pages allocated to each one.

15 The Get/Set Page Map subfunction saves or restores the mapping context for all mappable memory regions (conventional and expanded) in a destination array which the application supplies.

16 The Get/Set Partial Page Map subfunction provides a mechanism for saving a partial mapping context for specific mappable memory regions in a system.

17 The Map/Unmap Multiple Handle Pages function can, in a single invocation, map (or unmap) logical pages into as many physical pages as the system supports.

18 The Reallocate Pages function can increase or decrease the amount of expanded memory allocated to a handle.

19 The Get/Set Handle Attribute function allows an application program to determine and set the attribute associated with a handle.

20 The Get/Set Handle Name function gets the eight character name currently assigned to a handle and can assign an eight character name to a handle.

21 The Get Handle Directory function returns information about active handles and the names assigned to each.

22 The Alter Page Map & Jump function alters the memory mapping context and transfers control to the specified address.

23 The Alter Page Map & Call function alters the specified mapping context and transfers control to the specified address. A return can then restore the context and return control to the caller.

24 The Move/Exchange Memory Region function copies or exchanges a region of memory from conventional to conventional memory, conventional to expanded memory, expanded to conventional memory, or expanded to expanded memory.

25 The Get Mappable Physical Address Array function returns an array containing the segment address and physical page number for each mappable physical page in a system.

26 The Get Expanded Memory Hardware Information function returns an array containing the hardware capabilities of the expanded memory system.

27 The Allocate Standard/Raw Pages function allocates the number of standard or non-standard size pages that the operating system requests and assigns a unique EMM handle to these pages.

28 The Alternate Map Register Set function enables an application to simulate alternate sets of hardware mapping registers.

29 The Prepare Expanded Memory Hardware for Warm Boot function prepares the expanded memory hardware for an "impending" warm boot.

30 The Enable/Disable OS/E function enables operating systems developers to enable and disable functions designed for operating system use.

PROGRAMMING GUIDELINES

The following section contains guidelines for programmers writing applications that use EMM.

o Do not put a program's stack in expanded memory.

- Do not replace interrupt 67h. This is the interrupt vector the EMM uses. Replacing interrupt 67h could result in disabling the Expanded Memory Manager.

- Do not map into conventional memory address space your application doesn't own. Applications that use the EMM to swap into conventional memory space, must first allocate this space from the operating system. If the operating system is not aware that a region of memory it manages is in use, it will think it is available. This could have disastrous results. EMM should not be used to 'allocate' conventional memory. DOS is the proper manager of conventional memory space. EMM should only be used to swap data in conventional memory space previously allocated from DOS.

- Applications that plan on using data aliasing in expanded memory must check for the presence of expanded memory hardware. Data aliasing occurs when mapping one logical page into two or more mappable segments. This makes one 16K-byte expanded memory page appear to be in more than one 16K-byte memory address space. Data aliasing is legal and sometimes useful for applications.

  Software-only expanded memory emulators cannot perform data aliasing. A simple way to distinguish software emulators from actual expanded memory hardware is to attempt data aliasing and check the results. For example, map one logical page into four physical pages. Write to physical page 0. Read physical pages 1-3 to see if the data is there as well. If the data appears in all four physical pages, then expanded memory hardware is installed in the system, and data aliasing is supported.

- Applications should always return expanded memory pages to the expanded memory manager upon termination. These pages will be made available for other applications. If unneeded pages are not returned to the expanded memory manager, the system could 'run out' of expanded memory pages or expanded memory handles.

- Terminate and stay resident programs (TSR's) should ALWAYS save the state of the map registers before changing them. Since TSR's may interrupt other programs which may be using expanded memory, they must not change the state of the page mapping registers without first saving them. Before exiting, TSR's must restore the state of the map registers.

  The following sections describe the three ways to save and restore the state of the map registers.

1 Save Page Map and Restore Page Map (Functions 8 and 9). This is the simplest of the three methods. The EMM saves the map register contents in its own data structures -- the application does not need to provide extra storage

locations for the mapping context. The last mapping context to be saved, under a particular handle, will be restored when a call to Restore Page Map is issued with the same handle. This method is limited to one mapping context for each handle and saves the context for only LIM standard 64K-byte page frames.

2 Get/Set Page Map (Function 15). This method requires the application to allocate space for the storage array. The EMM saves the mapping context in an array whose address is passed to the EMM. When restoring the mapping context with this method, an application passes the address of an array which contains a previously stored mapping context.

This method is preferable if an application needs to do more than one save before a restore. It provides a mechanism for switching between more than one mapping context.

3 Get/Set Partial Page Map (Function 16). This method provides a way for saving a partial mapping context. It should be used when the application does not need to save the context of all mappable memory. This function also requires that the storage array be part of the application's data.

- All functions using pointers to data structures must have those data structures in memory which will not be mapped out. Functions 22 and 23 (Alter Map and Call and Alter Map and Jump) are the only exceptions.

CHAPTER 3

EMM FUNCTIONS

This chapter provides you with a standardized set of expanded memory functions. Because they are standardized, you avoid potential compatibility problems with other expanded memory programs that also adhere to the memory manager specification. Programs that deal directly with the hardware or that don't adhere to this specification will be incompatible.

Table 3-1 presents a sequential list of the EMM functions. The remainder of this chapter provides detailed descriptions of each function.

TABLE 3-1. LIST OF EMM FUNCTIONS

Number	Function Name	Hex Value	Page
1	Get Status	40h	14
2	Get Page Frame Segment Address	41h	15
3	Get Unallocated Page Count	42h	16
4	Allocate Pages	43h	17
5	Map/Unmap Handle Page	44h	20
6	Deallocate Pages	45h	22
7	Get Version	46h	24
8	Save Page Map	47h	26
9	Restore Page Map	48h	28
10	Reserved	49h	30
11	Reserved	4Ah	31
12	Get Handle Count	4Bh	32
13	Get Handle Pages	4Ch	34
14	Get All Handle Pages	4Dh	36
15	Get Page Map	4E00h	38
	Set Page Map	4E01h	40
	Get & Set Page Map	4E02h	42

Get Size of Page Map Save Array 4E03h 44

11

16 Get Partial Page Map 4F00h 46

Set Partial Page Map 4F01h 48

Get Size of Partial Page Map Save Array 4F02h 50

17 Map/Unmap Multiple Handle Pages
(Physical page number mode) 5000h 52

Map/Unmap Multiple Handle Pages
(Segment address mode) 5001h 56

18 Reallocate Pages 51h 59

19 Get Handle Attribute 5200h 62

Set Handle Attribute 5201h 65

Get Handle Attribute Capability 5202h 67

20 Get Handle Name 5300h 68

Set Handle Name 5301h 70

21 Get Handle Directory 5400h 72

Search for Named Handle 5401h 74

Get Total Handles 5402h 76

22 Alter Page Map & Jump
(Physical page number mode) 5500h 77

Alter Page Map & Jump
(Segment address mode) 5501h 77

23 Alter Page Map & Call
(Physical page number mode) 5600h 80

Alter Page Map & Call
(Segment address mode) 5601h 80

Get Page Map Stack Space Size 5602h 84

24 Move Memory Region 5700h 86

Exchange Memory Region 5701h 91

25 Get Mappable Physical Address Array 5800h 96

Get Mappable Physical Address Array Entries 5801h 99

26 Get Hardware Configuration Array 5900h 101

Get Unallocated Raw Page Count 5901h 104

27 Allocate Standard Pages 5A00h 106

12

Allocate Raw Pages 5A01h 109

28 Get Alternate Map Register Set 5B00h 112

Set Alternate Map Register Set 5B01h 117

Get Alternate Map Save Array Size 5B02h 120

Allocate Alternate Map Register Set 5B03h 122

Deallocate Alternate Map Register Set 5B04h 124

Allocate DMA Register Set 5B05h 126

Enable DMA on Alternate Map Register Set 5B06h 128

Disable DMA on Alternate Map Register Set 5B07h 130

Deallocate DMA Register Set 5B08h 132

29 Prepare Expanded Memory Hardware for Warmboot 5Ch 134

30 Enable OS/E Function Set 5D00h 135

Disable OS/E Function Set 5D01h 138

Return OS/E Access Key 5D02h 140

13

FUNCTION 1 GET STATUS

PURPOSE

The Get Status function returns a status code indicating whether the memory manager is present and the hardware is working correctly.

CALLING PARAMETERS

AH = 40h
Contains the Get Status function.

REGISTERS MODIFIED

AX

STATUS

AH = 0 SUCCESSFUL
The manager is present in the system, and the hardware is working correctly.

AH = 80h NON-RECOVERABLE
The manager detected a malfunction in the memory manager software.

AH = 81h NON-RECOVERABLE
The manager detected a malfunction in the expanded memory hardware.

AH = 84h NON-RECOVERABLE
The function code passed to the memory manager is not defined.

EXAMPLE

```
MOV  AH, 40h                ; load function code
INT  67h                    ; call the memory manager
OR   AH, AH                 ; check EMM status
JNZ  emm_err_handler        ; jump to error handler on error
```


FUNCTION 2 GET PAGE FRAME ADDRESS

PURPOSE

The Get Page Frame Address function returns the segment address where the page frame is located.

CALLING PARAMETERS

AH = 41h
Contains the Get Page Frame function.

RESULTS

These results are valid only if the status returned is zero.

BX = page frame segment address
Contains the segment address of the page frame.

REGISTERS MODIFIED

AX, BX

STATUS

AH = 0 SUCCESSFUL
The manager has returned the page frame address in the BX register.

AH = 80h NON-RECOVERABLE
The manager detected a malfunction in the memory manager software.

AH = 81h NON-RECOVERABLE
The manager detected a malfunction in the expanded memory hardware.

AH = 84h NON-RECOVERABLE
The function code passed to the memory manager is not defined.

EXAMPLE

```
page_frame_segment              DW  ?

MOV  AH, 41h                    ; load function code
INT  67h                        ; call the memory manager
OR   AH, AH                     ; check EMM status
JNZ  emm_err_handler            ; jump to error handler on error
MOV  page_frame_segment, BX     ; save page frame address
```

15

FUNCTION 3 GET UNALLOCATED PAGE COUNT

PURPOSE

The Get Unallocated Page Count function returns the number of unallocated pages and the total number of expanded memory pages.

CALLING PARAMETERS

AH = 42h Contains the Get Unallocated Page Count function.

RESULTS

These results are valid only if the status returned is zero.

BX = unallocated pages
The number of expanded memory pages that are currently available for use (unallocated).

DX = total pages
The total number expanded memory pages.

REGISTERS MODIFIED

AX, BX, DX

STATUS

AH = 0 SUCCESSFUL
The manager has returned the number of unallocated pages and the number of total pages in expanded memory.

AH = 80h NON-RECOVERABLE
The manager detected a malfunction in the memory manager software.

AH = 81h NON-RECOVERABLE
The manager detected a malfunction in the expanded memory hardware.

AH = 84h NON-RECOVERABLE
The function code passed to the memory manager is not defined.

EXAMPLE

```
un_alloc_pages          DW  ?
total_pages             DW  ?

MOV  AH, 42h            ; load function code
INT  67h                ; call the memory manager
OR   AH, AH             ; check EMM status
JNZ  emm_err_handler    ; jump to error handler on error
MOV  un_alloc_pages, BX ; save unallocated page count
MOV  total_pages, DX    ; save total page count
```


FUNCTION 4 ALLOCATE PAGES

PURPOSE

The Allocate Pages function allocates the number of pages requested and assigns a unique EMM handle to these pages. The EMM handle owns these pages until the application deallocates them.

Handles which are assigned using this function will have 16K-byte pages, the size of a standard expanded memory page. If the expanded memory board hardware isn't able to supply 16K-byte pages, it will emulate them by combining multiple non-standard size pages to form a single 16K-byte page. All application programs and functions that use the handles this function returns will deal with 16K-byte pages.

The numeric value of the handles the EMM returns are in the range of 1 to 254 decimal (0001h to 00FEh). The OS handle (handle value 0) is never returned by the Allocate Pages function. Also, the uppermost byte of the handle will be zero and cannot be used by the application. A memory manager should be able to supply up to 255 handles, including the OS handle. An application can use Function 21 to find out how many handles an EMM supports.

Allocating zero pages to a handle is not valid. If an application needs to allocate 0 pages to a handle it should use Function 27 provided for this purpose.

Note.....

This note affects expanded memory manager implementers and operating system developers only. Applications should not use the following characteristic of the memory manager. An application violating this rule will be incompatible with future versions of Microsoft's operating systems and environments.

To be compatible with this specification, an expanded memory manager will provide a special handle which is available to the operating system only. This handle will have a value of 0000h and will have a set of pages allocated to it when the expanded memory manager driver installs. The pages that the memory manager will automatically allocate to handle 0000h are those that backfill conventional memory. Typically, this backfill occurs between addresses 40000h (256K) and 9FFFFh (640K). However, the range can extend below and above this limit if the hardware and memory manager have the capability.

An operating system won't have to invoke Function 4 to obtain this handle because it can assume the handle already exists and is available for use immediately after the expanded memory device driver installs. When an operating system wants to use this handle, it uses the special handle value of 0000h. The operating system will be able to invoke any EMM function using this special handle value. To allocate pages to this handle, the operating system need only invoke Function 18 (Reallocate pages)..

There are two special cases for this handle:

1. Function 4 (Allocate Pages). This function must never return zero as a handle value. Applications must always invoke Function 4 to allocate pages and obtain a handle which identifies the pages which belong to it. Since Function 4 never returns a handle value of zero, an application will never gain access to this special handle.

2. Function 6 (Deallocate Pages). If the operating system uses it to deallocate the pages which are allocated to this special handle, the pages the handle owns will be returned to the manager for use. But the handle will not be available for reassignment. The manager should treat a deallocate pages function request for this handle the same as a reallocate pages function request, where the number of pages to reallocate to this handle is zero.

CALLING PARAMETERS

AH = 43h
Contains the Allocate Pages function.

BX = num_of_pages_to_alloc
Contains the number of pages you want your program to allocate.

RESULTS

These results are valid only if the status returned is zero.

DX = handle
Contains a unique EMM handle. Your program must use this EMM handle (as a parameter) in any function that requires it. You can use up to 255 handles. The uppermost byte of the handle will be zero and cannot be used by the application.

REGISTERS MODIFIED

AX, DX

STATUS

AH = 0 SUCCESSFUL
The manager has allocated the requested pages to the assigned EMM handle.

AH = 80h NON-RECOVERABLE
The manager detected a malfunction in the memory manager software.

AH = 81h NON-RECOVERABLE
The manager detected a malfunction in the expanded memory hardware.

AH = 84h NON-RECOVERABLE
The function code passed to the memory manager is not defined.

AH = 85h RECOVERABLE
All EMM handles are being used.

AH = 87h RECOVERABLE
There aren't enough expanded memory pages present in the system to satisfy your program's request.

AH = 88h RECOVERABLE
There aren't enough unallocated pages to satisfy your program's request.

AH = 89h RECOVERABLE
Your program attempted to allocate zero pages.

EXAMPLE

```
num_of_pages_to_alloc        DW  ?
emm_handle                   DW  ?
```

```
MOV  BX, num_of_pages_to_alloc     ; load number of pages
MOV  AH, 43h                       ; load function code
INT  67h                           ; call the memory manager
OR   AH, AH                        ; check EMM status
JNZ  emm_err_handler               ; jump to error handler on error
MOV  emm_handle, DX                ; save EMM handle
```


FUNCTION 5 MAP/UNMAP HANDLE PAGES

PURPOSE

The Map/Unmap Handle Page function maps a logical page at a specific physical page anywhere in the mappable regions of system memory. The lowest valued physical page numbers are associated with regions of memory outside the conventional memory range. Use Function 25 (Get Mappable Physical Address Array) to determine which physical pages within a system are mappable and determine the segment addresses which correspond to a specific physical page number. Function 25 provides a cross reference between physical page numbers and segment addresses.

This function can also unmap physical pages, making them inaccessible for reading or writing. You unmap a physical page by setting its associated logical page to FFFFh.

You might unmap an entire set of mapped pages, for example, before loading and executing a program. Doing so ensures the loaded program, if it accesses expanded memory, won't access the pages your program has mapped. However, you must save the mapping context before you unmap the physical pages. This enables you to restore it later so you can access the memory you mapped there. To save the mapping context, use Function 8, 15, or 16. To restore the mapping context, use Function 9, 15, or 16.

The handle determines what type of pages are being mapped. Logical pages allocated by Function 4 and Function 27 (Allocate Standard Pages subfunction) are referred to as pages and are 16K bytes long. Logical pages allocated by Function 27 (Allocate Raw Pages subfunction) are referred to as raw pages and might not be the same size as logical pages.

CALLING PARAMETERS

AH = 44h

Contains the Map Handle Page function.

AL = physical_page_number
Contains the number of the physical page into which the logical page number is to be mapped. Physical pages are numbered zero relative.

BX = logical_page_number
Contains the number of the logical page to be mapped at the physical page within the page frame. Logical pages are numbered zero relative. The logical page must be in the range zero through (number of pages allocated to the EMM handle - 1). However, if BX contains logical page number FFFFh, the physical page specified in AL will be unmapped (be made inaccessible for reading or writing).

DX = emm_handle
Contains the EMM handle your program received from Function 4 (Allocate Pages).

REGISTERS MODIFIED

20

AX

STATUS

AH = 0 SUCCESSFUL
The manager has mapped the page. The page is ready to be accessed.

AH = 80h NON-RECOVERABLE
The manager detected a malfunction in the memory manager software.

AH = 81h NON-RECOVERABLE
The manager detected a malfunction in the expanded memory hardware.

AH = 83h NON-RECOVERABLE
The memory manager couldn't find the EMM handle your program specified.

AH = 84h NON-RECOVERABLE
The function code passed to the memory manager isn't defined.

AH = 8Ah RECOVERABLE

The logical page is out of the range of logical pages which are allocated to the EMM handle. This status is also returned if a program attempts map a logical page when no logical pages are allocated to the handle.

AH = 8Bh RECOVERABLE

The physical page number is out of the range of allowable physical pages. The program can recover by attempting to map into memory at a physical page which is within the range of allowable physical pages.

EXAMPLE

```
emm_handle              DW  ?
logical_page_number       DW  ?
physical_page_number       DB  ?

MOV  DX, emm_handle          ; load EMM handle
MOV  BX, logical_page_number   ; load logical page number
MOV  AL, physical_page_number  ; load physical page number
MOV  AH, 44h                 ; load function code
INT  67h                   ; call the memory manager
OR   AH, AH                  ; check EMM status
JNZ  emm_err_handler         ; jump to error handler on error
```


FUNCTION 6 DEALLOCATE PAGES

PURPOSE

Deallocate Pages deallocates the logical pages currently allocated to an EMM handle. Only after the application deallocates these pages can other applications use them. When a handle is deallocated, it name is set to all ASCII nulls (binary zeros).

Note.....

A program must perform this function before it exits to DOS. If it doesn't, no other programs can use these pages or the EMM handle. This means that a program using expanded memory should trap critical errors and control-break if there is a chance that the program will have allocated pages when either of these events occur.

Calling Parameters

AH = 45h

Contains the Deallocate Pages function.

DX = handle

Contains the EMM handle returned by Function 4 (Allocate Pages).

REGISTERS MODIFIED

AX

STATUS

AH = 0 SUCCESSFUL

The manager has deallocated the pages previously allocated to the EMM handle.

AH = 80h NON-RECOVERABLE

The manager detected a malfunction in the memory manager software.

AH = 81h NON-RECOVERABLE

The manager detected a malfunction in the expanded memory hardware.

AH = 83h NON-RECOVERABLE

The manager couldn't find the specified EMM handle.

AH = 84h NON-RECOVERABLE

The function code passed to the memory manager is not defined.

AH = 86h RECOVERABLE

The memory manager detected a save or restore page mapping context error (Function 8 or 9). There is a page mapping register state in the save area for the specified EMM handle. Save Page Map (Function 8) placed it there and a subsequent Restore Page Map (Function 9) has not removed

it. If you have saved the mapping context, you must restore it before you deallocate the EMM handle's pages.

EXAMPLE

```
emm_handle                DW  ?
```

```
MOV  DX, emm_handle          ; load EMM handle
MOV  AH, 45h                 ; load function code
INT  67h                     ; call the memory manager
OR   AH, AH                  ; check EMM status
JNZ  emm_err_handler         ; jump to error handler on error
```


FUNCTION 7 GET VERSION

PURPOSE

The Get Version function returns the version number of the memory manager software.

CALLING PARAMETERS

AH = 46h

Contains the Get Version function.

RESULTS

These results are valid only if the status returned is zero.

AL = version number

Contains the memory manager's version number in binary coded decimal (BCD) format. The upper four bits contain the integer digit of the version number. The lower four bits contain the fractional digit of version number. For example, version 4.0 is represented like this:

```
0100 0000
 /  \                    4 . 0
```

When checking for a version number, an application should check for a version number or greater. Vendors may use the fractional digit to indicate enhancements or corrections to their memory managers. Therefore, to allow for future versions of memory managers, an application shouldn't depend on an exact version number.

REGISTERS MODIFIED

AX

STATUS

AH = 0 SUCCESSFUL

The manager is present in the system and the hardware is working correctly.

AH = 80h NON-RECOVERABLE

The manager detected a malfunction in the memory manager software.

AH = 81h NON-RECOVERABLE

The manager detected a malfunction in the expanded memory hardware.

AH = 84h NON-RECOVERABLE

The function code passed to the memory manager is not defined.

EXAMPLE

24

```
emm_version                 DB  ?

MOV  AH, 46h                 ; load function code
INT  67h                   ; call the memory manager
OR   AH, AH                  ; check EMM status
JNZ  emm_err_handler           ; jump to error handler on error
MOV  emm_version, AL           ; save version number
```

25

FUNCTION 8 SAVE PAGE MAP

PURPOSE

Save Page Map saves the contents of the page mapping registers on all expanded memory boards in an internal save area. The function is typically used to save the memory mapping context of the EMM handle that was active when a software or hardware interrupt occurred. (See Function 9, Restore Page Map, for the restore operation.) If you're writing a resident program, an interrupt service, or a device driver that uses expanded memory, you must save the state of the mapping hardware. You must save this state because application software using expanded memory may be running when your program is invoked by a hardware interrupt, a software interrupt, or DOS.

The Save Page Map function requires the EMM handle that was assigned to your resident program, interrupt service routine, or device driver at the time it was initialized. This is not the EMM handle that the application software was using when your software interrupted it.

The Save Page Map function saves the state of the map registers for only the 64K-byte page frame defined in versions 3.x of this specification. Since all applications written to LIM versions 3.x require saving the map register state of only this 64K-byte page frame, saving the entire mapping state for a large number of mappable pages would be inefficient use of memory. Applications that use a mappable memory region outside the LIM 3.x page frame should use Function 15 or 16 to save and restore the state of the map registers.

CALLING PARAMETERS

AH = 47h
Contains the Save Page Map function.

DX = handle
Contains the EMM handle assigned to the interrupt service routine that's servicing the software or hardware interrupt. The interrupt service routine needs to save the state of the page mapping hardware before mapping any pages.

REGISTERS MODIFIED

AX

STATUS

AH = 0 SUCCESSFUL
The manager has saved the state of the page mapping hardware.

AH = 80h NON-RECOVERABLE
The manager detected a malfunction in the memory manager software.

AH = 81h NON-RECOVERABLE
The manager detected a malfunction in the expanded memory hardware.

AH = 83h NON-RECOVERABLE
The memory manager couldn't find the EMM handle your program specified.

AH = 84h NON-RECOVERABLE
The function code passed to the memory manager is not defined.

AH = 8Ch NON-RECOVERABLE
There is no room in the save area to store the state of the page mapping registers. The state of the map registers has not been saved.

AH = 8Dh CONDITIONALLY-RECOVERABLE
The save area already contains the page mapping register state for the EMM handle your program specified.

EXAMPLE

```
emm_handle                  DW  ?

MOV  DX, emm_handle             ; load EMM handle
MOV  AH, 47h                    ; load function code
INT  67h                    ; call the memory manager
OR   AH, AH                     ; check EMM status
JNZ  emm_err_handler            ; jump to error handler on error
```


FUNCTION 9 RESTORE PAGE MAP

PURPOSE

The Restore Page Map function restores the page mapping register contents on the expanded memory boards for a particular EMM handle. This function lets your program restore the contents of the mapping registers its EMM handle saved. (See Function 8, Save Page Map for the save operation.)

If you're writing a resident program, an interrupt service routine, or a device driver that uses expanded memory, you must restore the mapping hardware to the state it was in before your program took over. You must save this state because application software using expanded memory might have been running when your program was invoked.

The Restore Page Map function requires the EMM handle that was assigned to your resident program, interrupt service routine, or device driver at the time it was initialized. This is not the EMM handle that the application software was using when your software interrupted it.

The Restore Page Map function restores the state of the map registers for only the 64K-byte page frame defined in versions 3.x of this specification. Since all applications written to LIM versions 3.x require restoring the map register state of only this 64K-byte page frame, restoring the entire mapping state for a large number of mappable pages would be inefficient use of memory. Applications that use a mappable memory region outside the LIM 3.x page frame should use Function 15 or 16 to save and restore the state of the map registers.

CALLING PARAMETERS

AH = 48h
Contains the Restore Page Map function.

DX = emm_handle
Contains the EMM handle assigned to the interrupt service routine that's servicing the software or hardware interrupt. The interrupt service routine needs to restore the state of the page mapping hardware.

REGISTERS MODIFIED

AX

STATUS

AH = 0 SUCCESSFUL
The manager has restored the state of the page mapping registers.

AH = 80h NON-RECOVERABLE
The manager detected a malfunction in the memory manager software.

AH = 81h NON-RECOVERABLE
The manager detected a malfunction in the expanded memory hardware.

AH = 83h NON-RECOVERABLE

The memory manager couldn't find the EMM handle your program specified.

AH = 84h NON-RECOVERABLE

The function code passed to the memory manager is not defined.

AH = 8Eh CONDITIONALLY-RECOVERABLE

There is no page mapping register state in the save area for the specified EMM handle. Your program didn't save the contents of the page mapping hardware, so Restore Page Map can't restore it.

EXAMPLE

```
emm_handle                  DW  ?

MOV  DX, emm_handle             ; load EMM handle
MOV  AH, 48h                    ; load function code
INT  67h                    ; call the memory manager
OR   AH, AH                     ; check EMM status
JNZ  emm_err_handler            ; jump to error handler on error
```

29

FUNCTION 10 RESERVED

In earlier versions of the Lotus/Intel/Microsoft Expanded Memory Specification the use of this function was documented. This function is now documented as "RESERVED" (i.e. not documented) in the public versions of the specification and new programs should not use it.

Existing programs that use this function may still work correctly if the hardware is capable of supporting them. Since IBM Microchannel hardware is not capable of supporting this function, it cannot be implemented in the IBM Microchannel architecture. However, programs that use Functions 16 through 30 in Version 4.0 of this specification must not use Functions 10 and 11. These functions won't work correctly if a program attempts to mix the use of the new functions (Functions 16 through 30) and functions 10 and 11. Functions 10 and 11 are specific to the hardware on Intel expanded memory boards and will not work correctly on all vendors expanded memory boards.

30

FUNCTION 11 RESERVED

In earlier versions of the Lotus/Intel/Microsoft Expanded Memory Specification the use of this function was documented. This function is now documented as "RESERVED" (i.e. not documented) in the public versions of the specification and new programs should not use it.

Existing programs that use this function may still work correctly if the hardware is capable of supporting them. Since IBM Microchannel hardware is not capable of supporting this function, it cannot be implemented in the IBM Microchannel architecture. However, programs that use Functions 16 through 30 in Version 4.0 of this specification must not use Functions 10 and 11. These functions won't work correctly if a program attempts to mix the use of the new functions (Functions 16 through 30) and functions 10 and 11. Functions 10 and 11 are specific to the hardware on Intel expanded memory boards and will not work correctly on all vendors expanded memory boards.

31

FUNCTION 12 GET HANDLE COUNT

PURPOSE

The Get Handle Count function returns the number of open EMM handles (including the operating system handle 0) in the system.

CALLING PARAMETERS

AH = 4Bh
Contains the Get Handle Count function.

RESULTS

These results are valid only if the status returned is zero.

BX = total_open_emm_handles
Contains the number of open EMM handles [including the operating system handle (0)]. This number will not exceed 255.

REGISTERS MODIFIED

AX, BX

STATUS

AH = 0 SUCCESSFUL
The manager has returned the number of active EMM handles.

AH = 80h NON-RECOVERABLE
The manager detected a malfunction in the memory manager software.

AH = 81h NON-RECOVERABLE
The manager detected a malfunction in the expanded memory hardware.

AH = 84h NON-RECOVERABLE
The function code passed to the memory manager is not defined.

EXAMPLE

```
total_open_emm_handles          DW  ?

MOV  AH, 4Bh                    ; load function code
INT  67h                        ; call the memory manager
OR   AH, AH                     ; check EMM status
JNZ  emm_err_handler            ; jump to error handler on error
MOV  total_open_emm_handles, BX    ; save total active handle count
```


FUNCTION 13 GET HANDLE PAGES

PURPOSE

The Get Handle Pages function returns the number of pages allocated to a specific EMM handle.

CALLING PARAMETERS

AH = 4Ch
Contains the Get Handle Pages function.

DX = emm_handle
Contains the EMM handle.

RESULTS

These results are valid only if the status returned is zero.

BX = num_pages_alloc_to_emm_handle

Contains the number of logical pages allocated to the specified EMM handle. This number never exceeds 2048 because the memory manager allows a maximum of 2048 pages (32M bytes) of expanded memory.

REGISTERS MODIFIED

AX, BX

STATUS

AH = 0 SUCCESSFUL

The manager has returned the number of pages allocated to the EMM handle.

AH = 80h NON-RECOVERABLE

The manager detected a malfunction in the memory manager software.

AH = 81h NON-RECOVERABLE

The manager detected a malfunction in the expanded memory hardware.

AH = 83h NON-RECOVERABLE

The memory manager couldn't find the EMM handle your program specified.

AH = 84h NON-RECOVERABLE

The function code passed to the memory manager is not defined.

EXAMPLE

```
emm_handle                  DW  ?
pages_alloc_to_handle       DW  ?

MOV  DX, emm_handle             ; load EMM handle
MOV  AH, 4Ch                    ; load function code
INT  67h                        ; call the memory manager
OR   AH, AH                     ; check EMM status
JNZ  emm_err_handler            ; jump to error handler on error
MOV  pages_alloc_to_handle, BX  ; save number of pages allocated
                                ; to specified handle
```

FUNCTION 14 GET ALL HANDLES PAGES

PURPOSE

The Get All Handle Pages function returns an array of the open emm handles and the number of pages allocated to each one.

CALLING PARAMETERS

AH = 4Dh

Contains the Get All Handle Pages function.

```
handle_page_struct        STRUC
  emm_handle              DW ?
  pages_alloc_to_handle   DW ?
handle_page_struct        ENDS
```

ES:DI = pointer to handle_page

Contains a pointer to an array of structures where a copy of all open EMM handles and the number of pages allocated to each will be stored. Each structure has these two members:

.emm_handle

The first member is a word which contains the value of the open EMM handle. The values of the handles this function returns will be in the range of 0 to 255 decimal (0000h to 00FFh). The uppermost byte of the handle is always zero.

.pages_alloc_to_handle

The second member is a word which contains the number of pages allocated to the open EMM handle.

RESULTS

These results are valid only if the status returned is zero.

BX = total_open_emm_handles

Contains the number of open EMM handles (including the operating system handle [0]). The number cannot be zero because the operating system handle is always active and cannot be deallocated. This number will not exceed 255.

REGISTERS MODIFIED

AX, BX

STATUS

AH = 0 SUCCESSFUL

The manager has returned the array.

AH = 80h NON-RECOVERABLE

The manager detected a malfunction in the memory manager software.

AH = 81h NON-RECOVERABLE

The manager detected a malfunction in the expanded memory hardware.

AH = 84h NON-RECOVERABLE

The function code passed to the memory manager isn't defined.

EXAMPLE

```
handle_page                 handle_page_struct 255 DUP (?)
total_open_handles            DW  ?

MOV  AX, SEG handle_page
MOV  ES, AX
LEA  DI, handle_page          ; ES:DI points to handle_page
MOV  AH, 4Dh                  ; load function code
INT  67h                    ; call the memory manager
OR   AH, AH                   ; check EMM status
JNZ  emm_err_handler          ; jump to error handler on error
MOV  total_open_handles, BX   ; save total open handle count
```


FUNCTION 15 GET/SET PAGE MAP

GET PAGE MAP SUBFUNCTION

PURPOSE

The Get Page Map subfunction saves the mapping context for all mappable memory regions (conventional and expanded) by copying the contents of the mapping registers from each expanded memory board to a destination array. The application must pass a pointer to the destination array. This subfunction doesn't require an EMM handle. Use this function instead of Functions 8 and 9 if you need to save or restore the mapping context but don't want (or have) to use a handle.

CALLING PARAMETERS

AX = 4E00h

Contains the Get Page Map subfunction.

ES:DI = dest_page_map

Contains a pointer to the destination array address in segment:offset format. Use the Get Size of Page Map Save Array subfunction to determine the size of the desired array.

RESULTS

These results are valid only if the status returned is zero.

dest_page_map

The array contains the state of all the mapping registers on all boards in the system. It also contains any additional information necessary to restore the boards to their original state when the program invokes a Set subfunction.

Pages 17

Find a page or section...

Home

All about the x86 reset vector

Compatibility

Documentation references for old PCs

DPMI 0.9 Specs

LIM 4.0 Specs (EMS)

MS-DOS 2.0 Device Driver Specs

Proposition for better code generation from execution trace

Q95555: Overview of Memory-Management Functionality in MS-DOS

Spice86 internal debugger

Spice86 v10 release notes

Spice86 v11 release notes

Spice86 v11.1 release notes

Spice86 v12 release notes

VESA VBE 1.2 specs

Show 2 more pages...

Clone this wiki locally

```
https://github.com/OpenRakis/Spice86.wiki.git
```